```python
# -*- coding: utf-8 -*-
# 起動時に不正なクォート文字を自動修正
import os as _os, sys as _sys
def _fix_quotes():
    _path = __file__
    with open(_path, 'r', encoding='utf-8') as _f:
        _s = _f.read()
    _bad = {chr(0x2018): chr(39), chr(0x2019): chr(39),
            chr(0x201c): chr(34), chr(0x201d): chr(34)}
    _fixed = False
    for _b, _g in _bad.items():
        if _b in _s:
            _s = _s.replace(_b, _g); _fixed = True
    if _fixed:
        with open(_path, 'w', encoding='utf-8') as _f:
            _f.write(_s)
        print('Quote chars fixed. Restarting...')
        _os.execv(_sys.executable, [_sys.executable] + _sys.argv)
_fix_quotes()
del _fix_quotes, _os, _sys

from nicegui import ui
import random, math, copy, json

# ============================================================
# 定数・設定
# ============================================================
APP_TITLE    = 'The Gaffer'
MAX_SQUAD    = 25
MIN_YOUTH    = 10
MAX_YOUTH    = 15
SEASON_WEEKS = 36
CUP_START    = 10
CC_GROUP_START = 15   # コンチネンタルカップ グループ開始週
CC_KO_START    = 25   # コンチネンタルカップ 決勝T開始週
CC_PRIZE = {          # CL賞金
    'group': 80000,   # グループステージ突破
    'quarterfinal': 200000,
    'semifinal': 400000,
    'final': 800000,
    'winner': 1500000,
}
TRANSFER_WINDOWS = [(1,6),(19,22)]
SALARY_CAP   = {'D1':420000,'D2':270000,'D3':165000}
```

```
COUNTRIES = ['England','Spain','Germany','Italy','France','Brazil','Japan']
POSITIONS = ['GK','DF','MF','FW']
LEAGUE_STRENGTH = {'England':92,'Spain':90,'Germany':88,'Italy':87,'France':86,'B

PLAY_STYLES = {
    'FW':['ゴールハンター','ポーチャー','ターゲットマン','カウンターランナー','トリックスター
          'ファルソ・ヌエベ','プレッシングFW','ドリブラー','セカンドストライカー','ウィンガー
    'MF':['プレイメーカー','ボックス・トゥ・ボックス','アンカー','チャンスクリエイター',
          'シャドーストライカー','ディープライイングMF','メッツァーラ','ハーフスペース番人',
          'プレスハウンド','テクニカルドリブラー','セットプレースペシャリスト'],
    'DF':['タックルマスター','リベロ','ビルドアップ型','空中戦マスター',
          'アグレッシブDFB','カバーシャドウ','インバーテッドWB','マンマーカー',
          'ラインコントローラー','スウィーパー'],
    'GK':['ショットストッパー','スイーパーキーパー','コマンダー','ディストリビューター',
          'ペナルティセーバー','リフレックスGK'],
}
PERSONALITIES = [
    '真面目','野心家','ムードメーカー','気分屋','忠誠心高い','努力家',
    'スター気質','短気','チームプレイヤー','負けず嫌い','クレバー',
    'サポーター思い','プロフェッショナル','自由奔放','リーダー気質',
    'ハードワーカー','繊細','お調子者'
]
YOUTH_POLICIES = {
    'バランス':{'SPD':1.00,'TEC':1.00,'PHY':1.00},
    'スピード':{'SPD':1.15,'TEC':0.95,'PHY':1.00},
    'テクニック':{'SPD':0.95,'TEC':1.15,'PHY':0.95},
    'フィジカル':{'SPD':0.95,'TEC':0.95,'PHY':1.15},
}
TACTICS = {
    'バランス':{'atk':1.00,'def':1.00},
    '攻撃的':{'atk':1.12,'def':0.90},
    '守備的':{'atk':0.90,'def':1.12},
    'プレス':{'atk':1.06,'def':1.06},
    'カウンター':{'atk':1.08,'def':1.04},
}
FORMATIONS = {
    '4-3-3':{'GK':1,'DF':4,'MF':3,'FW':3},
    '4-4-2':{'GK':1,'DF':4,'MF':4,'FW':2},
    '3-5-2':{'GK':1,'DF':3,'MF':5,'FW':2},
    '5-3-2':{'GK':1,'DF':5,'MF':3,'FW':2},
    '4-2-3-1':{'GK':1,'DF':4,'MF':5,'FW':1},
}
MANAGER_SKILLS = {
    '戦術家':{'desc':'戦術補正+10%','req_exp':30},
    '育成型':{'desc':'ユース成長+15%','req_exp':50},
    'モチベーター':{'desc':'morale+5/週','req_exp':40},
```

```
    'スカウト眼':{'desc':'スカウット質+3','req_exp':35},
    '財務手腕':{'desc':'スポンサー+10%','req_exp':60},
    '鉄のメンタル':{'desc':'評価下落-50%','req_exp':45},
}
PLAYER_STATES = {
    'normal':{'label':'通常','goal_mult':1.00,'assist_mult':1.00},
    'hot':{'label':'🔥覚醒','goal_mult':1.40,'assist_mult':1.30},
    'slump':{'label':'📉スランプ','goal_mult':0.65,'assist_mult':0.75},
}
CAPTAIN_EFFECTS = {
    'ムードメーカー':{'locker_room':5,'morale':3},
    '真面目':{'locker_room':2,'morale':2},
    '忠誠心高い':{'locker_room':3,'fan_happiness':2},
    'チームプレイヤー':{'locker_room':4,'morale':2},
    'スター気質':{'club_brand':3,'fan_base':100},
    '短気':{'locker_room':-2,'morale':-1},
}
WEATHER = {
    '晴れ':{'atk_mod':1.00,'def_mod':1.00,'injury_mod':1.00,'att_mod':1.05},
    '曇り':{'atk_mod':1.00,'def_mod':1.00,'injury_mod':1.00,'att_mod':1.00},
    '雨':{'atk_mod':0.93,'def_mod':1.03,'injury_mod':1.15,'att_mod':0.90},
    '強風':{'atk_mod':0.88,'def_mod':1.05,'injury_mod':1.10,'att_mod':0.85},
    '雪':{'atk_mod':0.80,'def_mod':1.08,'injury_mod':1.25,'att_mod':0.75},
}
# ランダムイベント定義
RANDOM_EVENTS = [
    {'id':'scandal','text':'{name}がスキャンダル報道！','choices':['謝罪声明を出す','沈
    {'id':'rival_poach','text':'ライバル{rival}が{name}の引き抜きを企てている','choices
    {'id':'ob_coach','text':'クラブOB {name}がコーチ就任を申し出ている','choices':['採用
    {'id':'local_hero','text':'地元出身の{name}がスタメン定着！ファン熱狂','choices':['
    {'id':'anniversary','text':'クラブ創立記念日！特別試合を開催','choices':['盛大に祝う
    {'id':'youth_bigclub','text':'ビッグクラブが育成選手{name}を狙っている','choices':[
    {'id':'intl_return_boost','text':'代表から帰還した{name}が一回り成長した','choices
    {'id':'media_praise','text':'メディアが監督の采配を絶賛！','choices':['コメントする'
    {'id':'training_accident','text':'トレーニング中に{name}が軽傷','choices':['安静に
    {'id':'fan_protest','text':'ファンが成績に不満でデモ！','choices':['謝罪する','強気
]

ACHIEVEMENTS = {
    'first_win':{'name':'初勝利','desc':'リーグ戦で初勝利','icon':'⚽'},
    'first_title':{'name':'初優勝','desc':'リーグ優勝','icon':'🏆'},
    'cup_winner':{'name':'カップ制覇','desc':'国内カップ優勝','icon':'🥇'},
    'unbeaten_5':{'name':'5連勝','desc':'5試合連続勝利','icon':'🔥'},
    'youth_promoted':{'name':'育成の鬼','desc':'ユース昇格5人達成','icon':'🌱'},
    'season3':{'name':'3年生存','desc':'3シーズン続投','icon':'📅'},
    'cc_winner':{'name':'国際制覇','desc':'国際大会優勝','icon':'🌍'},
    'promoted':{'name':'昇格！','desc':'ディビジョン昇格','icon':'🔺'},
```

```
    'rich_club':{'name':'大富豪','desc':'予算$5,000,000超','icon':'💰'},
    'wonder_scout':{'name':'名スカウト','desc':'ウォンダーキッド発見3人','icon':'🌟'},
    'max_facility':{'name':'最高施設','desc':'全施設Lv5達成','icon':'🏗️'},
    'career_move':{'name':'渡り鳥監督','desc':'別クラブへ転身','icon':'✈️'},
    'pk_hero':{'name':'PKの英雄','desc':'PKシューターアウトを制した','icon':'🥅'},
}

COUNTRY_NAME_POOLS = {
    'England':{'first':['Jack','Noah','Oliver','Harry','George','Leo','James','Ch
        'Ethan','Mason','Liam','William','Henry','Edward','Thomas','Samuel','Arch
        'Theo','Rory','Elliot','Callum','Ryan','Dylan','Jake','Kyle','Aiden','Lew
    'last':['Smith','Brown','Taylor','Johnson','Walker','Wilson','Davis','Miller'
        'White','Moore','Martin','Thompson','Young','Harris','Lewis','Robinson','
        'Wright','Scott','Green','Baker','Adams','Nelson','Carter','Mitchell','Tu
    'clubs':['London','Manchester','Merseyside','Northbridge','Royal','River','Ea
        'Irongate','Silverport','Blackmoor','Goldfield','Ashford','Cromwell','Har
    'Spain':{'first':['Pablo','Alvaro','Hugo','Diego','Javier','Adrian','Sergio',
        'Carlos','Miguel','Roberto','Antonio','Fernando','Alejandro','Eduardo','R
        'Tomas','Daniel','Jesus','Francisco','Lorenzo','Santiago','Marcos','Nicol
    'last':['Garcia','Lopez','Martinez','Sanchez','Perez','Gomez','Diaz','Torres'
        'Jimenez','Hernandez','Morales','Molina','Delgado','Castro','Ortega','Veg
        'Iglesias','Vargas','Ramos','Suarez','Blanco','Aguilar','Santos','Medina'
    'clubs':['Madrid','Catalunya','Valencia','Sevilla','Bilbao','Costa','Atletico
        'Burgos','Toledo','Almeria','Cordoba','Alicante','Zaragoza','Valladolid',
    'Germany':{'first':['Lukas','Leon','Felix','Max','Finn','Jonas','Noah','Paul'
        'Julian','Tobias','Sebastian','Christian','Dominik','Florian','Stefan','M
        'Niklas','Philipp','Simon','Jan','Lars','Sven','Markus','Dennis','Fabian'
    'last':['Muller','Schmidt','Schneider','Fischer','Weber','Meyer','Wagner','Be
        'Richter','Klein','Wolf','Schroeder','Neumann','Schwarz','Zimmermann','Br
        'Lange','Werner','Lehmann','Kraus','Huber','Frank','Berger','Keller','Rot
    'clubs':['Berlin','Munich','Rhein','Nord','Union','Eintracht','Bayern','Ruhr'
        'Frankfurt','Cologne','Dortmund','Leipzig','Bremen','Hannover','Nurnberg'
    'Italy':{'first':['Luca','Marco','Matteo','Davide','Andrea','Federico','Stefa
        'Alessandro','Riccardo','Filippo','Lorenzo','Giovanni','Nicola','Roberto'
        'Daniele','Paolo','Francesco','Luigi','Emanuele','Valentino','Cristiano',
    'last':['Rossi','Russo','Ferrari','Esposito','Romano','Gallo','Costa','Fontan
        'Ricci','Marino','Bruno','De Luca','Colombo','Mancini','Lombardi','Barbie
        'Caruso','Fiore','Serra','Leone','Gentile','Pellegrini','Cattaneo','Amato
    'clubs':['Milano','Roma','Torino','Napoli','Lazio','Verona','Atletico','Inter
        'Bologna','Firenze','Genova','Palermo','Venezia','Bari','Parma','Cagliari
    'France':{'first':['Lucas','Nathan','Hugo','Leo','Gabriel','Louis','Enzo','Ar
        'Mathis','Antoine','Baptiste','Maxime','Romain','Nicolas','Pierre','Alexa
        'Florian','Sebastien','Quentin','Valentin','Axel','Dylan','Kevin','Etienn
    'last':['Martin','Bernard','Dubois','Thomas','Robert','Richard','Petit','Dura
        'Simon','Laurent','Lefebvre','Michel','Garcia','David','Bertrand','Roux',
        'Morin','Girard','Andre','Mercier','Rousseau','Blanc','Guerin','Faure','G
    'clubs':['Paris','Lyon','Marseille','Monaco','Nord','Bleu','Rouge','Avenir','
```

```
        'Bordeaux','Nantes','Strasbourg','Toulouse','Rennes','Montpellier','Nice'
    'Brazil':{'first':['Lucas','Joao','Mateus','Gabriel','Rafael','Pedro','Thiago
        'Gustavo','Henrique','Leandro','Rodrigo','Vinicius','Marcelo','Fabio','Ca
        'Alexandre','Murilo','Caio','Victor','Leonardo','Eduardo','Alan','Patrick
    'last':['Silva','Souza','Costa','Santos','Oliveira','Pereira','Almeida','Gome
        'Carvalho','Ribeiro','Fernandes','Araujo','Martins','Nascimento','Barbosa
        'Moreira','Correia','Pinto','Azevedo','Teixeira','Cunha','Lopes','Monteir
    'clubs':['Rio','Samba','Verde','Cruzeiro','Porto','Atletico','Nova','Sol','Sa
        'Flamengo','Palmeiras','Santos','Gremio','Fluminense','Botafogo','Corinth
    'Japan':{'first':['Haruto','Ren','Yuto','Sota','Kaito','Riku','Takumi','Daiki
        'Kenta','Ryota','Naoki','Kenji','Hiroshi','Masaki','Toshiki','Yusei','Aki
        'Shinya','Tatsuya','Daisuke','Kohei','Ryusei','Hayato','Sora','Itsuki','H
    'last':['Sato','Suzuki','Takahashi','Tanaka','Ito','Yamamoto','Watanabe','Nak
        'Yoshida','Yamada','Sasaki','Yamaguchi','Matsumoto','Inoue','Kimura','Hay
        'Mori','Abe','Ikeda','Hashimoto','Ishikawa','Maeda','Fujita','Ogawa','Got
    'clubs':['Blue','Red','Phoenix','Ocean','North','East','West','South','Rising
        'Thunder','Spirit','Galaxy','Frontier','Brave','Glory','Force','Wings','S
}
STAFF_TYPES=['ヘッドコーチ','スカウト','フィジオ','アシスタント']
FACILITY_TYPES=['youth','training','medical','scouting','stadium']
BASE_ATTRS={
    'GK':{'SPD':45,'TEC':52,'PAS':50,'PHY':62,'SHT':30,'DEF':72,'MEN':60,'STA':75
    'DF':{'SPD':56,'TEC':53,'PAS':54,'PHY':66,'SHT':38,'DEF':68,'MEN':58,'STA':72
    'MF':{'SPD':60,'TEC':63,'PAS':65,'PHY':58,'SHT':52,'DEF':52,'MEN':60,'STA':70
    'FW':{'SPD':67,'TEC':61,'PAS':54,'PHY':62,'SHT':68,'DEF':35,'MEN':62,'STA':68
}
OVR_WEIGHTS={
    'GK':{'SPD':0.05,'TEC':0.10,'PAS':0.08,'PHY':0.15,'SHT':0.02,'DEF':0.35,'MEN'
    'DF':{'SPD':0.10,'TEC':0.12,'PAS':0.12,'PHY':0.18,'SHT':0.05,'DEF':0.25,'MEN'
    'MF':{'SPD':0.12,'TEC':0.18,'PAS':0.20,'PHY':0.12,'SHT':0.12,'DEF':0.10,'MEN'
    'FW':{'SPD':0.15,'TEC':0.15,'PAS':0.10,'PHY':0.12,'SHT':0.28,'DEF':0.03,'MEN'
}
ATTR_ICONS={'SPD':'💨','TEC':'⚽','PAS':'🎯','PHY':'💪','SHT':'🔫','DEF':'🛡️','M
ATTR_NAMES={'SPD':'スピード','TEC':'テクニック','PAS':'パス','PHY':'フィジカル','SHT':'
TRAIT_DESC={
    'なし':'特になし',
    '勝負師':'重要局面でゴール確率+12%',
    'ムードメーカー':'チームmorale+8/節',
    'スペ体質':'怪我リスク×1.5',
    '大舞台男':'カップ戦・ダービーで能力+5%',
    '技巧派':'TEC・PAS属性の成長率+20%',
    '鉄人':'疲労回復速度+30%',
    '速攻屋':'カウンター戦術時にSPD補正+8',
    'キャプテン体質':'キャプテン時のケミストリー+5',
    '若手の星':'23歳以下の時の成長率+25%',
    '経験値豊富':'30歳以降の能力低下-50%',
    '熱血漢':'ホームゲームで能力+4%',
```

```
    '冷静沈着':'アウェイゲームで能力+4%',
    '孤高の天才':'個人スタッツ+15%、チームケミストリー-3',
    'チームの心臓':'在籍中のチームmorale底上げ+5',
    'マルチロール':'コンバート時のペナルティ-50%',
    'ラストミニッツ':'後半終盤（75分以降）の得点確率+20%',
    'プレッシャー耐性':'重要試合での能力低下なし',
    '怪物':'全属性の成長上限+3',
    'サイレントリーダー':'表には出ないが周囲の不満-10',
}

game_state=None
nav_state={'tab':'dashboard','live_mode':False,'live_events':[]}
dashboard_box=status_box=None

# ============================================================
# ユーティリティ
# ============================================================
def clamp(v,lo,hi): return max(lo,min(v,hi))
def avg(vals): return sum(vals)/len(vals) if vals else 0
def wc(items,weights): return random.choices(items,weights=weights,k=1)[0]
def fmt_m(v): return f'${int(v):,}'
def find_team(name):
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==name: return t
    return None
def get_sel(): return find_team(game_state['selected_club']) if game_state['selec
def all_players(youth=False):
    pl=[]
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            pl+=t['players']
            if youth: pl+=t['youth']
    return pl
def player_by_id(pid):
    for p in all_players(True):
        if p['id']==pid: return p
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            for p in t.get('loan_in',[]):
                if p['id']==pid: return p
    return None
def div_of(name):
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==name: return d
    return None
```

```
def sorted_table(div):
    def _key(t):
        s=t.get('season_stats',{'pts':0,'gd':0,'gf':0})
        return (s.get('pts',0),s.get('gd',0),s.get('gf',0),t.get('reputation',50)
    return sorted(game_state['divisions'][div],key=_key,reverse=True)
def add_news(txt,cat='general'):
    game_state['news'].insert(0,{'text':txt,'cat':cat,'season':game_state['season
    game_state['news']=game_state['news'][:200]
def add_fin(cat,amt,note=''):
    game_state.setdefault('finance_logs',[]).insert(0,{'season':game_state['seaso
    game_state['finance_logs']=game_state['finance_logs'][:200]
def add_sf(cat,amt):
    game_state['season_finance'][cat]=game_state['season_finance'].get(cat,0)+amt
def has_skill(s): return s in game_state.get('manager_skills',[])
def gain_exp(amt):
    game_state['manager_exp']=game_state.get('manager_exp',0)+amt
    pts=sum(1 for t in [50,120,220,350,500] if game_state['manager_exp']>=t)
    game_state['manager_skill_points']=max(0,pts-len(game_state.get('manager_skil
def in_transfer_window():
    w=game_state['week']
    return any(lo<=w<=hi for lo,hi in TRANSFER_WINDOWS)
def get_weather(): return game_state.get('current_weather','晴れ')
def roll_weather():
    w=wc(['晴れ','曇り','雨','強風','雪'],[40,25,20,10,5])
    game_state['current_weather']=w; return w
def check_achievement(key):
    done=game_state.setdefault('achievements',[])
    if key not in done and key in ACHIEVEMENTS:
        done.append(key); a=ACHIEVEMENTS[key]
        add_news(f'🎖️実績解除: {a["icon"]}{a["name"]} - {a["desc"]}','club')

# ============================================================
# 多言語対応
# ============================================================
LANG = 'ja'  # 'ja' or 'en'

TRANSLATIONS = {
    # ナビゲーション
    'ホーム':           {'ja':'ホーム',          'en':'Home'},
    'チーム':           {'ja':'チーム',           'en':'Squad'},
    '試合':             {'ja':'試合',             'en':'Matches'},
    '移籍':             {'ja':'移籍',             'en':'Transfers'},
    '経営':             {'ja':'経営',             'en':'Management'},
    # 試合進行
    '次の週へ進む':      {'ja':'次の週へ進む',      'en':'Next Week'},
    '5週一括進行':       {'ja':'5週一括進行',       'en':'Skip 5 Weeks'},
    '10週一括進行':      {'ja':'10週一括進行',      'en':'Skip 10 Weeks'},
```

```
'試合進行':          {'ja':'試合進行',          'en':'Match'},
'プレシーズン実施':   {'ja':'プレシーズン実施',   'en':'Pre-Season'},
# クラブ選択
'クラブを選択':       {'ja':'🏟 クラブを選択',   'en':'🏟 Select Club'},
'D1説明':            {'ja':'1部リーグ（最強）',  'en':'Division 1 (Top)'},
'D2説明':            {'ja':'2部リーグ',         'en':'Division 2'},
'D3説明':            {'ja':'3部リーグ（初心者向け）','en':'Division 3 (Beginner)'
# 目標
'優勝を狙う':         {'ja':'優勝を狙う',        'en':'Win the Title'},
'上位進出':           {'ja':'上位進出',          'en':'Top Half Finish'},
'残留で十分':         {'ja':'残留で十分',        'en':'Stay Up'},
'昇格を目指す':       {'ja':'昇格を目指す',      'en':'Win Promotion'},
# チュートリアル
'はじめに':           {'ja':'📖 はじめに',       'en':'📖 How to Play'},
'わかった！クラブを選ぶ':{'ja':'わかった！クラブを選ぶ','en':'Got it! Choose Club'},
'目標を宣言してください':{'ja':'📋今季目標を宣言してください','en':'📋 Set Season Goa
'目標未宣言警告':     {'ja':'目標を宣言しないとプレシーズンに進めません',
                      'en':'Set a goal before pre-season'},
# プレシーズン・トライアウト
'プレシーズン':       {'ja':'⚽プレシーズン',    'en':'⚽ Pre-Season'},
'プレシーズンボタン': {'ja':'プレシーズンを実施する（親善試合3試合）',
                      'en':'Start Pre-Season (3 Friendlies)'},
'トライアウト':       {'ja':'👀 トライアウト',   'en':'👀 Tryouts'},
'トライアウト開催':   {'ja':'トライアウトを開催する','en':'Hold Tryouts'},
'トライアウト終了':   {'ja':'トライアウト終了',   'en':'End Tryouts'},
'契約する':           {'ja':'契約する',          'en':'Sign'},
# 移籍
'FA市場':             {'ja':'🆓フリーエージェント市場','en':'🆓 Free Agents'},
'FA説明':             {'ja':'移籍金不要。ただし年俸要求は高め。',
                      'en':'No transfer fee. Higher wage demands.'},
'国内スカウト':       {'ja':'🔍国内スカウト市場', 'en':'🔍 Scout Market'},
'海外スカウト':       {'ja':'🌍海外スカウト派遣', 'en':'🌍 International Scout'},
# セーブ
'セーブロード':       {'ja':'💾セーブ / ロード', 'en':'💾 Save / Load'},
'セーブ':             {'ja':'💾セーブ',          'en':'💾 Save'},
# 順位表
'順位表':             {'ja':'順位表',            'en':'Table'},
'対戦カード':         {'ja':'節カード',          'en':'Fixtures'},
# ステータス
'移籍中':             {'ja':'🔒移籍',            'en':'🔒 Window'},
'移籍外':             {'ja':'🔓',                'en':'🔓'},
'監督解任危機':       {'ja':'🚨監督解任危機！',  'en':'🚨 Sack Warning!'},
'財政危機':           {'ja':'💸財政危機！',      'en':'💸 Financial Crisis!'},
# 評価テキスト
'理事会大絶賛':       {'ja':'理事会: 大絶賛',    'en':'Board: Delighted'},
'理事会満足':         {'ja':'理事会: 満足',      'en':'Board: Pleased'},
'理事会普通':         {'ja':'理事会: 普通',      'en':'Board: Okay'},
```

```
'理事会不満':          {'ja':'理事会: 不満',          'en':'Board: Unhappy'},
'理事会激怒':          {'ja':'理事会: 激怒⚠️',      'en':'Board: Furious⚠️'},
# 選手状態
'不満爆発':            {'ja':'😡 不満爆発',          'en':'😡 Furious'},
'かなり不満':          {'ja':'😠 かなり不満',        'en':'😠 Unhappy'},
'やや不満':            {'ja':'😟 やや不満',          'en':'😟 Unsettled'},
'普通':                {'ja':'😐 普通',              'en':'😐 Content'},
'満足':                {'ja':'😊 満足',              'en':'😊 Happy'},
'絶好調':              {'ja':'🔥 絶好調',            'en':'🔥 On Fire'},
'好調':                {'ja':'😊 好調',              'en':'😊 Good Form'},
'低調':                {'ja':'😟 低調',              'en':'😟 Poor Form'},
'最悪morale':          {'ja':'💔 最悪',              'en':'💔 Terrible'},
'まもなく復帰':        {'ja':'🏥 まもなく復帰',      'en':'🏥 Nearly Back'},
'復帰まで少し':        {'ja':'🏥 復帰まで少し',      'en':'🏥 Out Short Term'},
'長期離脱中':          {'ja':'🏥 長期離脱中',        'en':'🏥 Long Term Injury'},
'重傷長期':            {'ja':'🏥 重傷・長期',        'en':'🏥 Serious Injury'},
'元気':                {'ja':'元気',                 'en':'Fresh'},
'やや疲労':            {'ja':'やや疲労',             'en':'Tired'},
'疲労':                {'ja':'疲労',                 'en':'Fatigued'},
'かなり疲労':          {'ja':'かなり疲労',           'en':'Exhausted'},
# CL
'🏆コンチネンタルカップ':{'ja':'🌍🏆コンチネンタルカップ','en':'🌍 Continental Cup'
# 後半戦術
'後半戦術プリセット': {'ja':'⚙️ 後半戦術プリセット（任意）','en':'⚙️ 2nd Half Tactic
'未設定':              {'ja':'未設定（通常戦術で進行）','en':'Not set (use default
'解除':                {'ja':'解除',                 'en':'Clear'},
# 共通
'詳細':                {'ja':'詳細',                 'en':'Detail'},
'獲得':                {'ja':'獲得',                 'en':'Sign'},
'断る':                {'ja':'断る',                 'en':'Decline'},
'受け入れる':          {'ja':'受け入れる',           'en':'Accept'},
'閉じる':              {'ja':'閉じる',               'en':'Close'},
'未設定_short':        {'ja':'未設定',               'en':'Not set'},
'候補更新':            {'ja':'候補更新',             'en':'Refresh'},
'自動選出':            {'ja':'自動選出',             'en':'Auto Pick'},
'確定':                {'ja':'確定',                 'en':'Confirm'},
'延長':                {'ja':'延長',                 'en':'Renew'},
'売却':                {'ja':'売却',                 'en':'Sell'},
'ローン放出':          {'ja':'ローン放出',           'en':'Loan Out'},
'セーブ_btn':          {'ja':'💾セーブ',             'en':'💾 Save'},

# notify メッセージ
'予算不足':            {'ja':'予算不足',             'en':'Insufficient funds'},
'スクワッド満員':      {'ja':'スクワッド満員',       'en':'Squad full'},
'移籍ウィンドウ外':    {'ja':'移籍ウィンドウ外',     'en':'Transfer window closed'
'最低11人':            {'ja':'最低11人必要',         'en':'Need at least 11 play
'受け入れ先なし':      {'ja':'受け入れ先なし',       'en':'No destination availab
```

```
'最大Lv':              {'ja':'最大Lv',              'en':'Max level reached'},
'資金不足':            {'ja':'資金不足',            'en':'Not enough funds'},
'SPなし':              {'ja':'SPなし',              'en':'No skill points'},
'習得済み':            {'ja':'習得済み',            'en':'Already learned'},
'クラブ未選択':        {'ja':'クラブ未選択',        'en':'No club selected'},
'監督危機':            {'ja':'監督評価が危機的！',  'en':'Manager rating critic
'ロード完了':          {'ja':'📂ロード完了',        'en':'📂 Loaded'},
'トライアウト2名制限':{'ja':'トライアウトから獲得できるのは最大2名です','en':'Max 2 try
'まもなく復帰notify': {'ja':'まもなく復帰',        'en':'Almost back'},
'最低11人youth':       {'ja':'19歳以上は不可',      'en':'Must be under 19'},
'枠満員':              {'ja':'枠満員',              'en':'Academy full'},
'VIP最大':             {'ja':'VIP最大Lv3',          'en':'VIP max level 3'},
# ステータス表示
'位':                  {'ja':'{n}位',               'en':'#{n}'},
'ローン件数':          {'ja':'🏦ローン{n}件',       'en':'🏦{n} Loan(s)'},
'監督危機label':       {'ja':'🚨監督解任危機！',    'en':'🚨 Sack Warning!'},
'財政危機label':       {'ja':'💸財政危機！',        'en':'💸 Financial Crisis!
# クラブ選択
'D1=最上位':           {'ja':'D1=最上位 D2=2部 D3=3部','en':'D1=Top  D2=Second  D
'評判':                {'ja':'評判{n}',             'en':'Rep:{n}'},
# トライアウト
'トライアウト説明':    {'ja':'無名選手を試せるチャンス。能力は契約後に判明。スカウト施設Lv
                        'en':'Unknown players on trial. Stats revealed after si
'スカウト施設Lv':      {'ja':'スカウト施設 Lv{n}',  'en':'Scout Facility Lv{n}'
'トライアウト開催中': {'ja':'👀 トライアウト開催中（契約済{a}/2名 / 残{b}名）',
                        'en':'👀 Tryouts ({a}/2 signed / {b} remaining)'},
'スタイル':            {'ja':'スタイル:{n}',        'en':'Style:{n}'},
'特性label':           {'ja':'特性: {n}',           'en':'Trait: {n}'},
'サイン費':            {'ja':'サイン費: {a} / 年俸: {b}','en':'Sign fee: {a} / Wa
# プレシーズン
'今季目標label':       {'ja':'今季目標: {n}',       'en':'Season Goal: {n}'},
'未設定goal':          {'ja':'未設定',              'en':'Not set'},
# ダッシュボード各種
'イベント':            {'ja':'📌イベント: {n}',     'en':'📌 Event: {n}'},
'プレスカンファレンス':{'ja':'📰試合前プレスカンファレンス','en':'📰 Press Conference
'強気発言':            {'ja':'強気発言',            'en':'Bold Statement'},
'慎重姿勢':            {'ja':'慎重姿勢',            'en':'Cautious Approach'},
'選手を称える':        {'ja':'選手を称える',        'en':'Praise Players'},
'批判を受け入れる':    {'ja':'批判を受け入れる',    'en':'Accept Criticism'},
'監督オファー':        {'ja':'📩監督オファー',      'en':'📩 Manager Offer'},
'転身':                {'ja':'に転身',              'en':'→ Move to {n}'},
'全て断る':            {'ja':'全て断る',            'en':'Decline All'},
'ネーミングライツ':    {'ja':'💼ネーミングライツ打診',  'en':'💼 Naming Rights Offe
'買収オファー':        {'ja':'買収オファー',        'en':'Takeover Offer'},
'スポンサー交渉label':{'ja':'スポンサー交渉',      'en':'Sponsor Negotiation'}
'契約するbtn':         {'ja':'契約する',            'en':'Accept'},
'受け入れるbtn':       {'ja':'受け入れる',          'en':'Accept'},
```

```
'断るbtn':            {'ja':'断る',                    'en':'Decline'},
# ダッシュボード情報
'直近試合':           {'ja':'直近試合',                'en':'Last Match'},
'観客収入MOM':        {'ja':'観客{a}人 | {b} | MOM:{c}','en':'Att:{a} | {b} | M(
'天候label':          {'ja':'天候: {n}',               'en':'Weather: {n}'},
'ハイライト':         {'ja':'試合ハイライト実況:',     'en':'Match Highlights:'},
'ダービー記録':       {'ja':'⚔️ダービー',              'en':'⚔️ Derby Record'},
'勝分敗':             {'ja':'{w}勝{d}分{l}敗',         'en':'{w}W {d}D {l}L'},
'実績label':          {'ja':'🎖️実績',                 'en':'🎖️ Achievements'},
'監督キャリア':       {'ja':'👔監督キャリア',          'en':'👔 Manager Career'},
'キャリア行':         {'ja':'S{s}: {c} {d} {r}位',     'en':'S{s}: {c} {d} #{r}'}
'前期成績':           {'ja':'📊 前期シーズン成績',     'en':'📊 Last Season Revie
'クラブ歴史書':       {'ja':'📖クラブ歴史書',          'en':'📖 Club History'},
'歴史行':             {'ja':'S{s}: {d} {r}位{a} {w}勝 得点{g} 予算{b}',
                      'en':'S{s}: {d} #{r}{a} {w}W Goals:{g} Budget:{b}'},
'順位推移label':      {'ja':'📈順位推移',              'en':'📈 League Position'
'直近10週':           {'ja':'直近10週: ',              'en':'Last 10 weeks: '},
'移籍要求':           {'ja':'⚠️移籍要求',              'en':'⚠️ Transfer Request
'交渉btn':            {'ja':'交渉',                    'en':'Negotiate'},
'トレーニング結果':   {'ja':'🏃トレーニング結果',      'en':'🏃 Training Results'}
# チームタブ
'スターティングXI':   {'ja':'📋スターティングイレブン', 'en':'📋 Starting XI'},
'選択中':             {'ja':'選択中: {a}/11人',        'en':'Selected: {a}/11'},
'XI確定':             {'ja':'✅確定',                  'en':'✅ Set'},
'PKセットプレー':     {'ja':'⚽PKシューター / セットプレー担当指定','en':'⚽ PK / Se
'PKシューター':       {'ja':'PKシューター: {n} (SHT:{s})','en':'PK Taker: {n} (SI
'CK担当':             {'ja':'CK担当: {n} (TEC:{s})', 'en':'CK Taker: {n} (TEC:
'FK担当':             {'ja':'FK担当: {n} (SHT:{s})', 'en':'FK Taker: {n} (SHT:
'PKシューター指定':   {'ja':'PKシューター指定（SHT順）:','en':'Select PK Taker (by
'CK担当指定':         {'ja':'CK担当指定（TEC順）:',   'en':'Select CK Taker (by T
'FK担当指定':         {'ja':'FK担当指定（SHT順）:',   'en':'Select FK Taker (by S
'戦術label':          {'ja':'⚙️戦術',                 'en':'⚙️ Tactics'},
'フォーメーション':   {'ja':'フォーメーション:',       'en':'Formation:'},
'監督スキル':         {'ja':'🎓監督スキル',            'en':'🎓 Manager Skills'},
'トップチーム':       {'ja':'トップチーム ({n}名)',    'en':'First Team ({n})'},
'年俸契約':           {'ja':'{a}歳 | 年俸{b} | 契約{c}年','en':'{a}y | Wage:{b}
'出場スタッツ':       {'ja':'出場{a} G{b} A{c} MOM{d}','en':'Apps:{a} G:{b} A:{c
'放出btn':            {'ja':'🔄放出',                  'en':'🔄 Loan'},
'集中label':          {'ja':'集中:',                   'en':'Focus:'},
'役割btn':            {'ja':'役割:',                   'en':'Role:'},
'コンバート':         {'ja':'コンバート→',             'en':'Convert→'},
'レンタル放出中':     {'ja':'📩レンタル放出中',        'en':'📩 On Loan'},
'レンタル借用中':     {'ja':'📩レンタル借用中',        'en':'📩 Loan Players'},
'召還btn':            {'ja':'召還',                    'en':'Recall'},
'18歳ユース':         {'ja':'18歳到達ユース',          'en':'Promotion Eligible (1
'ユースアカデミー':   {'ja':'ユースアカデミー',        'en':'Youth Academy'},
'昇格btn':            {'ja':'昇格',                    'en':'Promote'},
```

```
'残留btn':              {'ja':'残留',                  'en':'Keep'},
'放出btn2':             {'ja':'放出',                  'en':'Release'},
'育成方針':             {'ja':'育成方針:',             'en':'Dev Policy:'},
'ユース引き抜き':       {'ja':'ユース引き抜き',        'en':'Poach Youth'},
# 試合タブ
'試合進行label':        {'ja':'試合進行',              'en':'Match'},
'節/節':                {'ja':'第{w}節/{t}節 | 天候:{s}','en':'Round {w}/{t} | Wea
'国内カップ':           {'ja':'🏆国内カップ R{r} / {s}', 'en':'🏆 Domestic Cup R{r
'出場中':               {'ja':'出場中',                'en':'Participating'},
'敗退済み':             {'ja':'敗退済み',              'en':'Eliminated'},
'節カードlabel':        {'ja':'第{w}節カード',         'en':'Round {w} Fixtures'}
'直近スコア':           {'ja':'直近: {r}',             'en':'Last: {r}'},
'観客MOM':              {'ja':'観客{a}人 MOM:{b}',     'en':'Att:{a} MOM:{b}'},
'順位表label':          {'ja':'順位表 [{d}]',          'en':'Table [{d}]'},
# 移籍タブ
'移籍ウィンドウ外label':{'ja':'🔒移籍ウィンドウ外',  'en':'🔒 Transfer Window Clos
'移籍ウィンドウ説明': {'ja':'完全移籍は夏(週1-6)と冬(週19-22)のみ可能。現在はレンタルのみ
                       'en':'Full transfers: Summer (W1-6) & Winter (W19-22) o
'FA空':                 {'ja':'現在FA選手はいません（シーズン終了後に出現）',
                       'en':'No free agents yet (appear after season end)'},
'残週':                 {'ja':'残{n}週',               'en':'{n}w left'},
'年俸要求サイン費':    {'ja':'年俸要求: {a} / サイン費: {b}','en':'Wage demand: {a}
'元所属':               {'ja':'元所属: {n}',           'en':'From: {n}'},
'海外派遣中':           {'ja':'に派遣中（残{n}週）',   'en':'Scout sent ({n}w rema
'海外費用':             {'ja':'費用: $50,000 / 4週後に帰還','en':'Cost: $50,000 / F
'海外候補':             {'ja':'海外スカウット候補:',   'en':'International Candida
'国内移籍オファー':    {'ja':'🏠国内移籍オファー',    'en':'🏠 Domestic Offers'},
'なし':                 {'ja':'なし',                  'en':'None'},
'承諾btn':              {'ja':'承諾',                  'en':'Accept'},
'カウンター':           {'ja':'カウンター×1.3',        'en':'Counter ×1.3'},
'買戻条項':             {'ja':'買戻条項付き',          'en':'With Buyback'},
'拒否btn':              {'ja':'拒否',                  'en':'Reject'},
'海外オファー':         {'ja':'✈️海外オファー',       'en':'✈️ Foreign Offers'},
'レンタル借用オファー':{'ja':'📩レンタル借用オファー', 'en':'📩 Loan Offers'},
'借りるbtn':            {'ja':'借りる',                'en':'Take Loan'},
'レンタル放出label':   {'ja':'📩レンタル放出',        'en':'📩 Loan Out'},
'選手比較label':        {'ja':'⚖️選手比較',           'en':'⚖️ Player Comparison
'自OVR':                {'ja':'自: {n} OVR{o}',        'en':'Mine: {n} OVR{o}'},
'候補OVR':              {'ja':'候補: {n} OVR{o}',      'en':'Target: {n} OVR{o}'},
# 経営タブ
'施設管理':             {'ja':'🏗️施設管理',           'en':'🏗️ Facilities'},
'強化btn':              {'ja':'{n}強化',               'en':'Upgrade {n}'},
'スタジアム拡張btn':   {'ja':'スタジアム拡張',        'en':'Expand Stadium'},
'ユース強化btn':        {'ja':'ユース強化',            'en':'Upgrade Youth'},
'VIPエリア':            {'ja':'VIPエリア: Lv{n}/3',    'en':'VIP Area: Lv{n}/3'},
'VIP拡充btn':           {'ja':'VIP拡充({n})',          'en':'Upgrade VIP ({n})'},
'スタッフ':             {'ja':'👥スタッフ',            'en':'👥 Staff'},
```

```
'スタッフ雇用':        {'ja':'スタッフ雇用 ($60,000)', 'en':'Hire Staff ($60,000)'
'財務サマリー':        {'ja':'💰財務サマリー',          'en':'💰 Finances'},
'総給与':              {'ja':'総給与: {a} / 上限: {b}','en':'Total wages: {a} / C
'銀行ローン':          {'ja':'🏦銀行ローン',           'en':'🏦 Bank Loans'},
'ローン行':            {'ja':'残{w}週 金利{r}%',       'en':'{w}w remaining {r}% i
'ローンなし':          {'ja':'ローンなし',             'en':'No loans'},
'借りるamt':           {'ja':'{n}借りる',              'en':'Borrow {n}'},
'選手保険':            {'ja':'🏥選手保険',             'en':'🏥 Player Insurance
'保険状態':            {'ja':'状態: {n}',              'en':'Status: {n}'},
'保険契約中':          {'ja':'契約中（週$8,000）',     'en':'Active (Wkly $8,000)'
'保険未契約':          {'ja':'未契約',                 'en':'Inactive'},
'保険契約btn':         {'ja':'保険契約（週$8,000）',   'en':'Activate Insurance ($
'予算配分':            {'ja':'💹予算配分',             'en':'💹 Budget Allocatio
'攻撃的配分':          {'ja':'攻撃的(移籍60%)',        'en':'Aggressive (Transfer
'バランス配分':        {'ja':'バランス(各33%)',        'en':'Balanced (Equal 33%)
'育成重視配分':        {'ja':'育成重視(施設50%)',      'en':'Youth Focus (Facilit
'CL今季なし':          {'ja':'今季終了後に自動開幕（D1上位4チームが出場）',
                        'en':'Starts after season end (Top 4 D1 teams qualify)'
'コンカップ優勝':          {'ja':'🏆優勝: {n}',        'en':'🏆 Champions:
'コンカップ出場中':        {'ja':'✅ 自クラブ出場中',  'en':'✅ Your club i
'コンカップ出場なし':      {'ja':'今季は出場資格なし', 'en':'Not qualified t
'CL進行':              {'ja':'CL進行: 週{a}〜グループ / 週{b}〜決勝T（自動進行）',
                        'en':'CL: Groups from W{a} / Knockout from W{b} (auto)'
'グループ':            {'ja':'グループ{n}:',           'en':'Group {n}:'},
'残クラブ':            {'ja':'残{n}クラブ',            'en':'{n} clubs remaining'
'セーブロードlabel':   {'ja':'💾セーブ/ロード',        'en':'💾 Save/Load'},
'セーブbtn2':          {'ja':'セーブ',                 'en':'Save'},
# 選手詳細
'所属国籍地元':        {'ja':'{c} / {n} {l}',          'en':'{c} / {n} {l}'},
'地元':                {'ja':'🏠地元',                 'en':'🏠 Local'},
'年齢ポジOVR':         {'ja':'{a}歳 | {p} | OVR{o} | POT{pt}','en':'{a}y | {p} |
'成長状態':            {'ja':'成長:{g} | 状態:{s}',    'en':'Growth:{g} | Form:{s}
'契約年俸':            {'ja':'契約{c}年 | 年俸{w}',    'en':'Contract:{c}y | Wage:
'代表召集':            {'ja':'代表召集中 {n}週',       'en':'International duty {n
'帰還成長':            {'ja':'→帰還後成長あり',        'en':'→ Growth boost on ret
'エージェント':        {'ja':'エージェントあり（手数料{n}%）','en':'Agent (fee:{n}%)'
'引退予定':            {'ja':'💐今季引退予定',         'en':'💐 Retiring this sea
'コンバート中':        {'ja':'🔄コンバート中→{p} ({w}週)',
                        'en':'🔄 Converting→{p} ({w}w)'},
'役割希望':            {'ja':'役割希望: {n}',          'en':'Role preference: {n}'
'買戻し':              {'ja':'買戻し:{n}',             'en':'Buyback:{n}'},
'閉じるbtn':           {'ja':'閉じる',                 'en':'Close'},
# 施設名
'施設youth':           {'ja':'ユース',                 'en':'Youth'},
'施設training':        {'ja':'トレーニング',           'en':'Training'},
'施設medical':         {'ja':'メディカル',             'en':'Medical'},
'施設scouting':        {'ja':'スカウト',               'en':'Scouting'},
```

```
    '施設stadium':        {'ja':'スタジアム',            'en':'Stadium'},
    'ロッカー最高':       {'ja':'ロッカー: 🎉 最高の雰囲気','en':'Locker: 🎉 Buzzing'}
    'ロッカー良い':       {'ja':'ロッカー: 😊 良い雰囲気', 'en':'Locker: 😊 Good vibes
    'ロッカー普通':       {'ja':'ロッカー: 😐 普通',      'en':'Locker: 😐 Okay'},
    'ロッカー緊張':       {'ja':'ロッカー: 😤 緊張感',    'en':'Locker: 😤 Tense'},
    'ロッカー険悪':       {'ja':'ロッカー: 💣 険悪',      'en':'Locker: 💣 Toxic'},
}

def T(key):
    """現在の言語に応じたテキストを返す"""
    entry = TRANSLATIONS.get(key)
    if entry:
        return entry.get(LANG, entry.get('ja', key))
    return key

def set_lang(lang):
    global LANG
    LANG = lang
    refresh_ui()


def get_top_scorers(n=5):
    """全チームから得点王ランキングを取得"""
    all_players=[]
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            for p in t['players']:
                if p['stats']['goals']>0 or p['stats']['assists']>0:
                    all_players.append({
                        'name':p['name','club':t['name','div':d,
                        'goals':p['stats']['goals'],
                        'assists':p['stats']['assists'],
                        'mom':p['stats']['mom'],
                        'pos':p['pos','overall':p['overall']
                    })
    return sorted(all_players,key=lambda x:-x['goals'])[:n],           sorted(all


def render_formation_view(t, xi_ids):
    """フォーメーション形式でスターティングイレブンを表示（SVG）"""
    import math
    formation=t.get('formation','4-4-2')
    req=FORMATIONS.get(formation,FORMATIONS['4-4-2'])
    avail=[p for p in t['players'] if p['injury_weeks']<=0 and p.get('intl_weeks'
    xi_players=[p for p in avail if p['id'] in xi_ids]

    # ポジション別に分類
```

```
    pos_map={'GK':[],'DF':[],'MF':[],'FW':[]}
    for p in xi_players:
        if p['pos'] in pos_map: pos_map[p['pos']].append(p)

    W,H=320,380
    svg=[f'<svg width="{W}" height="{H}" viewBox="0 0 {W} {H}" style="display:blo
    # ピッチ背景
    svg.append(f'<rect width="{W}" height="{H}" rx="8" fill="#0d2b1a"/>')
    svg.append(f'<rect x="20" y="10" width="{W-40}" height="{H-20}" rx="4" fill="
    # センターライン・サークル
    svg.append(f'<line x1="20" y1="{H//2}" x2="{W-20}" y2="{H//2}" stroke="#1a5c2
    svg.append(f'<circle cx="{W//2}" cy="{H//2}" r="40" fill="none" stroke="#1a5c
    # ペナルティエリア（上下）
    px,pw,ph=W//2-60,120,50
    svg.append(f'<rect x="{px}" y="10" width="{pw}" height="{ph}" fill="none" str
    svg.append(f'<rect x="{px}" y="{H-10-ph}" width="{pw}" height="{ph}" fill="no

    # ポジション行のY座標（GK=下、FW=上）
    rows={'FW':50,'MF':155,'DF':255,'GK':325}

    for pos,players in pos_map.items():
        y=rows[pos]; n_req=req.get(pos,0)
        n=max(len(players),1)
        for j,p in enumerate(players):
            x=(j+1)*W/(n+1)
            # プレイヤーアイコン（円）
            col='#3b82f6' if p['id'] in xi_ids else '#475569'
            svg.append(f'<circle cx="{x:.0f}" cy="{y}" r="18" fill="{col}" stroke
            # 番号/OVR
            svg.append(f'<text x="{x:.0f}" y="{y+1}" text-anchor="middle" dominan
            # 名前（短縮）
            short=p['name'].split()[-1][:8] if ' ' in p['name'] else p['name'][:8
            svg.append(f'<text x="{x:.0f}" y="{y+25}" text-anchor="middle" font-s
            # ポジション
            svg.append(f'<text x="{x:.0f}" y="{y-24}" text-anchor="middle" font-s

    svg.append('</svg>')
    ui.html("".join(svg))


# ============================================================
# 不透明化ヘルパー（数値をテキストヒントに変換）
# ============================================================
def mood_text(unhappiness):
    """不満値をテキストに変換"""
    if unhappiness>=80: return (T('不満爆発'),    'text-red')
    if unhappiness>=60: return (T('かなり不満'),   'text-red')
```

```
    if unhappiness>=40: return (T('やや不満'),     'text-orange')
    if unhappiness>=20: return (T('普通'),         'text-grey')
    return                (T('満足'),              'text-positive')

def morale_text(morale):
    """モラルをテキストに変換"""
    if morale>=80: return ('🔥 絶好調',  'text-positive')
    if morale>=60: return ('😊 好調',    'text-positive')
    if morale>=40: return ('😐 普通',    'text-grey')
    if morale>=20: return ('😟 低調',    'text-orange')
    return                ('💔 最悪',    'text-red')

def injury_text(weeks, severity):
    """怪我回復をテキストに変換"""
    if weeks<=0: return ('', '')
    if weeks==1: return (T('まもなく復帰'), 'text-orange')
    if weeks<=3: return (T('復帰まで少し'), 'text-orange')
    if weeks<=8: return (T('長期離脱中'),   'text-red')
    return                (T('重傷長期'),    'text-red')

def board_text(rating):
    """理事会評価をテキストに変換"""
    if rating>=80: return (T('理事会大絶賛'), 'text-positive')
    if rating>=60: return (T('理事会満足'),   'text-positive')
    if rating>=40: return (T('理事会普通'),   'text-grey')
    if rating>=25: return (T('理事会不満'),   'text-orange')
    return                (T('理事会激怒'),   'text-red')

def rival_activity_text():
    """ライバルクラブの補強動向ヒント（ランダム）"""
    import random as _r
    sel=get_sel()
    if not sel: return None
    rivals=sel.get('rivals',[])
    if not rivals: return None
    if _r.random()>0.3: return None  # 70%の確率でヒントなし
    rival=_r.choice(rivals)
    hints=[
        f'噂: {rival}が補強に動いているようだ',
        f'情報: {rival}のスカウトが活発に動いている',
        f'噂: {rival}に有力な移籍候補がいるらしい',
        f'情報: {rival}の選手の一部が不満を持っているようだ',
    ]
    return _r.choice(hints)

def stamina_text(stamina):
    """スタミナをテキストに変換"""
```

```
    if stamina>=80: return ('元気', 'text-positive')
    if stamina>=60: return ('やや疲労', 'text-grey')
    if stamina>=40: return ('疲労', 'text-orange')
    return                ('かなり疲労', 'text-red')


# ============================================================
# 選手・チーム生成
# ============================================================
def _recompute(p):
    w=OVR_WEIGHTS.get(p['pos'],{k:0.125 for k in p['attrs']})
    p['overall']=clamp(int(sum(p['attrs'].get(k,50)*v for k,v in w.items())),35,9
    p['value']=int(p['overall']*2500+p['potential']*1800)
    p['wage']=int(1800+p['overall']*140)

def gen_player(country,club,idx,youth=False):
    pos=random.choice(POSITIONS)
    attrs=copy.deepcopy(BASE_ATTRS[pos])
    lmod=(LEAGUE_STRENGTH.get(country,80)-80)*0.1
    for k in attrs: attrs[k]+=random.randint(-8,8)+int(lmod)
    if youth:
        pol=YOUTH_POLICIES.get(game_state['youth_policy'] if game_state else 'バラ
        attrs['SPD']=int(attrs['SPD']*pol['SPD']); attrs['TEC']=int(attrs['TEC']*
        attrs['PAS']=int(attrs['PAS']*pol['TEC']); attrs['PHY']=int(attrs['PHY']*
        fac=game_state['facilities']['youth'] if game_state else 1
        for k in attrs: attrs[k]-=random.randint(1,max(2,6-min(4,fac)))
    for k in attrs: attrs[k]=clamp(attrs[k],30,95)
    age=random.randint(16,18) if youth else random.randint(18,35)
    p={
        'id':f'{club}_{idx}_{random.randint(10000,99999)}',
        'name':f'{random.choice(COUNTRY_NAME_POOLS[country]["first"])} {random.ch
        'club':club,'nationality':country,'pos':pos,'age':age,'attrs':attrs,
        'overall':50,'potential':0,
        'playstyle':random.choice(PLAY_STYLES[pos]),
        'personality':random.choice(PERSONALITIES),
        'trait':wc(['なし','勝負師','ムードメーカー','スペ体質','大舞台男','技巧派','鉄人'
                    'キャプテン体質','若手の星','経験値豊富','熱血漢','冷静沈着','孤高の天
                    'チームの心臓','マルチロール','ラストミニッツ','プレッシャー耐性','怪物
                   [25,8,8,6,5,5,5,5,4,4,4,3,3,3,3,3,3,3,2,2]),
        'growth_type':wc(['早熟','標準','晩成'],[30,50,20]) if youth else None,
        'contract_years':random.randint(2,4) if youth else random.randint(1,5),
        'unhappiness':0,'injury_weeks':0,'injury_severity':'なし',
        'stamina':100,'morale':50,'is_wonderkid':False,'transfer_request':False,
        'state':'normal','state_weeks':0,'intl_weeks':0,
        'stats':{'apps':0,'goals':0,'assists':0,'mom':0,'clean_sheets':0,'yellow_
        'value':0,'wage':0,
        'loan_club':None,'loan_origin':None,'loan_weeks':0,
```

```
        'buyback_fee':None,'buyback_club':None,
        'retiring':False,'training_focus':None,
        'has_agent':random.random()<0.3,'agent_fee_pct':random.choice([5,8,10]),
        # スカウトグレード（不確実性）
        'scout_grade':wc(['A','B','C','D','E'],[10,20,35,25,10]),
        # コンバート
        'original_pos':pos,'convert_weeks':0,
        # 希望ポジション
        'preferred_pos':pos,
        # プレー頻度希望
        'role_wish':'どちらでも',  # 主力/控えでも/主力のみ
        # 代表帰還ブースト
        'intl_boost_pending':False,
        # 地元出身フラグ
        'is_local':random.random()<0.15,
    }
    _recompute(p); p['potential']=clamp(p['overall']+random.randint(5,20),p['over
    if youth and random.random()<0.02:
        p['potential']=random.randint(88,94)
        for k in p['attrs']: p['attrs'][k]=clamp(p['attrs'][k]+random.randint(6,1
        p['is_wonderkid']=True
    _recompute(p); return p

def create_team(country,name,rmin=50,rmax=75):
    players=[gen_player(country,name,i) for i in range(18)]
    youth=[gen_player(country,name,1000+i,True) for i in range(random.randint(MIN
    return {
        'name':name,'country':country,'players':players,'youth':youth,'loan_in':[
        'budget':random.randint(700000,1600000),
        'reputation':random.randint(rmin,rmax),
        'fan_base':random.randint(8000,20000),
        'stadium_capacity':random.randint(12000,35000),
        'tactic':'バランス','formation':'4-4-2',
        'rivals':[],'board_expectation':'中位確保',
        'derby_record':{},'sponsor':None,'player_season_goal':None,
        'captain_id':None,'vip_level':0,'rank_history':[],
        'starting_xi':[],'preseason_done':False,
        'board_meeting_week':8,'naming_rights':None,
        'chemistry':50,'budget_transfer':0,'budget_facility':0,'budget_staff':0,
        # PKシューター・セットプレー担当
        'pk_taker_id':None,'ck_taker_id':None,'fk_taker_id':None,
        # 監督キャリア用
        'manager_offer_pending':False,
        # ローン記録
        'bank_loan':None,
        # 歴史記録
        'season_history':[],
```

```
    }

def build_league(country):
    used=set()
    def rc(rmin,rmax): return [create_team(country,_rname(country,used),rmin,rmax
    divs={'D1':rc(62,78),'D2':rc(54,70),'D3':rc(46,64)}
    for div in divs:
        tms=divs[div]
        for i in range(0,len(tms),2):
            if i+1<len(tms):
                tms[i]['rivals'].append(tms[i+1]['name']); tms[i+1]['rivals'].app
        for t in tms:
            if div=='D1': t['board_expectation']=wc(['優勝争い','上位進出','残留'],[
            elif div=='D2': t['board_expectation']=wc(['昇格争い','中位確保','残留'],
            else: t['board_expectation']=wc(['昇格','上位進出','中位確保'],[30,35,35
    return divs

def _rname(country,used):
    pool=COUNTRY_NAME_POOLS[country]['clubs']
    suffs=['FC','United','City','SC','Athletic','Club']
    while True:
        n=f'{random.choice(pool)} {random.choice(suffs)}'
        if n not in used: used.add(n); return n

def build_world(country):
    divs=build_league(country)
    return {
        'season':1,'week':1,
        'selected_country':country,'selected_club':'','selected_player_id':'',
        'divisions':divs,
        'news':[{'text':f'{country}で新しい世界を作成しました','cat':'general','seaso
        'transfer_offers':[],'foreign_offers':[],'loan_offers':[],'loan_out_reque
        'season_awards':None,'history_awards':[],'club_hall_of_fame':[],'last_mat
        'manager_rating':50,'board_rating':50,'locker_room_mood':50,'fan_happines
        'manager_exp':0,'manager_skill_points':0,'manager_skills':[],
        # 監督キャリア
        'manager_career':[],  # [{club, season, div, rank}]
        'manager_career_offers':[],
        'facilities':{'youth':1,'training':1,'medical':1,'scouting':1,'stadium':1
        'staff':[],'international_cup':None,
        'youth_policy':'バランス','youth_decision_queue':[],
        'season_finance':{k:0 for k in ['sponsor','matchday','transfers_in','tran
        'pending_sponsor_negotiation':None,
        'manager_fired':False,'season_goal_declared':False,
        'domestic_cup':None,'pending_press':False,'press_effect':None,
        'intl_call_queue':[],'financial_crisis':False,
        'rank_history':[],'current_weather':'晴れ',
```

```
        'preseason_phase':True,
        'pending_lineup':False,'halftime_data':None,
        'pending_board_meeting':False,
        'buyout_offer':None,'achievements':[],'training_results':[],
        'win_streak':0,'clean_streak':0,'youth_promoted_count':0,'wonderkid_found
        'tutorial_done':False,
        # ランダムイベントキュー
        'pending_event':None,
        'event_history':[],
        # テキスト実況
        'live_commentary':[],
        # クラブ歴史書
        'club_history':[],
        'news_filter':'全て',
        # 複数週一括進行中フラグ
        'bulk_advancing':False,'bulk_weeks_left':0,
        # 銀行ローン
        'bank_loans':[],
        # 選手保険
        'injury_insurance_active':False,
        # 肖像権収入
        'portrait_income_weekly':0,
        # 海外スカウト派遣
        'overseas_scout':None,
        # win_bonus支払い記録
        'last_match_won':False,
    }

def init_stats():
    # チームのseason_stats補完
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if 'season_stats' not in t:
                t['season_stats']={'p':0,'w':0,'d':0,'l':0,'gf':0,'ga':0,'gd':0,'
    # game_stateの必須キーを全て補完（古いセーブデータ対応）
    defaults={
        'fa_market':[],'cc_data':None,'tryout_pool':[],'tryout_done':False,
        'tryout_signed_count':0,'preset_halftime_tactic':None,
        'last_season_summary':None,'tutorial_done':False,
        'news_filter':None,'overseas_scout':None,'overseas_pool':[],
        'bank_loans':[],'injury_insurance_active':False,
        'portrait_income_weekly':0,'achievements':{},'rank_history':[],
        'manager_career':[],'manager_career_offers':[],'manager_skills':{},
        'manager_skill_points':0,'manager_exp':0,'manager_rating':50,
        'board_rating':50,'fan_happiness':50,'locker_room_mood':50,
        'club_brand':50,'financial_crisis':False,'pending_event':None,
        'pending_press':False,'press_effect':None,'pending_sponsor_negotiation':N
```

```
        'naming_rights_offer':None,'buyout_offer':None,'win_streak':0,
        'season_goal_declared':False,'halftime_data':None,'live_commentary':[],
        'club_history':[],'season_finance':{},
    }
    for key,val in defaults.items():
        if key not in game_state:
            game_state[key]=val if not callable(val) else val()

# ============================================================
# スターティングイレブン
# ============================================================
def best_lineup(team,override_xi=None):
    pool=team['players']+team.get('loan_in',[])
    if not pool: return []
    avail=[p for p in pool if p['injury_weeks']<=0 and p.get('intl_weeks',0)<=0]
    if not avail: return pool[:11]
    xi_ids=override_xi or team.get('starting_xi',[])
    if xi_ids:
        xi=[p for p in avail if p['id'] in xi_ids]
        if len(xi)<11:
            rest=sorted([p for p in avail if p['id'] not in xi_ids],key=lambda p:
            xi+=rest[:11-len(xi)]
        return xi[:11]
    req=FORMATIONS.get(team.get('formation','4-4-2'),FORMATIONS['4-4-2'])
    lineup=[]; rem=list(avail)
    for pos,cnt in req.items():
        chosen=sorted([p for p in rem if p['pos']==pos],key=lambda p:p['overall']
        lineup+=chosen
        for p in chosen: rem.remove(p)
    if len(lineup)<11:
        lineup+=sorted(rem,key=lambda p:p['overall'],reverse=True)[:11-len(lineup
    return lineup[:11]

def set_starting_xi(player_ids):
    sel=get_sel()
    if not sel: return
    sel['starting_xi']=player_ids[:11]; game_state['pending_lineup']=False
    add_news('スターティングイレブンを設定','match'); refresh_ui()

def auto_lineup():
    sel=get_sel()
    if not sel: return
    sel['starting_xi']=[]; game_state['pending_lineup']=False
    add_news('ラインナップを自動選出に設定','match'); refresh_ui()

# ============================================================
# 試合強度計算（ホーム補正追加）
```

```
# ============================================================
def team_str(team,opp=None,tactic_override=None,is_home=False):
    lu=best_lineup(team)
    if not lu: return 40
    base=avg([p['overall'] for p in lu])
    league_b=(LEAGUE_STRENGTH.get(team.get('country','England'),80)-80)*0.05
    morale_b=avg([p['morale'] for p in lu])*0.05
    stamina_b=avg([p['stamina'] for p in lu])*0.03
    style_b=sum({'ストライカー':0.30,'ポーチャー':0.22,'プレイメーカー':0.25,'ディストラク
    derby_b=2.5 if opp and opp['name'] in team.get('rivals',[]) else 0
    # ホーム補正
    home_b=2.5 if is_home else 0
    fac_b=game_state['facilities']['training']*0.8+game_state['facilities']['medi
    tn=tactic_override or team.get('tactic','バランス')
    tac=TACTICS.get(tn,TACTICS['バランス'])
    tac_b=(tac['atk']+tac['def'])/2*base*0.05-base*0.05
    staff_b=sum(s['skill']*0.01 for s in game_state['staff'] if s['type']=='ヘッド
    staff_b+=sum(s['skill']*0.005 for s in game_state['staff'] if s['type']=='フィ
    brand_b=(game_state['club_brand']-50)*0.02
    frm=FORMATIONS.get(team.get('formation','4-4-2'),FORMATIONS['4-4-2'])
    pos_cnt={p['pos']:0 for p in lu}
    for p in lu: pos_cnt[p['pos']]=pos_cnt.get(p['pos'],0)+1
    frm_b=sum(0.3 for pos,cnt in frm.items() if pos_cnt.get(pos,0)>=cnt)
    skill_b=base*0.10 if has_skill('戦術家') else 0
    cap_id=team.get('captain_id')
    cap_b=0
    if cap_id:
        cap=next((p for p in lu if p['id']==cap_id),None)
        if cap: cap_b=CAPTAIN_EFFECTS.get(cap['personality'],{}).get('morale',0)*
    # セットプレー担当者ボーナス（CK/FKのTEC・SHT属性を反映）
    setpiece_b=0
    ck_id=team.get('ck_taker_id')
    fk_id=team.get('fk_taker_id')
    if ck_id:
        ck=next((p for p in lu if p['id']==ck_id),None)
        if ck: setpiece_b+=ck['attrs'].get('TEC',50)*0.015
    if fk_id:
        fk=next((p for p in lu if p['id']==fk_id),None)
        if fk: setpiece_b+=fk['attrs'].get('SHT',50)*0.012
    state_b=(avg([1.1 if p.get('state')=='hot' else(0.9 if p.get('state')=='slump
    chem_b=(team.get('chemistry',50)-50)*0.01
    # 特性ボーナス
    trait_b=0
    for p in lu:
        tr=p.get('trait','なし')
        if tr=='大舞台男' and (opp and opp['name'] in team.get('rivals',[])): trai
        if tr=='速攻屋' and (tactic_override or team.get('tactic','’))=='カウンター'
```

```
        if tr=='鉄人': trait_b+=0.2
    weather=WEATHER.get(get_weather(),WEATHER['晴れ'])
    weather_b=(weather['atk_mod']-1.0)*base
    # コンバート中ペナルティ
    convert_penalty=sum(3 for p in lu if p.get('convert_weeks',0)>0)
    return base+league_b+morale_b+stamina_b+style_b+derby_b+home_b+fac_b+tac_b+st

# ============================================================
# テキスト実況生成
# ============================================================
def gen_live_events(ta,tb,ga1,gb1,ga2,gb2,cards,weather):
    events=[]
    t=0
    lu_a=best_lineup(ta) or ta.get('players',[])
    lu_b=best_lineup(tb) or tb.get('players',[])
    if not lu_a or not lu_b: return events  # 選手がいなければスキップ
    events.append({'t':t,'txt':f'⚽ キックオフ！ {ta["name"]} vs {tb["name"]} | 天候
    apos="'"
    for i in range(ga1):
        t=random.randint(5,44)
        scorer=random.choice(lu_a)
        events.append({'t':t,'txt':f'⚽ GOAL! {ta["name"]} | {scorer["name"]} ({t
    for i in range(gb1):
        t=random.randint(5,44)
        scorer=random.choice(lu_b)
        events.append({'t':t,'txt':f'⚽ GOAL! {tb["name"]} | {scorer["name"]} ({t
    events.append({'t':45,'txt':f'📯 ハーフタイム | {ta["name"]} {ga1}-{gb1} {tb["n
    for col,name in cards[:3]:
        t=random.randint(46,89)
        events.append({'t':t,'txt':f'{col} カード: {name} ({t}{apos})'})
    for i in range(ga2):
        t=random.randint(46,90)
        scorer=random.choice(lu_a)
        events.append({'t':t,'txt':f'⚽ GOAL! {ta["name"]} | {scorer["name"]} ({t
    for i in range(gb2):
        t=random.randint(46,90)
        scorer=random.choice(lu_b)
        events.append({'t':t,'txt':f'⚽ GOAL! {tb["name"]} | {scorer["name"]} ({t
    if random.random()<0.12:
        t=random.randint(90,93)
        events.append({'t':t,'txt':f'⚡ アディショナルタイム弾！ ({t}{apos})'})
    events.append({'t':95,'txt':f'🔔 試合終了 | {ta["name"]} {ga1+ga2}-{gb1+gb2} {
    events.sort(key=lambda e:e['t'])
    return events

# ============================================================
# 試合シミュレーション（PKシュートアウト対応）
```

```
# ==========================================================
def simulate_pk_shootout(ta,tb):
    """PKシューターアウト（カップ戦延長後）"""
    pk_a=ta.get('pk_taker_id')
    pk_b=tb.get('pk_taker_id')
    taker_a=next((p for p in ta['players'] if p['id']==pk_a),None) if pk_a else N
    taker_b=next((p for p in tb['players'] if p['id']==pk_b),None) if pk_b else N
    shot_a=taker_a['attrs'].get('SHT',60)/100 if taker_a else 0.65
    shot_b=taker_b['attrs'].get('SHT',60)/100 if taker_b else 0.65
    score_a=sum(1 for _ in range(5) if random.random()<shot_a)
    score_b=sum(1 for _ in range(5) if random.random()<shot_b)
    while score_a==score_b:
        score_a+=1 if random.random()<shot_a else 0
        score_b+=1 if random.random()<shot_b else 0
    winner=ta if score_a>score_b else tb
    return winner,score_a,score_b

def simulate_half(ta,tb,tactic_a=None,tactic_b=None,home_team=None):
    sa=team_str(ta,tb,tactic_a,is_home=(home_team==ta['name']))
    sb=team_str(tb,ta,tactic_b,is_home=(home_team==tb['name']))
    ga=min(4,max(0,int(round(random.gauss(sa/48,0.7)))))
    gb=min(4,max(0,int(round(random.gauss(sb/48,0.7)))))
    return ga,gb

def simulate_match_full(ta,tb,cup=False,ht_tac_a=None,ht_tac_b=None,substitutions
    derby=tb['name'] in ta.get('rivals',[])
    weather=get_weather(); wdat=WEATHER.get(weather,WEATHER['晴れ'])
    ga1,gb1=simulate_half(ta,tb,home_team=ta['name'])
    # 交代処理（ハーフタイム）
    if substitutions_a:
        _apply_substitutions(ta,substitutions_a)
    ga2,gb2=simulate_half(ta,tb,ht_tac_a,None,home_team=ta['name'])
    ga=min(8,ga1+ga2); gb=min(8,gb1+gb2)
    # カップ戦PKシュートアウト
    pk_result=None
    if cup and ga==gb:
        # 延長戦
        gex_a,gex_b=simulate_half(ta,tb,home_team=ta['name'])
        ga+=gex_a; gb+=gex_b
        if ga==gb:
            winner,sa_pk,sb_pk=simulate_pk_shootout(ta,tb)
            pk_result={'winner':winner['name'],'score_a':sa_pk,'score_b':sb_pk}
            if winner['name']==ta['name']: ga+=1
            else: gb+=1
            add_news(f'⚽PKシュートアウト: {ta["name"]} {sa_pk}-{sb_pk} {tb["name"]
            if winner['name']==game_state['selected_club']: check_achievement('pk
```

```
    sa=team_str(ta,tb,is_home=True); sb=team_str(tb,ta)
    shots_a=max(3,int(sa/5.5)+random.randint(-2,3)); shots_b=max(3,int(sb/5.5)+ra
    tot=max(1,sa+sb); pa=clamp(int(sa/tot*100+random.randint(-6,6)),34,66)
    stats={'team_a':{'name':ta['name'],'shots':shots_a,'on_target':min(shots_a,ma
    mom,sclog,cards=_match_stats_full(ta,tb,ga,gb,wdat)
    # win_bonus支払い
    if ta['name']==game_state['selected_club'] and ga>gb:
        _pay_win_bonus(ta)
    elif tb['name']==game_state['selected_club'] and gb>ga:
        _pay_win_bonus(tb)
    hl=[]
    if weather not in ['晴れ','曇り']: hl.append(f'🌦{weather}の中での試合')
    if derby: hl.append('🔥ダービーマッチ')
    if abs(ga-gb)>=3: hl.append('一方的な試合展開')
    if random.random()<0.18: hl.append('スーパーセーブ')
    if random.random()<0.12: hl.append('ポスト直撃')
    if random.random()<0.08: hl.append('⚠️VARチェック発生')
    if ga1<gb1 and ga>gb: hl.append('🔄逆転劇！')
    if random.random()<0.12: hl.append('⚡ロスタイムの劇的展開')
    if pk_result: hl.append(f'📖PKシュートアウト({pk_result["score_a"]}-{pk_result[
    att=_attendance(ta,tb,derby,wdat); rev=_match_income(ta,att)
    res_a='win' if ga>gb else('draw' if ga==gb else 'loss')
    res_b='loss' if res_a=='win' else('draw' if res_a=='draw' else 'win')
    _update_fanbase(ta,res_a,derby); _update_fanbase(tb,res_b,derby)
    _update_table(ta,ga,gb); _update_table(tb,gb,ga)
    if derby:
        ra=ta.setdefault('derby_record',{}).setdefault(tb['name'],{'w':0,'d':0,'l
        rb=tb.setdefault('derby_record',{}).setdefault(ta['name'],{'w':0,'d':0,'l
        if ga>gb: ra['w']+=1;rb['l']+=1
        elif ga==gb: ra['d']+=1;rb['d']+=1
        else: ra['l']+=1;rb['w']+=1
    _fatigue_weather(ta,wdat); _fatigue_weather(tb,wdat)
    # テキスト実況イベント生成
    live_evts=gen_live_events(ta,tb,ga1,gb1,ga2,gb2,cards,weather)
    game_state['live_commentary']=live_evts
    return ga,gb,att,rev,stats,{'mom':mom,'scorers':sclog,'highlights':hl,'derby'

def _pay_win_bonus(team):
    sp=team.get('sponsor')
    if not sp: return
    bonus=sp.get('win_bonus',0)
    if bonus>0:
        team['budget']+=bonus; add_fin('スポンサー勝利ボーナス',bonus,sp['name']); ad
        if team['name']==game_state['selected_club']:
            add_news(f'🎉スポンサー勝利ボーナス: {fmt_m(bonus)}','club')

def _apply_substitutions(team,subs):
```

```
    """ハーフタイム交代: [(out_id, in_id), ...]"""
    for out_id,in_id in subs[:3]:
        out_p=next((p for p in team['players'] if p['id']==out_id),None)
        in_p=next((p for p in team['players'] if p['id']==in_id and p['id'] not i
        if out_p and in_p and out_id in team.get('starting_xi',[]):
            xi=list(team.get('starting_xi',[])); xi.remove(out_id); xi.append(in_

def _match_stats_full(ta,tb,ga,gb,wdat):
    lu_a=best_lineup(ta) or ta.get('players',[])[:11]
    lu_b=best_lineup(tb) or tb.get('players',[])[:11]
    if not lu_a or not lu_b:
        # 選手がいない場合はダミーのmomを返す
        dummy={'name':'Unknown','id':'dummy','pos':'MF','overall':50,'attrs':{},'
        return dummy,[],[]
    for p in lu_a: p['stats']['apps']+=1; p['stats']['shots']=p['stats'].get('sho
    for p in lu_b: p['stats']['apps']+=1; p['stats']['shots']=p['stats'].get('sho
    for p in lu_a:
        if p['pos']=='GK': p['stats']['saves']=p['stats'].get('saves',0)+gb
        if p['pos']=='DF': p['stats']['tackles']=p['stats'].get('tackles',0)+rand
    for p in lu_b:
        if p['pos']=='GK': p['stats']['saves']=p['stats'].get('saves',0)+ga
        if p['pos']=='DF': p['stats']['tackles']=p['stats'].get('tackles',0)+rand
    if gb==0:
        for p in lu_a:
            if p['pos']=='GK': p['stats']['clean_sheets']=p['stats'].get('clean_s
    if ga==0:
        for p in lu_b:
            if p['pos']=='GK': p['stats']['clean_sheets']=p['stats'].get('clean_s
    contrib={p['id']:0 for p in lu_a+lu_b}; sclog=[]
    for _ in range(ga):
        weights=[PLAYER_STATES.get(p.get('state','normal'),PLAYER_STATES['normal'
        sc=random.choices(lu_a,weights=weights,k=1)[0]
        sc['stats']['goals']+=1; contrib[sc['id']]+=4
        if sc.get('trait')=='勝負師': contrib[sc['id']]+=1
        ast=random.choice(lu_a)
        if ast['id']!=sc['id']: ast['stats']['assists']+=1; contrib[ast['id']]+=2
        sclog.append((ta['name'],sc['name']))
    for _ in range(gb):
        weights=[PLAYER_STATES.get(p.get('state','normal'),PLAYER_STATES['normal'
        sc=random.choices(lu_b,weights=weights,k=1)[0]
        sc['stats']['goals']+=1; contrib[sc['id']]+=4
        if sc.get('trait')=='勝負師': contrib[sc['id']]+=1
        ast=random.choice(lu_b)
        if ast['id']!=sc['id']: ast['stats']['assists']+=1; contrib[ast['id']]+=2
        sclog.append((tb['name'],sc['name']))
    cards=[]
    for p in lu_a+lu_b:
```

```
        if random.random()<0.08*wdat.get('injury_mod',1.0):
            p['stats']['yellow_cards']=p['stats'].get('yellow_cards',0)+1
            cards.append(('🟨',p['name']))
            if p['stats']['yellow_cards']%5==0: p['injury_weeks']=max(p['injury_w
        if random.random()<0.01:
            p['stats']['red_cards']=p['stats'].get('red_cards',0)+1
            cards.append(('🟥',p['name'])); p['injury_weeks']=max(p['injury_weeks
    mom=max(lu_a+lu_b,key=lambda p:contrib[p['id']]+p['overall']*0.1+random.unifo
    mom['stats']['mom']+=1; return mom,sclog,cards

def _update_table(t,gf,ga):
    s=t['season_stats']; s['p']+=1; s['gf']+=gf; s['ga']+=ga; s['gd']=s['gf']-s['
    if gf>ga: s['w']+=1;s['pts']+=3
    elif gf==ga: s['d']+=1;s['pts']+=1
    else: s['l']+=1

def _update_fanbase(t,res,derby=False):
    ch={'win':random.randint(120,260),'draw':random.randint(-30,60),'loss':random
    if derby: ch*=2
    t['fan_base']=max(2000,t['fan_base']+ch)

def _attendance(ta,tb,derby=False,wdat=None):
    att=int(ta['fan_base']*random.uniform(0.65,1.10))+(ta['reputation']+tb['reput
    if derby: att=int(att*1.35)
    att=int(att*(1+game_state['facilities']['stadium']*0.02+(game_state['club_bra
    if wdat: att=int(att*wdat.get('att_mod',1.0))
    return min(att,ta['stadium_capacity'])

def _match_income(ht,att):
    ticket=random.randint(18,28)
    vip_bonus=ht.get('vip_level',0)*random.randint(8000,15000)
    nr_bonus=50000 if ht.get('naming_rights') else 0
    rev=att*ticket+vip_bonus+nr_bonus
    ht['budget']+=rev; add_fin('観客収入',rev,'ホームゲーム'); add_sf('matchday',rev)

def _fatigue_weather(team,wdat):
    tr=game_state['facilities']['training']
    for p in best_lineup(team):
        loss=max(3,int(random.randint(6,12)-(tr*0.3))); loss=int(loss*wdat.get('i
        p['stamina']=clamp(p['stamina']-loss,20,100)
    res=_injure_weather(team,wdat)
    if res and team['name']==game_state['selected_club']:
        p,sev=res; add_news(f'🏥 {p["name"]}が{sev}（{p["injury_weeks"]}週間）','in

def _injure_weather(team,wdat):
    med=game_state['facilities']['medical']
    phys=sum(s['skill']*0.002 for s in game_state['staff'] if s['type']=='フィジオ
```

```
    cands=[p for p in team['players']+team.get('loan_in',[]) if p['injury_weeks']
    if not cands: return None
    for p in cands:
        risk=0.20 if p.get('trait')=='スペ体質' else 0.10
        risk*=(1-med*0.05-phys)*wdat.get('injury_mod',1.0); risk=max(0.02,risk)
        if random.random()<risk/len(cands):
            r=random.random()
            if r<0.60: weeks,sev=random.randint(1,2),'軽傷'
            elif r<0.85: weeks,sev=random.randint(3,8),'中傷'
            else: weeks,sev=random.randint(9,20),'重傷'
            if p.get('trait')=='スペ体質': weeks=int(weeks*1.5)
            p['injury_weeks']=weeks; p['injury_severity']=sev
            # 保険支払い
            if game_state.get('injury_insurance_active') and sev in ['中傷','重傷'
                payout=int(p['wage']*weeks*0.5); get_sel()['budget']+=payout
                add_news(f'🏥選手保険支払い: {p["name"]} {fmt_m(payout)}','club')
            return p,sev
    return None

# ============================================================
# 選手状態・ケミストリー
# ============================================================
def update_player_states(team):
    for p in team['players']+team.get('loan_in',[]):
        if p.get('state','normal')!='normal':
            p['state_weeks']=p.get('state_weeks',0)-1
            if p['state_weeks']<=0:
                p['state']='normal'; p['state_weeks']=0
        elif random.random()<0.04:
            state=random.choice(['hot','slump']); dur=random.randint(2,5)
            p['state']=state; p['state_weeks']=dur
            if team['name']==game_state['selected_club']:
                # 覚醒は直接発表、スランプは曖昧に
                if state=='hot':
                    add_news(f'⚡{p["name"]}が覚醒！絶好調が続いている','player')
                else:
                    add_news(f'😔{p["name"]}が最近やや元気がないように見える','player'
        elif random.random()<0.02 and p.get('state','normal')=='normal':
            # 予兆ニュース（実際の状態変化前）
            if team['name']==game_state['selected_club']:
                if random.random()<0.5:
                    add_news(f'👀{p["name"]}のトレーニングが最近特に目立っている','play
        # 代表帰還ブースト
        if p.get('intl_boost_pending') and p.get('intl_weeks',0)<=0:
            for k in ['SHT','TEC','MEN']: p['attrs'][k]=clamp(p['attrs'][k]+rando
            _recompute(p); p['intl_boost_pending']=False
            add_news(f'🌍{p["name"]}が代表経験で成長！','player')
```

```
        # コンバート進行
        if p.get('convert_weeks',0)>0:
            p['convert_weeks']-=1
            if p['convert_weeks']==0:
                p['pos']=p.get('target_pos',p['pos']); _recompute(p)
                add_news(f'{p["name"]}のコンバート完了 → {p["pos"]}','player')

def update_chemistry(team):
    pl=team['players']
    if not pl: return
    nat_counts={}
    for p in pl: nat_counts[p['nationality']]=nat_counts.get(p['nationality'],0)+
    dom_ratio=max(nat_counts.values())/len(pl)
    young=sum(1 for p in pl if p['age']<=24); old=sum(1 for p in pl if p['age']>=
    age_balance=1.0-(abs(young-old)/max(1,len(pl)))*0.5
    cap_bonus=5 if team.get('captain_id') else 0
    mood_bonus=sum(3 for p in pl if p.get('trait')=='ムードメーカー')
    local_bonus=sum(2 for p in pl if p.get('is_local'))
    team['chemistry']=clamp(int(50+dom_ratio*20+age_balance*10+cap_bonus+mood_bon

def update_player_feelings(team,won=False,drew=False,lost=False):
    mood_d=0
    for p in team['players']+team.get('loan_in',[]):
        d=2 if lost else(-1 if won else 0)
        pers=p.get('personality','')
        if pers=='野心家' and div_of(team['name']) in ['D2','D3'] and team['reputa
        if pers=='気分屋': d+=random.choice([-1,0,1,2])
        if pers=='忠誠心高い': d-=1
        if pers=='ムードメーカー': mood_d+=1
        if pers=='短気' and lost: d+=2
        # 役割不満（主力のみ希望なのにスタメン外）
        xi_set=set(team.get('starting_xi',[]))
        if p.get('role_wish')=='主力のみ' and xi_set and p['id'] not in xi_set: d+=
        p['unhappiness']=clamp(p.get('unhappiness',0)+d,0,100)
        p['morale']=clamp(p['morale']+(clamp(100-p['unhappiness'],0,100)-p['moral
        if p['unhappiness']>=80 and not p.get('transfer_request'):
            p['transfer_request']=True
            if team['name']==game_state['selected_club']:
                add_news(f'⚠️{p["name"]}が移籍要求（不満{p["unhappiness"]}）','trans
    cap_id=team.get('captain_id')
    if cap_id:
        cap=next((p for p in team['players'] if p['id']==cap_id),None)
        if cap: mood_d+=CAPTAIN_EFFECTS.get(cap['personality'],{}).get('locker_ro
    base_mood=100-avg([p['unhappiness'] for p in team['players']])+mood_d
    if won: base_mood+=4
    elif lost: base_mood-=4
    game_state['locker_room_mood']=clamp(int(base_mood),0,100)
```

```
def update_fan_happiness(team,won=False,drew=False,lost=False):
    d=4 if won else(1 if drew else -4)
    obj=team.get('board_expectation','')
    tbl=sorted_table(div_of(team['name']))
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==team['name']),10)
    if obj=='優勝争い': d+=2 if rank<=3 else(-2 if rank>=7 else 0)
    elif obj=='上位進出': d+=2 if rank<=5 else(-2 if rank>=8 else 0)
    elif obj in ['昇格争い','昇格']: d+=2 if rank<=3 else(-2 if rank>=7 else 0)
    elif obj=='残留': d+=1 if rank<=7 else -2
    elif obj=='中位確保': d+=1 if 4<=rank<=7 else(-2 if rank>=9 else 0)
    if game_state['club_brand']>=70: d+=1
    elif game_state['club_brand']<=30: d-=1
    game_state['fan_happiness']=clamp(game_state['fan_happiness']+d,0,100)

def update_manager_rating(team,won=False,drew=False,lost=False):
    d=3 if won else(1 if drew else -3)
    if has_skill('鉄のメンタル') and lost: d=int(d*0.5)
    tbl=sorted_table(div_of(team['name']))
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==team['name']),10)
    obj=team.get('board_expectation','')
    if obj=='優勝争い' and rank<=3: d+=2
    elif obj=='上位進出' and rank<=5: d+=2
    elif obj in ['昇格争い','昇格'] and rank<=2: d+=3
    elif obj=='残留' and rank>=9: d-=3
    elif obj=='中位確保' and rank>=9: d-=2
    if game_state['locker_room_mood']>=70: d+=1
    elif game_state['locker_room_mood']<=30: d-=2
    game_state['manager_rating']=clamp(game_state['manager_rating']+d,0,100)
    gain_exp(2 if won else(1 if drew else 0))
    if game_state['manager_rating']<=25 and game_state['board_rating']<=30 and no
        game_state['manager_fired']=True; add_news('🚨監督解任危機！','club')

def update_board_rating(team):
    tbl=sorted_table(div_of(team['name']))
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==team['name']),10)
    d=4 if rank<=3 else(1 if rank<=6 else(-4 if rank>=9 else 0))
    if team['budget']>800000: d+=1
    elif team['budget']<100000: d-=2
    game_state['board_rating']=clamp(game_state['board_rating']+d,0,100)
    team['reputation']=clamp(team.get('reputation',60)+(3 if rank<=2 else(1 if ra
    bd=2 if rank<=3 else(-1 if rank>=8 else 0)
    if team['budget']>1000000: bd+=1
    game_state['club_brand']=clamp(game_state['club_brand']+bd,0,100)
    if team['budget']<-200000 and not game_state['financial_crisis']:
        game_state['financial_crisis']=True; add_news('💸財政危機！','club')
    elif team['budget']>0 and game_state['financial_crisis']:
```

```
        game_state['financial_crisis']=False; add_news('財政が回復しました','club')
    _check_buyout(team)
    # 肖像権収入更新
    stars=sum(1 for p in team['players'] if p['overall']>=80)
    game_state['portrait_income_weekly']=stars*random.randint(5000,12000)

# ============================================================
# 収入・支出
# ============================================================
def weekly_expenses(team):
    fac=sum(game_state['facilities'].values())*5000
    wage=sum(p['wage'] for p in team['players'])
    loan_w=sum(p['wage']//2 for p in team.get('loan_in',[]))
    staff=sum(s['salary'] for s in game_state['staff'])
    # 銀行ローン返済
    loan_payment=0
    for loan in game_state['bank_loans']:
        loan['remaining_weeks']=loan.get('remaining_weeks',1)-1
        weekly=int(int(loan['amount']/loan['weeks'])*(1+loan['interest_rate']))
        loan_payment+=weekly; team['budget']-=weekly; add_sf('loan_interest',int(
    total=fac+wage+loan_w+staff
    team['budget']-=total
    add_fin('クラブ運営費',-total,'週次支出'); add_sf('wages',wage+loan_w); add_sf('
    game_state['bank_loans']=[l for l in game_state['bank_loans'] if l.get('remai
    dn=div_of(team['name']); cap=SALARY_CAP.get(dn,999999)
    if wage>cap:
        penalty=int((wage-cap)*0.1); team['budget']-=penalty
        add_news(f'⚠️給与上限超過ペナルティ: {fmt_m(penalty)}','club')

def weekly_sponsor(team):
    if not team.get('sponsor'): team['sponsor']=_gen_sponsor(team['country'],div_
    sp=team['sponsor']; inc=sp['weekly_income']
    if has_skill('財務手腕'): inc=int(inc*1.10)
    team['budget']+=inc; add_fin('スポンサー収入',inc,sp['name']); add_sf('sponsor',

def weekly_broadcast(team):
    d=div_of(team['name']); base={'D1':30000,'D2':15000,'D3':5000}.get(d,0)
    rev=int(base*(1+(team['reputation']-60)*0.01)); team['budget']+=rev; add_sf('

def weekly_merch(team):
    # グッズ収入修正（係数を0.02に）
    rev=int(team['fan_base']*game_state['club_brand']*0.02)
    team['budget']+=rev; add_sf('merch',rev)

def weekly_naming_rights(team):
    nr=team.get('naming_rights')
    if nr: team['budget']+=nr['weekly']; add_sf('naming_rights',nr['weekly'])
```

```
def weekly_portrait_income(team):
    inc=game_state.get('portrait_income_weekly',0)
    if inc>0: team['budget']+=inc; add_sf('merch',inc)

def weekly_overseas_scout(team):
    os_data=game_state.get('overseas_scout')
    if not os_data: return
    os_data['weeks_left']=os_data.get('weeks_left',4)-1
    if os_data['weeks_left']<=0:
        # 海外スカウトの結果
        country=os_data['country']
        ps=[gen_player(country,'OverseasPool',random.randint(500000,599999)) for
        for p in ps:
            sb=game_state['facilities']['scouting']*3+(3 if has_skill('スカウト眼')
            for k in p['attrs']: p['attrs'][k]=clamp(p['attrs'][k]+random.randint
            _recompute(p)
            # スカウト不確実性
            p['scout_ovr_min']=max(35,p['overall']-10)
            p['scout_ovr_max']=min(99,p['overall']+10)
        # グレード付与（スカウト施設レベルに応じて精度が上がる）
        grades=['A','B','C','D','E']
        grade_weights_by_lv={1:[5,15,35,30,15],2:[8,20,35,25,12],3:[10,25,35,20,1
        lv=min(5,game_state['facilities']['scouting'])
        for pp in ps:
            pp['scout_grade']=wc(grades,grade_weights_by_lv[lv])
        game_state['overseas_pool']=ps
        game_state['overseas_scout']=None
        add_news(f'🌍海外スカウト({country})帰還！{len(ps)}名の候補を発見','player')
        refresh_ui()

def recover_players():
    for p in all_players(True):
        if p['injury_weeks']>0:
            p['injury_weeks']-=1
            if p['injury_weeks']==0: p['injury_severity']='なし'
        if p.get('intl_weeks',0)>0: p['intl_weeks']-=1
        p['stamina']=clamp(p['stamina']+random.randint(5,12),40,100)

def apply_individual_training(team):
    tr_lv=game_state['facilities']['training']; results=[]
    for p in team['players']:
        focus=p.get('training_focus')
        if not focus or focus not in p['attrs']: continue
        if random.random()<(0.3+tr_lv*0.1):
            p['attrs'][focus]=clamp(p['attrs'][focus]+1,30,99); _recompute(p)
            results.append(f'{p["name"]}の{ATTR_ICONS.get(focus,focus)}{focus}が+1
```

```
    if results: game_state['training_results']=results[-5:]

def check_retirements(team):
    for p in list(team['players']):
        # 引退の兆候（実際の発表前に示唆ニュース）
        if p['age']>=34 and not p.get('retiring') and not p.get('retire_hint') an
            p['retire_hint']=True
            add_news(f'💬{p["name"]}（{p["age"]}歳）が引退についてインタビューで言及','
        if p['age']>=35 and not p.get('retiring') and random.random()<(p['age']-3
            p['retiring']=True; add_news(f'💐{p["name"]}が今季限りの引退を正式発表（{p
        if p.get('retiring') and p['age']>=36 and random.random()<0.6:
            team['players'].remove(p)
            score=p['stats']['goals']*3+p['stats']['assists']*2+p['stats']['mom']
            game_state['club_hall_of_fame'].append({'name':p['name'],'score':scor
            add_news(f'🎖️{p["name"]}が現役引退','player')

# ==========================================================
# ランダムイベント
# ==========================================================
def gen_random_event():
    sel=get_sel()
    if not sel or random.random()>0.20: return
    if game_state.get('pending_event'): return
    evt_template=random.choice(RANDOM_EVENTS)
    # プレースホルダー埋め
    player=random.choice(sel['players']) if sel['players'] else None
    rival=sel['rivals'][0] if sel.get('rivals') else 'ライバル'
    text=evt_template['text'].replace('{name}',player['name'] if player else '選手
    event={'id':evt_template['id'],'text':text,'choices':evt_template['choices'],
    game_state['pending_event']=event
    add_news(f'📌イベント発生: {text}','club')

def resolve_event(choice):
    evt=game_state.get('pending_event')
    if not evt: return
    effs=evt['effects'].get(choice,{})
    sel=get_sel()
    if not sel: return
    pid=evt.get('player_id')
    p=player_by_id(pid) if pid else None
    for k,v in effs.items():
        if k=='fan_happiness': game_state['fan_happiness']=clamp(game_state['fan_
        elif k=='locker_room_mood': game_state['locker_room_mood']=clamp(game_sta
        elif k=='manager_rating': game_state['manager_rating']=clamp(game_state['
        elif k=='board_rating': game_state['board_rating']=clamp(game_state['boar
        elif k=='club_brand': game_state['club_brand']=clamp(game_state['club_bra
        elif k=='fan_base' and sel: sel['fan_base']+=v
```

```
        elif k=='budget' and sel: sel['budget']+=v; add_fin('イベント支出',v,evt['t
        elif k=='morale_player' and p: p['morale']=clamp(p.get('morale',50)+v,0,1
        elif k=='unhappiness_player' and p: p['unhappiness']=clamp(p.get('unhappi
        elif k=='attr_boost' and p:
            attr=random.choice(list(p['attrs'].keys())); p['attrs'][attr]=clamp(p
        elif k=='injury_minor' and p: p['injury_weeks']=max(p['injury_weeks'],ran
        elif k=='new_staff':
            game_state['staff'].append({'type':'ヘッドコーチ','skill':random.randin
        elif k=='transfer_youth' and p:
            if sel and p in sel['youth']: sel['youth'].remove(p); sel['budget']+=
    game_state['event_history'].append({'text':evt['text'],'choice':choice,'seaso
    game_state['pending_event']=None
    add_news(f'イベント対応:「{choice}」','club'); refresh_ui()

# ============================================================
# 銀行ローン・保険
# ============================================================
def take_bank_loan(amount):
    sel=get_sel()
    if not sel: return
    weeks=random.choice([12,18,24]); rate=random.choice([0.03,0.05,0.08])
    loan={'amount':amount,'weeks':weeks,'interest_rate':rate,'remaining_weeks':we
    game_state['bank_loans'].append(loan); sel['budget']+=amount
    add_fin('銀行ローン',amount,f'返済{weeks}週 金利{int(rate*100)}%'); add_news(f'銀

def activate_insurance():
    sel=get_sel()
    if not sel: return
    cost=8000
    if sel['budget']<cost: ui.notify(T('予算不足')); return
    sel['budget']-=cost; game_state['injury_insurance_active']=True
    add_fin('選手保険料',-cost,'週次'); add_news('🏥選手保険を契約（重傷時に給与補償）','

# ============================================================
# 監督キャリア
# ============================================================
def gen_manager_offers():
    """高評価時に他クラブからオファー"""
    if game_state['manager_rating']<70: return
    if game_state.get('manager_career_offers'): return
    candidates=[]
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==game_state['selected_club']: continue
            if t['reputation']>get_sel().get('reputation',50) and random.random()
                candidates.append(t['name'])
    if candidates:
```

```
        game_state['manager_career_offers']=candidates[:2]
        add_news(f'📩監督オファー: {", ".join(candidates[:2])}','club')

def accept_manager_offer(new_club_name):
    sel=get_sel()
    if not sel: return
    dn=div_of(sel['name']); tbl=sorted_table(dn)
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==sel['name']),10)
    # キャリア記録
    game_state['manager_career'].append({'club':sel['name'],'season':game_state['
    game_state['selected_club']=new_club_name
    game_state['manager_career_offers']=[]
    # 評価リセット（新クラブで再スタート）
    game_state['manager_rating']=50; game_state['board_rating']=50
    new_t=find_team(new_club_name)
    if new_t and not new_t.get('sponsor'):
        new_t['sponsor']=_gen_sponsor(new_t['country'],div_of(new_club_name))
    check_achievement('career_move')
    add_news(f'✈️{new_club_name}に転身！','club'); refresh_ui()

# ============================================================
# プレシーズン
# ============================================================
def run_preseason():
    sel=get_sel()
    if not sel: return
    results=[]
    for _ in range(3):
        opp=random.choice([t for d in game_state['divisions'] for t in game_state
        ga,gb,*_=simulate_match_full(sel,opp)
        results.append(f'{sel["name"]} {ga}-{gb} {opp["name"]}')
        for p in sel['players']: p['stamina']=clamp(p['stamina']+random.randint(5
    income=random.randint(50000,150000); sel['budget']+=income; add_sf('preseason
    game_state['preseason_phase']=False; sel['preseason_done']=True
    add_news(f'プレシーズン完了！収入: {fmt_m(income)}','club')
    for r in results: add_news(f'親善試合: {r}','match')
    refresh_ui()

# ============================================================
# スポンサー
# ============================================================
def _gen_sponsor(country,div):
    bw={'D1':65000,'D2':42000,'D3':26000}[div]; bb={'D1':220000,'D2':150000,'D3':
    pool={'England':['NorthSea Media','Union Logistics','Royal Finance','Harbor T
    bm=1.0+(game_state.get('club_brand',50)-50)*0.005
    return {'name':random.choice(pool[country]),'weekly_income':int(bw*random.uni
```

```
def gen_sponsor_nego():
    sel=get_sel()
    if not sel: return
    offer=_gen_sponsor(sel['country'],div_of(sel['name'])); demand=random.choice(
    game_state['pending_sponsor_negotiation']={'offer':offer,'demand':demand}
    add_news(f'スポンサー交渉: {offer["name"]}が「{demand}」を要求','club')

def accept_sponsor():
    sel=get_sel(); data=game_state.get('pending_sponsor_negotiation')
    if not sel or not data: return
    o,dem=data['offer'],data['demand']
    if dem=='若手起用': o['weekly_income']=int(o['weekly_income']*0.95); o['season_
    elif dem=='上位進出': o['target_rank']=max(1,o['target_rank']-1); o['season_bo
    elif dem=='堅実経営': o['weekly_income']=int(o['weekly_income']*1.05)
    elif dem=='観客動員増': o['win_bonus']=int(o['win_bonus']*1.15)
    sel['sponsor']=o; game_state['pending_sponsor_negotiation']=None
    add_news(f'スポンサー契約成立: {o["name"]}','club'); refresh_ui()

def reject_sponsor():
    data=game_state.get('pending_sponsor_negotiation')
    if data: game_state['pending_sponsor_negotiation']=None; add_news(f'スポンサー

# ============================================================
# プレス・代表召集・エージェント
# ============================================================
def do_press(choice):
    effs={'強気発言':{'fan_happiness':3,'locker_room_mood':4,'manager_rating_risk'
    eff=effs.get(choice,{})
    game_state['fan_happiness']=clamp(game_state['fan_happiness']+eff.get('fan_ha
    game_state['locker_room_mood']=clamp(game_state['locker_room_mood']+eff.get('
    game_state['board_rating']=clamp(game_state['board_rating']+eff.get('board_ra
    sel=get_sel()
    if sel and eff.get('morale_all'):
        for p in sel['players']: p['morale']=clamp(p['morale']+eff['morale_all'],
    if eff.get('manager_rating_risk') and random.random()<0.3:
        game_state['manager_rating']=clamp(game_state['manager_rating']+eff['mana
    game_state['pending_press']=False; add_news(f'📰プレス発言:「{choice}」','club'

def process_intl_calls(team):
    if random.random()>0.15: return
    cands=[p for p in team['players'] if p['overall']>=68 and p.get('intl_weeks',
    if not cands: return
    p=random.choice(cands); weeks=random.randint(2,4); p['intl_weeks']=weeks
    p['intl_boost_pending']=True  # 帰還後に成長ブースト
    add_news(f'🌍{p["name"]}が代表召集（{weeks}週間）','player')

def agent_intervention(team):
```

```
    if random.random()>0.12: return
    agents=[p for p in team['players'] if p.get('has_agent') and not p.get('trans
    if not agents: return
    p=random.choice(agents)
    add_news(f'🤝{p["name"]}のエージェントが介入。移籍金+{p["agent_fee_pct"]}%要求','t

# ============================================================
# 理事会・ネーミングライツ・買収
# ============================================================
def trigger_board_meeting():
    sel=get_sel()
    if not sel: return
    dn=div_of(sel['name']); tbl=sorted_table(dn)
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==sel['name']),10)
    mood='満足' if rank<=4 else('不満' if rank>=8 else '普通')
    add_news(f'📋理事会ミーティング（現状:{mood} {rank}位）','club')
    events=[('追加補強資金',lambda: sel.update({'budget':sel['budget']+200000}) or
    evt=random.choice(events); add_news(f'理事会決定: {evt[0]}','club'); evt[1]()
    # 次回理事会は中盤頃（年2回）
    sel['board_meeting_week']=game_state['week']+18 if game_state['week']<18 else
    game_state['pending_board_meeting']=False

def offer_naming_rights():
    sel=get_sel()
    if not sel or sel.get('naming_rights'): return
    companies=['TechCorp Arena','MegaBank Stadium','AutoGroup Park','AirlineOne F
    upfront=random.randint(500000,1500000); weekly=random.randint(15000,40000)
    game_state['naming_rights_offer']={'company':random.choice(companies),'upfron
    add_news(f'💼ネーミングライツ打診 一時金{fmt_m(upfront)}/週{fmt_m(weekly)}','club

def accept_naming_rights():
    sel=get_sel(); offer=game_state.get('naming_rights_offer')
    if not sel or not offer: return
    sel['budget']+=offer['upfront']; sel['naming_rights']={'company':offer['compa
    add_fin('ネーミングライツ',offer['upfront'],offer['company']); add_sf('naming_ri
    game_state['fan_happiness']=clamp(game_state['fan_happiness']-3,0,100)
    game_state['club_brand']=clamp(game_state['club_brand']+4,0,100)
    game_state['naming_rights_offer']=None; add_news(f'ネーミングライツ契約成立','clu

def reject_naming_rights():
    game_state['naming_rights_offer']=None; refresh_ui()

def _check_buyout(team):
    if game_state.get('buyout_offer'): return
    if team['name']!=game_state['selected_club']: return
    if (game_state['club_brand']>=80 or game_state['financial_crisis']) and rando
        offer_type='富豪' if game_state['club_brand']>=80 else '再建型'
```

```
        boost=random.randint(2000000,5000000) if offer_type=='富豪' else random.ra
        game_state['buyout_offer']={'type':offer_type,'budget_boost':boost,'board
        add_news(f'💰オーナー{offer_type}買収オファー！補強資金+{fmt_m(boost)}','club'

def accept_buyout():
    offer=game_state.get('buyout_offer'); sel=get_sel()
    if not offer or not sel: return
    sel['budget']+=offer['budget_boost']; add_fin('買収資金',offer['budget_boost'],
    if offer.get('board_control_loss'): game_state['board_rating']=clamp(game_sta
    game_state['club_brand']=clamp(game_state['club_brand']+8,0,100)
    game_state['buyout_offer']=None; add_news(f'オーナー{offer["type"]}買収受諾！','

def reject_buyout():
    game_state['buyout_offer']=None; refresh_ui()

def set_budget_allocation(tp,fp,sp):
    sel=get_sel()
    if not sel: return
    total=sel['budget']
    sel['budget_transfer']=int(total*tp/100); sel['budget_facility']=int(total*fp
    add_news(f'予算配分: 移籍{tp}%/施設{fp}%/スタッフ{sp}%','club'); refresh_ui()

# ============================================================
# 移籍・レンタル・スカウト
# ============================================================
def renew_contract(pid):
    sel=get_sel()
    if not sel: return
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if not p: ui.notify('Player not found' if LANG=='en' else '選手なし'); return
    nw=int(p['wage']*random.uniform(1.10,1.25)); cost=nw*8
    if sel['budget']<cost: ui.notify(T('予算不足')); return
    sel['budget']-=cost; add_fin('契約更改',-cost,p['name']); add_sf('transfers_in
    p['wage']=nw; p['contract_years']=random.randint(2,5)
    p['unhappiness']=max(0,p['unhappiness']-20); p['transfer_request']=False
    add_news(f'{p["name"]}と契約延長 年俸{fmt_m(nw)}/{p["contract_years"]}年','club'

def process_expiry():
    sel=get_sel()
    if not sel: return
    stay=[]; go_players=[]
    for p in sel['players']:
        p['contract_years']-=1
        if p['contract_years']<=0: go_players.append(p)
        else: stay.append(p)
    if len(stay)>=11:
        sel['players']=stay
```

```
        for p in go_players:
            add_news(f'{p["name"]}が契約満了退団','transfer')
            # FAとして市場に登録（OVR60以上のみ）
            if p['overall']>=60:
                p['fa_wage_demand']=int(p['wage']*random.uniform(1.1,1.4))  # FA選
                p['fa_weeks']=random.randint(2,6)  # 何週間市場に残るか
                p['club']='フリーエージェント'
                game_state.setdefault('fa_market',[]).append(p)
                add_news(f'🆓{p["name"]}がフリーエージェントに（OVR{p["overall"]}）',
    else:
        for p in sel['players']:
            if p['contract_years']<=0: p['contract_years']=1
        add_news('契約満了選手を人数不足のため暫定残留','club')

    # CPU各クラブの契約切れ選手もFA市場へ
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==game_state['selected_club']: continue
            for p in list(t['players']):
                if p.get('contract_years',1)<=0 and p['overall']>=58 and random.r
                    p['fa_wage_demand']=int(p['wage']*random.uniform(1.05,1.3))
                    p['fa_weeks']=random.randint(2,5)
                    p['club']='フリーエージェント'
                    t['players'].remove(p)
                    game_state.setdefault('fa_market',[]).append(p)

def gen_domestic_offers():
    sel=get_sel()
    if not sel: return
    offers=[]
    for p in sel['players']:
        if random.random()<0.12 and p['overall']>=60:
            buyer=random.choice([t for d in game_state['divisions'] for t in game
            offers.append({'player_id':p['id'],'player_name':p['name'],'buyer':bu
    game_state['transfer_offers']=offers

def accept_transfer(pid,buyback=False):
    if not in_transfer_window(): ui.notify(T('移籍ウィンドウ外')); return
    sel=get_sel()
    if not sel: return
    o=next((x for x in game_state['transfer_offers'] if x['player_id']==pid),None
    if not o: return
    if len(sel['players'])<=11: ui.notify(T('最低11人')); return
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if not p: return
    fee=o.get('counter_fee') or o['fee']
    if p.get('has_agent'): agent_cut=int(fee*p['agent_fee_pct']/100); fee-=agent_
```

```
    sel['players']=[x for x in sel['players'] if x['id']!=pid]
    sel['budget']+=fee; add_fin('移籍売却',fee,p['name']); add_sf('transfers_out',
    if buyback: p['buyback_fee']=int(fee*1.5); p['buyback_club']=sel['name']
    game_state['transfer_offers']=[x for x in game_state['transfer_offers'] if x[
    add_news(f'{p["name"]}を{o["buyer"]}に{fmt_m(fee)}で売却','transfer'); refresh_

def counter_offer(pid):
    o=next((x for x in game_state['transfer_offers'] if x['player_id']==pid),None
    if not o: return
    o['counter_fee']=int(o['fee']*1.3)
    if random.random()<0.6: add_news(f'カウンター成立: {fmt_m(o["counter_fee"])}','
    else: game_state['transfer_offers']=[x for x in game_state['transfer_offers']
    refresh_ui()

def reject_transfer(pid):
    game_state['transfer_offers']=[x for x in game_state['transfer_offers'] if x[

def gen_foreign_offer():
    sel=get_sel()
    if not sel or not sel['players'] or random.random()>0.25: return
    p=random.choice(sel['players'])
    if p['overall']<65: return
    game_state['foreign_offers'].append({'player_id':p['id'],'club':random.choice
    add_news(f'海外クラブが{p["name"]}にオファー','transfer')

def accept_foreign(pid):
    if not in_transfer_window(): ui.notify(T('移籍ウィンドウ外')); return
    sel=get_sel()
    if not sel: return
    o=next((x for x in game_state['foreign_offers'] if x['player_id']==pid),None)
    if not o: return
    if len(sel['players'])<=11: ui.notify(T('最低11人')); return
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if not p: return
    sel['players']=[x for x in sel['players'] if x['id']!=pid]
    sel['budget']+=o['fee']; add_fin('海外移籍売却',o['fee'],p['name']); add_sf('tr
    game_state['foreign_offers']=[x for x in game_state['foreign_offers'] if x['p
    game_state['club_brand']=clamp(game_state['club_brand']+3,0,100)
    add_news(f'{p["name"]}を海外{o["club"]}に{fmt_m(o["fee"])}で売却','transfer'); 

def reject_foreign(pid):
    game_state['foreign_offers']=[x for x in game_state['foreign_offers'] if x['p

def gen_loan_offers():
    sel=get_sel()
    if not sel: return
    offers=[]
```

```
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==sel['name']: continue
            for p in t['players']:
                if p.get('loan_club') or random.random()>0.04 or p['overall']<58:
                offers.append({'player_id':p['id'],'player_name':p['name'],'playe
    random.shuffle(offers); game_state['loan_offers']=offers[:3]

def accept_loan_in(pid):
    sel=get_sel()
    if not sel: return
    o=next((x for x in game_state['loan_offers'] if x['player_id']==pid),None)
    if not o: return
    if len(sel['players'])+len(sel.get('loan_in',[]))>=MAX_SQUAD: ui.notify(T('ス
    if sel['budget']<o['loan_fee']: ui.notify(T('予算不足')); return
    src=find_team(o['from_club'])
    if not src: ui.notify('Origin club not found' if LANG=='en' else '元クラブが見つ
    p=next((x for x in src['players'] if x['id']==pid),None)
    if not p: return
    sel['budget']-=o['loan_fee']; add_fin('レンタル借用料',-o['loan_fee'],p['name'])
    p['loan_club']=sel['name']; p['loan_origin']=src['name']; p['loan_weeks']=o['
    src['players']=[x for x in src['players'] if x['id']!=pid]; sel.setdefault('l
    game_state['loan_offers']=[x for x in game_state['loan_offers'] if x['player_
    add_news(f'レンタル加入: {p["name"]}（{o["from_club"]}から{o["weeks"]}週間）','tr

def reject_loan_in(pid):
    game_state['loan_offers']=[x for x in game_state['loan_offers'] if x['player_

def send_on_loan(pid):
    sel=get_sel()
    if not sel: return
    if len(sel['players'])<=11: ui.notify(T('最低11人')); return
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if not p: return
    dests=[t for d in game_state['divisions'] for t in game_state['divisions'][d]
    if not dests: ui.notify(T('受け入れ先なし')); return
    dest=random.choice(dests); weeks=random.choice([4,8,12]); inc=int(p['wage']*r
    sel['players']=[x for x in sel['players'] if x['id']!=pid]; sel['budget']+=in
    p['loan_club']=dest['name']; p['loan_origin']=sel['name']; p['loan_weeks']=we
    dest.setdefault('loan_in',[]).append(p)
    add_news(f'レンタル放出: {p["name"]}→{dest["name"]}（{weeks}週間 収入{fmt_m(inc)}

def recall_loan(pid):
    sel=get_sel()
    if not sel: return
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
```

```
            for p in t.get('loan_in',[]):
                if p['id']==pid and p.get('loan_origin')==sel['name']:
                    if len(sel['players'])>=MAX_SQUAD: ui.notify(T('スクワッド満員'
                    t['loan_in']=[x for x in t['loan_in'] if x['id']!=pid]
                    p['loan_club']=None; p['loan_origin']=None; p['loan_weeks']=0
                    sel['players'].append(p); add_news(f'レンタル召還: {p["name"]}'

def process_loan_returns():
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            ret=[]; stay=[]
            for p in t.get('loan_in',[]):
                p['loan_weeks']=max(0,p['loan_weeks']-1)
                if p['loan_weeks']<=0: ret.append(p)
                else: stay.append(p)
            t['loan_in']=stay
            for p in ret:
                orig=find_team(p.get('loan_origin',''))
                p['loan_club']=None; p['loan_origin']=None
                if orig and len(orig['players'])<MAX_SQUAD: orig['players'].appen

def loaned_out():
    sel=get_sel()
    if not sel: return []
    return [(p,t['name']) for d in game_state['divisions'] for t in game_state['d

def settle_request(pid):
    sel=get_sel()
    if not sel: return
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if not p: return
    cost=int(p['wage']*4)
    if sel['budget']<cost: ui.notify(T('予算不足')); return
    sel['budget']-=cost; p['unhappiness']=max(0,p['unhappiness']-40)
    p['transfer_request']=False; p['wage']=int(p['wage']*1.15)
    add_news(f'{p["name"]}と交渉成立、移籍要求撤回','transfer'); refresh_ui()

def set_captain(pid):
    sel=get_sel()
    if not sel: return
    sel['captain_id']=pid
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if p: add_news(f'©{p["name"]}をキャプテンに任命','club')
    refresh_ui()

def set_pk_taker(pid):
    sel=get_sel()
```

```
    if not sel: return
    sel['pk_taker_id']=pid
    p=next((x for x in sel['players'] if x['id']==pid),None)
    if p: add_news(f'⚽{p["name"]}をPKシューターに指定','club')
    refresh_ui()

def convert_position(pid,target_pos):
    """ポジション変更/コンバート"""
    p=player_by_id(pid)
    if not p: return
    p['target_pos']=target_pos; p['convert_weeks']=6  # 6週間でコンバート完了
    add_news(f'🔄{p["name"]}を{target_pos}にコンバート中（6週間）','player'); refresh

def dispatch_overseas_scout(country):
    """海外スカウト派遣"""
    sel=get_sel()
    if not sel: return
    cost=50000
    if sel['budget']<cost: ui.notify(T('予算不足')); return
    sel['budget']-=cost; add_fin('海外スカウト派遣',-cost,country)
    game_state['overseas_scout']={'country':country,'weeks_left':4}
    add_news(f'🌍{country}に海外スカウトを派遣（4週間後に帰還）','player'); refresh_ui(

def sign_overseas_player(pid):
    """海外スカウト発掘選手の獲得"""
    sel=get_sel()
    if not sel: return
    p=next((x for x in game_state.get('overseas_pool',[]) if x['id']==pid),None)
    if not p: return
    if not in_transfer_window(): ui.notify(T('移籍ウィンドウ外')); return
    if sel['budget']<p['value']: ui.notify(T('予算不足')); return
    sel['budget']-=p['value']; add_fin('移籍加入',-p['value'],p['name']); add_sf('
    p['club']=sel['name']; sel['players'].append(p)
    game_state['overseas_pool']=[x for x in game_state['overseas_pool'] if x['id'
    add_news(f'海外選手獲得: {p["name"]}を{fmt_m(p["value"])}で','transfer'); refres

def create_scout_pool():
    sel=get_sel(); country=game_state['selected_country'] if not sel else sel['co
    pool=[]
    for i in range(10):
        p=gen_player(country,'ScoutPool',900000+i)
        sb=game_state['facilities']['scouting']*2+(3 if has_skill('スカウト眼') els
        sk=sum(s['skill']//10 for s in game_state['staff'] if s['type']=='スカウト
        for k in p['attrs']: p['attrs'][k]=clamp(p['attrs'][k]+random.randint(0,s
        _recompute(p)
        # スカウト不確実性: 実力の±10%の幅
        p['scout_ovr_min']=max(35,p['overall']-8); p['scout_ovr_max']=min(99,p['o
```

```
        pool.append(p)
    game_state['scout_pool']=pool; refresh_ui()

def sign_fa(pid):
    """フリーエージェント選手を獲得"""
    sel=get_sel()
    if not sel: return
    p=next((x for x in game_state.get('fa_market',[]) if x['id']==pid),None)
    if not p: ui.notify('Player not found' if LANG=='en' else '選手が見つかりません'
    if len(sel['players'])>=MAX_SQUAD: ui.notify(T('スクワッド満員')); return
    fa_wage=p.get('fa_wage_demand',p.get('wage',10000))
    wage_cost=fa_wage*8  # 違約金なし・サイン費用のみ
    if sel['budget']<wage_cost: ui.notify(f'Insufficient funds (Sign fee: {fmt_m(
    sel['budget']-=wage_cost
    add_fin('FA獲得サイン費',-wage_cost,p['name']); add_sf('transfers_in',wage_cost
    p['wage']=fa_wage; p['club']=sel['name']
    p['contract_years']=random.randint(1,3); p['unhappiness']=0
    sel['players'].append(p)
    game_state['fa_market']=[x for x in game_state.get('fa_market',[]) if x['id']
    add_news(f'🆓FA獲得: {p["name"]} OVR{p["overall"]} 年俸{fmt_m(p["wage"])}','tr
    refresh_ui()


# ============================================================
# トライアウトシステム
# ============================================================
def gen_tryout_pool():
    """トライアウト候補を生成（属性は隠蔽）"""
    sel=get_sel()
    if not sel: return
    country=sel['country']
    sc_lv=game_state['facilities']['scouting']
    # スカウト施設Lvが高いほど質が良い
    ovr_min=45+sc_lv*2; ovr_max=58+sc_lv*3
    count=random.randint(5,8)
    pool=[]
    for i in range(count):
        p=gen_player(country,'トライアウト',700000+i)
        p['age']=random.randint(17,23)
        # OVRを施設Lvに応じた範囲に調整
        target_ovr=random.randint(ovr_min,ovr_max)
        diff=target_ovr-p['overall']
        for k in p['attrs']:
            p['attrs'][k]=clamp(p['attrs'][k]+diff,30,85)
        _recompute(p)
        p['potential']=clamp(p['overall']+random.randint(8,25),p['overall'],92)
        # 印象ヒント（スカウト施設Lvで精度が変わる）
```

```
        if sc_lv>=4:
            hint_accurate=True
        elif sc_lv>=2:
            hint_accurate=random.random()<0.6
        else:
            hint_accurate=random.random()<0.3
        actual_good=(p['overall']>=55 or p['potential']>=72)
        if hint_accurate:
            p['tryout_hint']='印象◎ 光るものがある' if actual_good else '印象△ 可もな‹
        else:
            # スカウト精度低いと逆の印象になることも
            p['tryout_hint']='印象◎ 光るものがある' if not actual_good else '印象△ 可
        # 特別ヒント（稀に）
        if random.random()<0.15:
            p['tryout_hint']='印象◎◎ 際立った才能を感じる'
        pool.append(p)
    game_state['tryout_pool']=pool
    add_news(f'⚽トライアウト開催！{count}名が参加','club')
    refresh_ui()

def sign_tryout(pid):
    """トライアウト選手と契約"""
    sel=get_sel()
    if not sel: return
    p=next((x for x in game_state['tryout_pool'] if x['id']==pid),None)
    if not p: ui.notify('Player not found' if LANG=='en' else '選手が見つかりません'
    if len(sel['players'])>=MAX_SQUAD: ui.notify(T('スクワッド満員')); return
    # 契約済み人数チェック（最大2名）game_stateで管理
    signed_count=game_state.get('tryout_signed_count',0)
    if signed_count>=2: ui.notify(T('トライアウト2名制限')); return
    sign_cost=int(p['wage']*4)  # サイン費用（移籍金なし）
    if sel['budget']<sign_cost: ui.notify(f'Insufficient funds (Sign fee: {fmt_m(
    sel['budget']-=sign_cost; add_fin('トライアウト契約',-sign_cost,p['name']); add_
    p['club']=sel['name']; p['contract_years']=random.randint(1,3)
    game_state['tryout_signed_count']=game_state.get('tryout_signed_count',0)+1
    sel['players'].append(p)
    game_state['tryout_pool']=[x for x in game_state['tryout_pool'] if x['id']!=p
    # 契約後に実力が判明
    add_news(f'✍️トライアウト契約: {p["name"]} {p["pos"]} OVR{p["overall"]} POT隠（サ
    refresh_ui()

def dismiss_tryout():
    """トライアウト終了（残った選手を全員解散）"""
    game_state['tryout_pool']=[]
    game_state['tryout_done']=True
    add_news('トライアウト終了','club')
    refresh_ui()
```

```
def sign_scout(pid):
    sel=get_sel()
    if not sel: return
    p=next((x for x in game_state.get('scout_pool',[]) if x['id']==pid),None)
    if not p: return
    if len(sel['players'])>=MAX_SQUAD: ui.notify(T('スクワッド満員')); return
    if not in_transfer_window(): ui.notify(T('移籍ウィンドウ外')); return
    if sel['budget']<p['value']: ui.notify(T('予算不足')); return
    sel['budget']-=p['value']; add_fin('移籍加入',-p['value'],p['name']); add_sf('
    p['club']=sel['name']; sel['players'].append(p)
    game_state['scout_pool']=[x for x in game_state['scout_pool'] if x['id']!=pid
    add_news(f'{p["name"]}を{fmt_m(p["value"])}で獲得','transfer'); refresh_ui()

def youth_text(p):
    diff=p['potential']-p['overall']
    if p['potential']>=80 and diff>=10: return 'かなり有望'
    elif p['potential']>=72: return '伸びしろ十分'
    elif p['potential']>=65: return '将来性あり'
    return '即戦力ではない'

def youth_rec(p):
    if p['age']>=19: return '昇格/放出必須'
    if p['age']>=18 and p['overall']>=60: return '昇格推奨'
    if p['potential']-p['overall']>=12: return '育成推奨'
    return '様子見'

def refresh_youth_queue():
    t=get_sel()
    if t: game_state['youth_decision_queue']=[p['id'] for p in t['youth'] if p['a

def promote_youth(pid):
    t=get_sel()
    if not t: return
    p=next((x for x in t['youth'] if x['id']==pid),None)
    if not p: return
    if len(t['players'])>=MAX_SQUAD: ui.notify(T('スクワッド満員')); return
    t['youth']=[x for x in t['youth'] if x['id']!=pid]; p['club']=t['name']; p['c
    t['players'].append(p)
    game_state['youth_decision_queue']=[x for x in game_state['youth_decision_que
    game_state['youth_promoted_count']=game_state.get('youth_promoted_count',0)+1
    if game_state['youth_promoted_count']>=5: check_achievement('youth_promoted')
    add_news(f'ユース昇格: {p["name"]}','player'); refresh_ui()

def retain_youth(pid):
    t=get_sel()
```

```
    if not t: return
    p=next((x for x in t['youth'] if x['id']==pid),None)
    if not p or p['age']>=19: ui.notify(T('最低11人youth')); return
    p['contract_years']=max(p.get('contract_years',0),1)
    game_state['youth_decision_queue']=[x for x in game_state['youth_decision_que
    add_news(f'ユース残留: {p["name"]}','player'); refresh_ui()

def release_youth(pid):
    t=get_sel()
    if not t: return
    p=next((x for x in t['youth'] if x['id']==pid),None)
    if not p: return
    t['youth']=[x for x in t['youth'] if x['id']!=pid]
    game_state['youth_decision_queue']=[x for x in game_state['youth_decision_que
    add_news(f'ユース放出: {p["name"]}','player'); refresh_ui()

def refill_youth(team):
    target=random.randint(MIN_YOUTH,MAX_YOUTH)
    while len(team['youth'])<target:
        p=gen_player(team['country'],team['name'],random.randint(100000,999999),T
        p['age']=16; p['contract_years']=random.randint(2,4); team['youth'].appen
        if p['is_wonderkid'] and team['name']==game_state.get('selected_club'):
            game_state['wonderkid_found']=game_state.get('wonderkid_found',0)+1
            if game_state['wonderkid_found']>=3: check_achievement('wonder_scout'
            add_news(f'🌟世界的逸材加入: {p["name"]}(POT {p["potential"]})','player

def poach_youth():
    sel=get_sel()
    if not sel or len(sel['youth'])>=MAX_YOUTH: ui.notify(T('枠満員')); return
    srcs=[t for d in game_state['divisions'] for t in game_state['divisions'][d]
    if not srcs: return
    src=random.choice(srcs); cand=max(src['youth'],key=lambda p:p['potential'])
    fee=max(20000,int(cand['value']*0.35))
    if sel['budget']<fee: ui.notify(T('予算不足')); return
    sel['budget']-=fee; add_fin('ユース引き抜き',-fee,cand['name']); add_sf('transfe
    src['youth']=[x for x in src['youth'] if x['id']!=cand['id']]
    cand['club']=sel['name']; cand['contract_years']=random.randint(2,4); sel['yo
    add_news(f'ユース引き抜き: {cand["name"]} ({src["name"]}から{fmt_m(fee)}) ','tran

def set_youth_policy(pol):
    game_state['youth_policy']=pol; add_news(f'育成方針→{pol}','club'); refresh_ui

# ============================================================
# 施設・スタッフ
# ============================================================
def upgrade_facility(ft):
    sel=get_sel()
```

```
    if not sel: return
    lv=game_state['facilities'][ft]
    if lv>=5: ui.notify(T('最大Lv')); return
    cost=150000*lv
    if sel['budget']<cost: ui.notify(T('資金不足')); return
    sel['budget']-=cost; game_state['facilities'][ft]+=1; add_fin('施設投資',-cost,
    game_state['club_brand']=clamp(game_state['club_brand']+1,0,100)
    if all(v>=5 for v in game_state['facilities'].values()): check_achievement('m
    add_news(f'{ft}→Lv{game_state["facilities"][ft]}に強化','club'); refresh_ui()

def upgrade_youth_fac():
    sel=get_sel()
    if not sel: return
    lv=game_state['facilities']['youth']
    if lv>=5: ui.notify(T('最大Lv')); return
    cost=200000*lv
    if sel['budget']<cost: ui.notify(T('資金不足')); return
    sel['budget']-=cost; game_state['facilities']['youth']+=1; add_fin('施設投資',-
    add_news(f'ユース施設→Lv{game_state["facilities"]["youth"]}','club'); refresh_u

def expand_stadium():
    sel=get_sel()
    if not sel: return
    inc=random.randint(3000,8000); cost=inc*35
    if sel['budget']<cost: ui.notify(T('資金不足')); return
    sel['budget']-=cost; sel['stadium_capacity']+=inc; add_fin('スタジアム拡張',-cos
    game_state['club_brand']=clamp(game_state['club_brand']+2,0,100)
    add_news(f'スタジアム拡張+{inc}席','club'); refresh_ui()

def upgrade_vip():
    sel=get_sel()
    if not sel: return
    lv=sel.get('vip_level',0)
    if lv>=3: ui.notify(T('VIP最大')); return
    cost=300000*(lv+1)
    if sel['budget']<cost: ui.notify(T('資金不足')); return
    sel['budget']-=cost; sel['vip_level']=lv+1; add_fin('VIP席投資',-cost,'建設');
    add_news(f'VIPエリアLv{sel["vip_level"]}に拡充','club'); refresh_ui()

def hire_staff():
    sel=get_sel()
    if not sel or sel['budget']<60000: ui.notify(T('予算不足')); return
    s={'type':random.choice(STAFF_TYPES),'skill':random.randint(50,90),'salary':r
    sel['budget']-=60000; game_state['staff'].append(s); add_fin('スタッフ雇用',-60
    add_news(f'スタッフ加入: {s["type"]}(skill {s["skill"]})','club'); refresh_ui()

def set_tactic(tn):
```

```
    t=get_sel()
    if t: t['tactic']=tn; add_news(f'戦術→{tn}','club'); refresh_ui()

def set_formation(fn):
    t=get_sel()
    if t: t['formation']=fn; add_news(f'フォーメーション→{fn}','club'); refresh_ui()

def learn_skill(sk):
    if game_state.get('manager_skill_points',0)<=0: ui.notify(T('SPなし')); returr
    if sk in game_state.get('manager_skills',[]): ui.notify(T('習得済み')); return
    game_state.setdefault('manager_skills',[]).append(sk); game_state['manager_sk
    add_news(f'📚監督スキル:「{sk}」習得','club'); refresh_ui()

def set_training_focus(pid,focus):
    p=player_by_id(pid)
    if p: p['training_focus']=focus; add_news(f'{p["name"]}のトレーニング集中: {focu

def set_role_wish(pid,role):
    p=player_by_id(pid)
    if p: p['role_wish']=role; add_news(f'{p["name"]}の希望役割: {role}','player');

# ============================================================
# CPU管理
# ============================================================
def cpu_contracts(t):
    for p in list(t['players']):
        p['contract_years']-=1
        if p['contract_years']<=0:
            cost=int(p['wage']*8)
            if t['budget']>cost and random.random()<(0.75 if p['overall']>=65 els
                t['budget']-=cost; p['wage']=int(p['wage']*random.uniform(1.05,1.
            else: t['players'].remove(p)
    while len(t['players'])<18:
        np=gen_player(t['country'],t['name'],random.randint(10000,99999))
        np['overall']=clamp(np['overall']+random.randint(-4,2),45,75); t['players

def cpu_transfers():
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            if t['name']==game_state['selected_club']: continue
            if random.random()<0.25 and t['budget']>200000:
                p=gen_player(t['country'],t['name'],random.randint(10000,99999))
                if p['overall']>60 and t['budget']>p['value']:
                    t['budget']-=p['value']; t['players'].append(p)

def run_cpu():
    for d in game_state['divisions']:
```

```
        for t in game_state['divisions'][d]:
            if t['name']==game_state['selected_club']: continue
            cpu_contracts(t); refill_youth(t)
    cpu_transfers()

# ============================================================
# 昇降格・表彰
# ============================================================
def do_promotion():
    d1=sorted_table('D1'); d2=sorted_table('D2'); d3=sorted_table('D3')
    rel1=d1[-2:]; prom2=d2[:2]; rel2=d2[-2:]; prom3=d3[:2]
    def mv(t,fr,to):
        game_state['divisions'][fr]=[x for x in game_state['divisions'][fr] if x[
        game_state['divisions'][to].append(t)
    for t in rel1:
        mv(t,'D1','D2'); add_news(f'🔻降格: {t["name"]} D1→D2','club')
        if t['name']==game_state['selected_club']: game_state['club_brand']=clamp
    for t in prom2:
        if t not in rel1:
            mv(t,'D2','D1'); add_news(f'🔺昇格: {t["name"]} D2→D1','club')
            if t['name']==game_state['selected_club']:
                game_state['club_brand']=clamp(game_state['club_brand']+8,0,100);
    for t in rel2:
        if t not in prom2: mv(t,'D2','D3'); add_news(f'🔻降格: {t["name"]} D2→D3',
    for t in prom3:
        if t not in rel2: mv(t,'D3','D2'); add_news(f'🔺昇格: {t["name"]} D3→D2',
    country=game_state['selected_country']; used={t['name'] for d in game_state['
    for dn,(rmin,rmax) in [('D1',(62,78)),('D2',(54,70)),('D3',(46,64))]:
        while len(game_state['divisions'][dn])<10:
            nt=create_team(country,_rname(country,used),rmin,rmax); init_stats();
        while len(game_state['divisions'][dn])>10: game_state['divisions'][dn].po

def prize_money(dn,rank):
    if dn=='D1': t={1:900000,2:700000,3:550000,4:420000,5:320000,6:240000,7:18000
    elif dn=='D2': t={1:500000,2:380000,3:280000,4:220000,5:170000,6:130000,7:100
    else: t={1:300000,2:220000,3:170000,4:130000,5:100000,6:80000,7:60000,8:50000
    return t.get(rank,0)

def calc_awards():
    pl=all_players()
    if not pl: return
    gb=max(pl,key=lambda p:p['stats']['goals']); mvp=max(pl,key=lambda p:p['stats
    game_state['season_awards']={'golden_boot':gb,'mvp':mvp}
    game_state['history_awards'].append({'season':game_state['season'],'golden_bo
    add_news(f'年間MVP: {mvp["name"]}','club'); add_news(f'得点王: {gb["name"]}({gb

def update_hof():
```

```
    club=get_sel()
    if not club: return
    for p in club['players']:
        score=p['stats']['goals']*3+p['stats']['assists']*2+p['stats']['mom']*4
        e={'name':p['name'],'score':score,'reason':'シーズン表彰'}
        if score>120 and e not in game_state['club_hall_of_fame']:
            game_state['club_hall_of_fame'].append(e); add_news(f'殿堂入り: {p["nar

def declare_goal(goal):
    sel=get_sel()
    if not sel: return
    sel['player_season_goal']=goal; game_state['season_goal_declared']=True
    bonus={'優勝を狙う':5,'上位進出':3,'残留で十分':2,'昇格を目指す':4}.get(goal,0)
    game_state['board_rating']=clamp(game_state['board_rating']+bonus,0,100)
    add_news(f'⚽今季目標:「{goal}」','club'); refresh_ui()

def check_goal(dn,rank):
    sel=get_sel()
    if not sel or not sel.get('player_season_goal'): return
    goal=sel['player_season_goal']
    ok=(goal=='優勝を狙う' and rank==1) or (goal=='上位進出' and rank<=4) or (goal==
    if ok:
        sel['budget']+=100000; game_state['board_rating']=clamp(game_state['board
        game_state['club_brand']=clamp(game_state['club_brand']+5,0,100); add_new
    else:
        game_state['board_rating']=clamp(game_state['board_rating']-5,0,100); add
    sel['player_season_goal']=None; game_state['season_goal_declared']=False

def reset_fin():
    game_state['season_finance']={k:0 for k in ['sponsor','matchday','transfers_i

def start_domestic_cup():
    teams=[t for d in game_state['divisions'] for t in game_state['divisions'][d]
    random.shuffle(teams)
    game_state['domestic_cup']={'teams':teams[:16],'round':1,'winner':None}
    add_news('🏆国内カップ戦が開幕！16クラブ参加','cup')

def play_cup_round():
    cup=game_state.get('domestic_cup')
    if not cup or cup.get('winner'): return
    winners=[]
    for i in range(0,len(cup['teams']),2):
        if i+1>=len(cup['teams']): winners.append(cup['teams'][i]); continue
        a,b=cup['teams'][i],cup['teams'][i+1]
        ga,gb,*_=simulate_match_full(a,b,cup=True); w=a if ga>gb else b; winners.
        sel=get_sel()
        if sel and sel['name'] in (a['name'],b['name']): add_news(f'カップ: {a["na
```

```
    cup['teams']=winners; cup['round']+=1
    if len(cup['teams'])==1:
        champ=cup['teams'][0]; cup['winner']=champ['name']; champ['budget']+=5000
        add_news(f'🏆国内カップ優勝: {champ["name"]}','cup')
        if champ['name']==game_state['selected_club']:
            game_state['club_brand']=clamp(game_state['club_brand']+8,0,100); che
    refresh_ui()

def start_cc():
    """コンチネンタルカップ グループ組み合わせ抽選（D1上位4チームのみ）"""
    d1=sorted_table('D1')
    # D1上位4チーム＋他ディビジョンから補完（合計8チーム）
    cl_teams=d1[:4]
    for d in ['D2','D3']:
        if len(cl_teams)>=8: break
        cl_teams+=sorted_table(d)[:2]
    random.shuffle(cl_teams)
    cl_teams=cl_teams[:8]
    # 2グループに分割
    groups={'A':cl_teams[:4],'B':cl_teams[4:]}
    for gname,gteams in groups.items():
        for t in gteams: t.setdefault('cl_pts',0); t.setdefault('cl_gf',0); t.set
    sel=get_sel()
    in_cl=sel and any(t['name']==sel['name'] for t in cl_teams)
    game_state['cc_data']={
        'groups':groups,'stage':'group','round':0,
        'ko_teams':[],'ko_round':0,'winner':None,
        'in_cl':in_cl,
    }
    if in_cl:
        my_group='A' if any(t['name']==sel['name'] for t in groups['A']) else 'B'
        opponents=[t['name'] for t in groups[my_group] if t['name']!=sel['name']]
        sel['budget']+=CC_PRIZE['group']//2  # 出場ボーナス
        add_news(f'🌍コンチネンタルカップ グループ抽選！グループ{my_group}。対戦: {", ".jc
        add_news(f'コンカップ出場ボーナス: {fmt_m(CC_PRIZE["group"]//2)}','cup')
    else:
        add_news('🌍CLが開幕！（今季は出場資格なし）','cup')

def play_cc_group_round():
    """コンチネンタルカップ グループ1節を処理"""
    cl=game_state.get('cc_data')
    if not cl or cl['stage']!='group': return
    sel=get_sel()
    # 各グループで総当たり（最大3節）
    for gname,gteams in cl['groups'].items():
        pairs=[(gteams[i],gteams[j]) for i in range(len(gteams)) for j in range(i
        if cl['round'] < len(pairs):
```

```
            a,b=pairs[cl['round']]
            ga,gb,*_=simulate_match_full(a,b)
            # CL順位表更新
            if ga>gb: a['cl_pts']=a.get('cl_pts',0)+3
            elif ga==gb: a['cl_pts']=a.get('cl_pts',0)+1; b['cl_pts']=b.get('cl_p
            else: b['cl_pts']=b.get('cl_pts',0)+3
            a['cl_gf']=a.get('cl_gf',0)+ga; a['cl_ga']=a.get('cl_ga',0)+gb
            b['cl_gf']=b.get('cl_gf',0)+gb; b['cl_ga']=b.get('cl_ga',0)+ga
            if sel and sel['name'] in (a['name'],b['name']):
                add_news(f'🌍CL Gr{gname}: {a["name"]} {ga}-{gb} {b["name"]}','cu
    cl['round']+=1
    # グループステージ終了チェック（3節で終了）
    if cl['round']>=3:
        _cc_group_finish(cl)

def _cc_group_finish(cl):
    """グループステージ終了→決勝T組み合わせ"""
    if not cl: return
    sel=get_sel()
    ko_teams=[]
    for gname,gteams in cl['groups'].items():
        ranked=sorted(gteams,key=lambda t:(-t.get('cl_pts',0),-(t.get('cl_gf',0)-
        top2=ranked[:2]; ko_teams+=top2
        if sel:
            my_rank=next((i+1 for i,t in enumerate(ranked) if t['name']==sel['nam
            if my_rank:
                if my_rank<=2:
                    sel['budget']+=CC_PRIZE['group']
                    add_news(f'🌍CL Gr{gname} {my_rank}位通過！賞金{fmt_m(CC_PRIZE[
                else:
                    add_news(f'🌍CL Gr{gname} {my_rank}位 グループ敗退...','cup')
    cl['stage']='knockout'; cl['ko_teams']=ko_teams; cl['ko_round']=0
    add_news('🌍コンチネンタルカップ 決勝T開始！','cup')

def play_cc_ko_round():
    """コンチネンタルカップ 決勝T1ラウンドを処理"""
    cl=game_state.get('cc_data')
    if not cl or cl['stage']!='knockout': return
    teams=cl['ko_teams']
    if len(teams)<=1:
        if teams:
            champ=teams[0]; cl['winner']=champ['name']
            prize=CC_PRIZE['winner']
            add_news(f'🏆🌍コンカップ優勝: {champ["name"]}！','cup')
            if champ['name']==game_state['selected_club']:
                champ['budget']+=prize; add_fin('コンカップ優勝賞金',prize,'Continen
                game_state['club_brand']=clamp(game_state['club_brand']+15,0,100)
```

```
                check_achievement('cc_winner')
                add_news(f'コンカップ優勝賞金: {fmt_m(prize)}','cup')
        return
    round_names={0:'準々決勝',1:'準決勝',2:'決勝'}
    rname=round_names.get(cl['ko_round'],'ラウンド')
    prize_keys={0:'quarterfinal',1:'semifinal',2:'final'}
    prize_key=prize_keys.get(cl['ko_round'],'quarterfinal')
    winners=[]
    sel=get_sel()
    for i in range(0,len(teams),2):
        if i+1>=len(teams): winners.append(teams[i]); continue
        a,b=teams[i],teams[i+1]
        ga,gb,*_=simulate_match_full(a,b,cup=True)
        w=a if ga>gb else b; losers_name=(b if ga>gb else a)['name']
        winners.append(w)
        if sel and sel['name'] in (a['name'],b['name']):
            add_news(f'🌍CL {rname}: {a["name"]} {ga}-{gb} {b["name"]}','cup')
            if w['name']==sel['name']:
                prize=CC_PRIZE[prize_key]
                sel['budget']+=prize; add_fin(f'CL{rname}賞金',prize,'Continental
                add_news(f'🏆CL {rname}突破！賞金{fmt_m(prize)}','cup')
            else:
                add_news(f'😢CL {rname}敗退...','cup')
        else:
            add_news(f'🌍CL {rname}: {a["name"]} {ga}-{gb} {b["name"]}','cup')
    cl['ko_teams']=winners; cl['ko_round']+=1
    if len(winners)==1: play_cc_ko_round()

# 後方互換性のためエイリアスを残す
def start_cc(): start_cc()
def play_cc_round():
    cl=game_state.get('cc_data')
    if not cl: return
    if cl['stage']=='group': play_cc_group_round()
    elif cl['stage']=='knockout': play_cc_ko_round()

# ============================================================
# ユース大会
# ============================================================
def maybe_spawn_legacy_player(team):
    """15シーズン以上経過したら引退済み殿堂選手の子供がユースに登場"""
    if game_state['season'] < 15: return
    hof=game_state.get('club_hall_of_fame',[])
    if not hof or random.random()>0.15: return
    parent=random.choice(hof)
    country=team.get('country','England')
    p=gen_player(country,team['name'],random.randint(800000,899999),True)
```

```
    # 親の名字を受け継ぐ
    parent_lastname=parent['name'].split()[-1] if ' ' in parent['name'] else pare
    firstname=random.choice(COUNTRY_NAME_POOLS[country]['first'])
    p['name']=f'{firstname} {parent_lastname} Jr.'
    p['is_wonderkid']=True
    p['potential']=random.randint(82,94)
    for k in p['attrs']: p['attrs'][k]=clamp(p['attrs'][k]+random.randint(4,10),5
    _recompute(p); p['age']=16; p['contract_years']=3
    team['youth'].append(p)
    add_news(f'🌟伝説の2世: {p["name"]}がアカデミーに加入（{parent["name"]}の子）POT{p


def run_youth_cup(team):
    """シーズン終了時にユース大会を開催。施設Lvに応じて成長ボーナス"""
    if not team.get('youth'): return
    fac_lv=game_state['facilities']['youth']
    # 参加チームはユースがいる全クラブ
    participants=[t for d in game_state['divisions'] for t in game_state['divisio
    if len(participants)<4: return
    # 簡易トーナメント（チーム総合力で決定）
    def youth_strength(t):
        return avg([p['overall'] for p in t['youth']]) if t['youth'] else 0
    ranked=sorted(participants,key=youth_strength,reverse=True)
    winner=ranked[0]
    # 優勝チームのユース選手に成長ボーナス
    bonus=fac_lv*random.randint(1,3)
    for p in winner['youth']:
        attr=random.choice(list(p['attrs'].keys()))
        p['attrs'][attr]=clamp(p['attrs'][attr]+bonus,30,99)
        _recompute(p)
    add_news(f'🥇ユース大会優勝: {winner["name"]}（育成ボーナス+{bonus}）','cup')
    if winner['name']==game_state['selected_club']:
        game_state['locker_room_mood']=clamp(game_state['locker_room_mood']+5,0,1
        add_news('自クラブのユースが大会を制覇！ロッカールームが盛り上がっている','club')


# ============================================================
# メインゲームループ
# ============================================================
def get_week_pairs(dn,week):
    teams=game_state['divisions'][dn]; names=[t['name'] for t in teams]
    rot=names[:]; seed=game_state['season']*1000+week+(1 if dn=='D2' else 2 if dn
    random.Random(seed).shuffle(rot)
    pairs=[(rot[i],rot[i+1]) for i in range(0,len(rot)-1,2) if i+1<len(rot)]
    if week%2==0: pairs=[(b,a) for a,b in pairs]
    return pairs
```

```
def render_radar_chart(p, size=200):
    """選手の8属性レーダーチャートをSVGで描画（干渉なし版）"""
    import math
    attrs_order=['SPD','SHT','TEC','PAS','DEF','PHY','MEN','STA']
    vals=[p['attrs'].get(a,50) for a in attrs_order]
    n=len(attrs_order)
    # ラベル用に余白を広く取る
    margin=40
    cx,cy=size//2,size//2
    r=size//2-margin
    points=[]
    for i,v in enumerate(vals):
        angle=math.pi*2*i/n - math.pi/2
        ratio=v/99
        points.append((cx+r*ratio*math.cos(angle), cy+r*ratio*math.sin(angle)))
    svg=[f'<svg width="{size}" height="{size}" viewBox="0 0 {size} {size}" style=
    # グリッド
    for lvl in [0.25,0.5,0.75,1.0]:
        gp=[f'{cx+r*lvl*math.cos(math.pi*2*i/n-math.pi/2):.1f},{cy+r*lvl*math.sin
        col='#334155' if lvl<1.0 else '#475569'
        svg.append(f'<polygon points="{" ".join(gp)}" fill="none" stroke="{col}"
    # 軸線
    for i in range(n):
        angle=math.pi*2*i/n-math.pi/2
        svg.append(f'<line x1="{cx}" y1="{cy}" x2="{cx+r*math.cos(angle):.1f}" y2
    # データ多角形
    svg.append(f'<polygon points="{" ".join(f"{x:.1f},{y:.1f}" for x,y in points)
    # ドット
    for x,y in points:
        svg.append(f'<circle cx="{x:.1f}" cy="{y:.1f}" r="3" fill="#3b82f6"/>')
    # ラベル（属性名＋数値を背景付きで表示）
    for i,(attr,v) in enumerate(zip(attrs_order,vals)):
        angle=math.pi*2*i/n-math.pi/2
        lx=cx+(r+margin*0.7)*math.cos(angle)
        ly=cy+(r+margin*0.7)*math.sin(angle)
        col='#4ade80' if v>=70 else('#fbbf24' if v>=55 else '#f87171')
        # 背景矩形
        svg.append(f'<rect x="{lx-16:.1f}" y="{ly-14:.1f}" width="32" height="26"
        svg.append(f'<text x="{lx:.1f}" y="{ly-4:.1f}" text-anchor="middle" domin
        svg.append(f'<text x="{lx:.1f}" y="{ly+8:.1f}" text-anchor="middle" domin
    svg.append('</svg>')
    ui.html("".join(svg))


def auto_substitutions(team):
    """疲労・スランプ・怪我リスクを加味したオート交代（最大3枚）"""
    lu=best_lineup(team)
```

```
    if not lu: return []
    bench=[p for p in team['players'] if p['id'] not in [x['id'] for x in lu]
           and p['injury_weeks']<=0 and p.get('intl_weeks',0)<=0]
    if not bench: return []
    subs=[]
    # 交代候補：スタミナ45以下 or スランプ or 怪我リスク高い
    candidates=sorted(
        [(p, _sub_priority(p)) for p in lu if _sub_priority(p)>0],
        key=lambda x:-x[1]
    )[:3]
    for out_p, priority in candidates:
        # 同ポジションの最高OVRを探す
        same_pos=[p for p in bench if p['pos']==out_p['pos']]
        if not same_pos:
            same_pos=[p for p in bench]  # 同ポジがなければ誰でも
        if not same_pos: continue
        in_p=max(same_pos,key=lambda p:p['overall'])
        subs.append((out_p['id'],in_p['id']))
        bench=[p for p in bench if p['id']!=in_p['id']]
    return subs

def _sub_priority(p):
    """交代優先度スコア（高いほど交代すべき）"""
    score=0
    stamina=p.get('stamina',100)
    if stamina<35: score+=3
    elif stamina<50: score+=2
    elif stamina<65: score+=1
    if p.get('state')=='slump': score+=2
    if p.get('injury_weeks',0)>0: score+=5
    if p.get('trait')=='スペ体質' and stamina<60: score+=1
    return score


def play_next_week():
    init_stats()  # 毎回キーの補完を保証
    sel=get_sel()
    if not sel: ui.notify(T('クラブ未選択')); return
    if game_state.get('preseason_phase') and not sel.get('preseason_done'):
        nav_state['tab']='matches'; refresh_ui(); return
    if game_state.get('manager_fired'): ui.notify(T('監督危機'),'warning')
    # 複数週一括進行カウント
    if game_state.get('bulk_advancing') and game_state.get('bulk_weeks_left',0)>0
        game_state['bulk_weeks_left']-=1
        if game_state['bulk_weeks_left']<=0 or game_state.get('pending_event'):
            game_state['bulk_advancing']=False
    # 前週のプレス・ハーフタイムデータをクリア
```

```
game_state['pending_press']=False
game_state['halftime_data']=None
weather=roll_weather()
if weather in ['雨','雪','強風']: add_news(f'🌦今節の天候: {weather}','match')
# 事前設定された後半戦術を取得（任意）
preset_ht_tac=game_state.get('preset_halftime_tactic')
won=drew=lost=False; results=[]
for cd in ['D1','D2','D3']:
    for hn,an in get_week_pairs(cd,game_state['week']):
        ht=find_team(hn); at=find_team(an)
        if not ht or not at: continue
        # 試合を1回で完走（中断なし）
        ga,gb,att,rev,stats,extra=simulate_match_full(
            ht,at,
            ht_tac_a=preset_ht_tac if sel['name']==hn else None
        )
        results.append(f'{hn} {ga}-{gb} {an}')
        if sel['name'] in (hn,an):
            my=ga if sel['name']==hn else gb
            op=gb if sel['name']==hn else ga
            game_state['last_match']={
                'result':results[-1],'attendance':att,'revenue':rev,
                'stats':stats,'mom':extra['mom'],
                'highlights':extra['highlights'],'derby':extra['derby'],
                'weather':weather,'halfscore':extra.get('halfscore',(0,0,0,0)
            }
            if my>op: won=True
            elif my==op: drew=True
            else: lost=True
            if my>op and ht['name']==game_state['selected_club']: _pay_win_bo
            elif op>my and at['name']==game_state['selected_club']: _pay_win_
weekly_sponsor(sel); weekly_expenses(sel); weekly_broadcast(sel); weekly_merc
process_loan_returns(); process_intl_calls(sel)
update_player_feelings(sel,won,drew,lost); update_player_states(sel); update_
update_fan_happiness(sel,won,drew,lost); update_manager_rating(sel,won,drew,l
if game_state['manager_rating']>=45 and game_state['board_rating']>=40 and ga
    game_state['manager_fired']=False; add_news('監督評価回復、続投決定','club')
recover_players(); gen_domestic_offers(); gen_foreign_offer(); gen_loan_offer
agent_intervention(sel); apply_individual_training(sel); check_retirements(se
gen_random_event(); gen_manager_offers()
# FA市場の有効期限を更新
for fa in list(game_state.get('fa_market',[])):
    fa['fa_weeks']=fa.get('fa_weeks',1)-1
    if fa['fa_weeks']<=0:
        game_state.get('fa_market',[]).remove(fa) if fa in game_state.get('fa
# ライバル補強動向ヒント（30%の確率でニュースに追加）
_hint=rival_activity_text()
```

```
    if _hint: add_news(f'🕵{_hint}','club')
    if won:
        game_state['win_streak']=game_state.get('win_streak',0)+1
        if game_state['win_streak']>=5: check_achievement('unbeaten_5')
        if sel.get('season_stats',{}).get('w',0)==1: check_achievement('first_win
    else: game_state['win_streak']=0
    if sel['budget']>=5000000: check_achievement('rich_club')
    dn=div_of(sel['name']); tbl=sorted_table(dn)
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==sel['name']),10)
    game_state.setdefault('rank_history',[]).append({'week':game_state['week'],'r
    sel.setdefault('rank_history',[]).append(rank)
    if game_state.get('last_match'):
        lm=game_state['last_match']
        add_news(f'観客{lm["attendance"]:,}人|収入{fmt_m(lm["revenue"])}','match')
        add_news(f'MOM: {lm["mom"]["name"]}','match')
        if lm.get('derby'): add_news(f'ダービー: {lm["result"]}','match')
    for r in reversed(results[-6:]): add_news(f'第{game_state["week"]}節: {r}','ma
    wk=game_state['week']
    if wk==CUP_START: start_domestic_cup()
    elif wk>CUP_START and (wk-CUP_START)%2==0:
        cup=game_state.get('domestic_cup')
        if cup and not cup.get('winner'): play_cup_round()
    # CL自動進行
    cl=game_state.get('cc_data')
    if cl and not cl.get('winner'):
        if cl['stage']=='group' and wk>=CC_GROUP_START and (wk-CC_GROUP_START)%3=
            play_cc_group_round()
        elif cl['stage']=='knockout' and wk>=CC_KO_START and (wk-CC_KO_START)%3==
            play_cc_ko_round()
    if wk==sel.get('board_meeting_week',8): trigger_board_meeting()
    if random.random()<0.03 and not sel.get('naming_rights') and not game_state.g
    if game_state.get('injury_insurance_active'):
        cost=8000; sel['budget']-=cost; add_sf('insurance',cost)
    game_state['week']+=1
    if game_state['week']>SEASON_WEEKS: season_end()
    elif game_state.get('bulk_advancing') and game_state.get('bulk_weeks_left',0)
        play_next_week()
    else:
        # 試合完了後にプレスカンファレンスをセット（一括進行中はスキップ）
        if not game_state.get('bulk_advancing'):
            game_state['pending_press']=True
        refresh_ui()

def set_halftime_tactic(tactic_choice):
    """後半戦術を事前設定（任意）"""
    if tactic_choice:
        game_state['preset_halftime_tactic']=tactic_choice
```

```
        add_news(f'⚙️後半戦術プリセット: {tactic_choice}','match')
    else:
        game_state['preset_halftime_tactic']=None
        add_news('後半戦術: 通常戦術で進行','match')
    refresh_ui()

def do_halftime(tactic_choice):
    """後方互換: 後半戦術プリセットを設定"""
    set_halftime_tactic(tactic_choice)

def bulk_advance(n):
    """複数週一括進行"""
    weeks=max(1,min(n,SEASON_WEEKS-game_state['week']+1))
    game_state['bulk_advancing']=True; game_state['bulk_weeks_left']=weeks
    add_news(f'⏩{weeks}週まとめて進行中...','club'); play_next_week()

def season_end():
    sel=get_sel()
    if not sel: return
    calc_awards(); update_hof()
    dn=div_of(sel['name']); tbl=sorted_table(dn)
    rank=next((i+1 for i,t in enumerate(tbl) if t['name']==sel['name']),10)
    check_goal(dn,rank)
    pm=prize_money(dn,rank); sel['budget']+=pm; add_fin('リーグ賞金',pm,f'{dn} Rank
    add_news(f'リーグ賞金{fmt_m(pm)}獲得','club')
    if dn=='D2' and rank<=2: sel['budget']+=350000; add_news('昇格ボーナス$350,000'
    if dn=='D3' and rank<=2: sel['budget']+=220000; add_news('昇格ボーナス$220,000'
    if rank==1: check_achievement('first_title')
    # クラブ歴史書に記録
    _season_goals=sum(p['stats']['goals'] for p in sel['players'])
    _season_assists=sum(p['stats']['assists'] for p in sel['players'])
    _season_stats=sel.get('season_stats',{})
    sel.setdefault('season_history',[]).append({
        'season':game_state['season'],'div':dn,'rank':rank,
        'goals':_season_goals,'assists':_season_assists,
        'w':_season_stats.get('w',0),'d':_season_stats.get('d',0),'l':_season_sta
        'gf':_season_stats.get('gf',0),'ga':_season_stats.get('ga',0),
        'budget':sel['budget'],'fan_base':sel['fan_base'],
        'best_player':(max(sel['players'],key=lambda p:p['stats'].get('goals',0)*
    })
    game_state['club_history'].append({'season':game_state['season'],'club':sel['
    do_promotion(); process_expiry(); run_cpu()
    tr_b=game_state['facilities']['training']*0.3; ass_b=sum(s['skill']*0.003 for
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            t['season_stats']={'p':0,'w':0,'d':0,'l':0,'gf':0,'ga':0,'gd':0,'pts'
            for p in t['players']:
```

```
            p['age']+=1; gc=0.45+ass_b; dc=0.35
            if p.get('growth_type')=='早熟':
                if p['age']<=22: gc=0.60
                elif p['age']>=27: dc=0.50
            if p.get('growth_type')=='晩成':
                if p['age']<=21: gc=0.30
                elif 24<=p['age']<=29: gc=0.55
            if p['age']<=23 and p['overall']<p['potential'] and random.random
                for k in p['attrs']:
                    if random.random()<0.4: p['attrs'][k]=clamp(p['attrs'][k]
            elif p['age']>=30 and random.random()<dc:
                for k in p['attrs']:
                    if random.random()<0.35: p['attrs'][k]=clamp(p['attrs'][k
            if p.get('injury_severity')=='重傷':
                for k in p['attrs']: p['attrs'][k]=clamp(p['attrs'][k]-random
            _recompute(p); p['stamina']=100
            if p.get('injury_severity')!='重傷': p['injury_weeks']=0; p['injur
            p['state']='normal'; p['state_weeks']=0
        for p in t['youth']:
            p['age']+=1
            pol=YOUTH_POLICIES.get(game_state['youth_policy'],YOUTH_POLICIES[
            for attr in p['attrs']:
                g=int(random.randint(0,2)*youth_b)
                if attr=='SPD': g=int(g*pol['SPD'])
                if attr in ['TEC','PAS']: g=int(g*pol['TEC'])
                if attr=='PHY': g=int(g*pol['PHY'])
                p['attrs'][attr]=clamp(p['attrs'][attr]+g,30,99)
            _recompute(p)
        refill_youth(t)
run_youth_cup(get_sel()) if get_sel() else None
if get_sel(): maybe_spawn_legacy_player(get_sel())
refresh_youth_queue(); gen_sponsor_nego(); start_cc(); game_state['domestic_c
game_state['season']+=1; game_state['week']=1; game_state['season_goal_declar
game_state['preseason_phase']=True; sel['preseason_done']=False
game_state['tryout_done']=False; game_state['tryout_pool']=[]; game_state['tr
# FA市場をリセット（新シーズン開始時）
game_state['fa_market']=[x for x in game_state.get('fa_market',[]) if x.get('
reset_fin()
# シーズン成績サマリーをgame_stateに保存
_hist=sel.get('season_history',[])
if _hist:
    cur=_hist[-1]
    prev=_hist[-2] if len(_hist)>=2 else None
    def _arrow(a,b,higher_is_better=True):
        if b is None: return ''
        if a>b: return '↑' if higher_is_better else '↓'
        if a<b: return '↓' if higher_is_better else '↑'
```

```
            return '='
        lines=[f'【S{cur["season"]} シーズン終了】']
        lines.append(f'{cur["div"]} {cur["rank"]}位 {_arrow(cur["rank"],prev["ran
        lines.append(f'{cur["w"]}勝{cur["d"]}分{cur["l"]}敗 得点{cur["gf"]}失点{cur
        lines.append(f'チーム総得点{cur["goals"]} {_arrow(cur["goals"],prev["goals"
        lines.append(f'予算{fmt_m(cur["budget"])} ファン{cur["fan_base"]:,}人')
        lines.append(f'今季MVP: {cur["best_player"]}')
        if prev:
            rank_diff=prev["rank"]-cur["rank"]
            if rank_diff>0: lines.append(f'✅ 前期比 {rank_diff}位上昇！')
            elif rank_diff<0: lines.append(f'❌ 前期比 {abs(rank_diff)}位下降')
        game_state['last_season_summary']=lines
    add_news('シーズン終了。新シーズンへ','club')
    if game_state['season']>=3: check_achievement('season3')
    refresh_ui()

def save_game():
    try:
        data=json.dumps(game_state,ensure_ascii=False,default=str)
        ui.download(data.encode('utf-8'),'club_strive_v5_save.json'); add_news('🚩
    except Exception as e: ui.notify(f'Save failed: {e}' if LANG=='en' else f'セー

def load_game(content:bytes):
    global game_state
    try:
        game_state=json.loads(content.decode('utf-8')); init_stats(); add_news('📂
    except Exception as e: ui.notify(f'Load failed: {e}' if LANG=='en' else f'ロー

def new_world(country):
    global game_state
    game_state=build_world(country); init_stats()
    for d in game_state['divisions']:
        for t in game_state['divisions'][d]:
            t['sponsor']=_gen_sponsor(t['country'],d)
    nav_state['tab']='dashboard'; refresh_ui()

def select_club(name):
    game_state['selected_club']=name; t=get_sel()
    if t and not t.get('sponsor'): t['sponsor']=_gen_sponsor(t['country'],div_of(
    refresh_youth_queue(); add_news(f'クラブ選択: {name}','club'); refresh_ui()

# ============================================================
# UI
# ============================================================
def render_status():
    status_box.clear()
    with status_box:
```

```
        t=get_sel()
        if not t: return
        dn=div_of(t['name']); tbl=sorted_table(dn) if dn else []
        rank=next((i+1 for i,x in enumerate(tbl) if x['name']==t['name']),'-')
        weather=get_weather()
        wc_col='text-blue' if weather in ['雨','雪'] else('text-orange' if weather
        with ui.card().classes('w-full'):
            with ui.row().classes('w-full items-center justify-between'):
                ui.label(f'🏟️ {t["name"]}').classes('text-body1 text-bold')
                ui.label(f'{dn} #{rank}' if LANG=='en' else f'{dn} {rank}位').clas
            with ui.row().classes('w-full items-center').style('gap:6px;flex-wrap
                ui.label(fmt_m(t['budget'])).classes('text-body2'+(' text-red tex
                ui.label(weather).classes(f'text-caption {wc_col}')
                ui.label(('🔒Win' if LANG=='en' else '🔒移籍') if in_transfer_win
                ui.label(f'S{game_state["season"]} W{game_state["week"]}').classe
            if game_state.get('manager_fired'): ui.label(T('監督危機label')).class
            if game_state.get('financial_crisis'): ui.label(T('財政危機label')).cl
            if game_state.get('bank_loans'): ui.label(f'🏦{len(game_state["bank_l


def render_dashboard():
    dashboard_box.clear()
    with dashboard_box:
        if not game_state['selected_club']:
            # チュートリアル
            if not game_state.get('tutorial_done'):
                # 言語選択（最初に表示）
                with ui.card().classes('w-full q-mb-sm').style('border:1px solid
                    ui.label('🌐 Language / 言語').classes('text-h6')
                    with ui.row().classes('justify-center').style('gap:12px;'):
                        ui.button('🇯🇵 日本語',on_click=lambda: set_lang('ja')).prc
                            f'{"color=primary" if LANG=="ja" else "flat"}')
                        ui.button('🇬🇧 English',on_click=lambda: set_lang('en')).p
                            f'{"color=primary" if LANG=="en" else "flat"}')
                with ui.card().classes('w-full q-mb-sm').style('border:1px solid
                    ui.label('📖 はじめに' if LANG=='ja' else '📖 How to Play').cl
                    steps_ja=[
                        ('🌍 国を選ぶ','上部のENG/SPA等から国を選択するとリーグが変わりま
                        ('🏟️ クラブを選ぶ','下のリストから管理したいクラブを選択。D1が最上
                        ('⚽ 試合を進める','「試合」タブ→「次の週へ進む」で1節進行。5週・1
                        ('👥 チーム編成','「チーム」タブでスターティングイレブン・戦術・個人
                        ('💰 移籍','移籍ウィンドウ（夏:週1-6/冬:週19-22）に選手の売買・レ
                        ('💾 セーブ','いつでもホーム画面からセーブできます'),
                    ]
                    steps_en=[
                        ('🌍 Choose Country','Select ENG/SPA etc. from the top tc
                        ('🏟️ Choose Club','Pick a club from the list. D1 is the t
```

```
                ('⚽ Advance Weeks','Matches tab → Next Week. Skip 5 or 1
                ('👥 Squad','Set starting XI, tactics and individual trai
                ('💰 Transfers','Buy/sell/loan players during transfer wi
                ('💾 Save','Save anytime from the Home screen'),
            ]
            steps=steps_en if LANG=='en' else steps_ja
            for icon_title,desc in steps:
                with ui.row().classes('w-full items-start').style('gap:8p
                    ui.label(icon_title).classes('text-body2 text-bold').
                    ui.label(desc).classes('text-caption text-grey').styl
            ui.button(T('わかった！クラブを選ぶ'),on_click=lambda: (game_stat
        return
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('クラブを選択')).classes('text-h6')
        # タブ形式でD1/D2/D3を切り替え
        with ui.tabs().classes('w-full') as div_tabs:
            tab_d1=ui.tab('D1',label='D1 '+('Top' if LANG=='en' else '1部
            tab_d2=ui.tab('D2',label='D2 '+('2nd' if LANG=='en' else '2部
            tab_d3=ui.tab('D3',label='D3 '+('3rd' if LANG=='en' else '3部
        with ui.tab_panels(div_tabs,value='D1').classes('w-full'):
            for tab_name,tab_obj in [('D1',tab_d1),('D2',tab_d2),('D3',ta
                with ui.tab_panel(tab_name):
                    desc=T('D1説明') if tab_name=='D1' else(T('D2説明') if
                    ui.label(desc).classes('text-caption text-grey q-mb-s
                    for tm in game_state['divisions'][tab_name]:
                        with ui.card().classes('w-full q-mb-xs').style('p
                            with ui.row().classes('w-full items-center ju
                                ui.label(tm['name']).classes('text-body2
                                with ui.row().style('gap:8px;'):
                                    ui.label(f'Rep:{tm["reputation"]}' if
                                    ui.label(fmt_m(tm['budget'])).classes
                            # ミニバー（予算と評判）
                            with ui.row().classes('w-full').style('gap:4p
                                ui.label('💰').classes('text-caption')
                                filled=min(10,max(1,int(tm['budget']/5000
                                ui.label('▉'*filled+'▒'*(10-filled)).clas
                            ui.button('▶ '+('Select' if LANG=='en' else '
    return
t=get_sel(); dn=div_of(t['name']); tbl=sorted_table(dn)
rank=next((i+1 for i,x in enumerate(tbl) if x['name']==t['name']),10)

# 今季目標（プレシーズン前に必須）
if not game_state.get('season_goal_declared') and not t.get('player_seaso
    with ui.card().classes('w-full q-mb-sm').style('border:2px solid #3b8
        ui.label(T('目標を宣言してください')).classes('text-h6 text-blue')
        ui.label(T('目標未宣言警告')).classes('text-caption text-grey')
        with ui.row().style('flex-wrap:wrap;gap:6px;'):
```

```
            for g in [T('優勝を狙う'),T('上位進出'),T('残留で十分'),T('昇格を目
                ui.button(g,on_click=lambda e,x=g:declare_goal(x)).props(
    return

# プレシーズン
if game_state.get('preseason_phase') and not t.get('preseason_done'):
    # トライアウト（プレシーズン中に年1回）
    tryout_pool=game_state.get('tryout_pool',[])
    tryout_done=game_state.get('tryout_done',False)
    if not tryout_done and not tryout_pool:
        with ui.card().classes('w-full q-mb-sm').style('border:1px solid
            ui.label(T('トライアウト')).classes('text-h6').style('color:#a7
            ui.label(T('トライアウト説明')).classes('text-caption text-grey'
            ui.label(T('スカウト施設Lv').replace('{n}',str(game_state['faci
            ui.button(T('トライアウト開催'),on_click=gen_tryout_pool).props(
    elif tryout_pool:
        with ui.card().classes('w-full q-mb-sm').style('border:1px solid
            signed_count=game_state.get('tryout_signed_count',0)
            ui.label(T('トライアウト開催中').replace('{a}',str(signed_count)
            for p in tryout_pool:
                hint=p.get('tryout_hint','印象？ 調査中')
                hint_col='text-positive' if '◎' in hint else 'text-grey'
                with ui.card().classes('w-full q-mb-xs').style('padding:8
                    with ui.row().classes('w-full items-center justify-be
                        ui.label(f'{p["name"]}').classes('text-body2 text
                        ui.label(hint).classes(f'text-caption {hint_col}'
                    ui.label(f'{p["pos"]} {p["age"]}y | Style:{p["playsty
                    ui.label(f'Trait: {p.get("trait","None")}' if LANG=='
                    ui.label(f'Sign fee: {fmt_m(int(p["wage"]*4))} / Wage
                    # 属性は全て「?」で表示
                    attr_str=' '.join(f'{ATTR_ICONS.get(k,k)}{ATTR_NAMES.
                    ui.label(attr_str).classes('text-caption text-grey')
                    ui.button(T('契約する'),on_click=lambda e,pid=p['id']:
            with ui.row():
                ui.button(T('トライアウト終了'),on_click=dismiss_tryout).pro
    with ui.card().classes('w-full q-mb-sm').style('border:1px solid #10b
        ui.label(T('プレシーズン')).classes('text-h6 text-positive')
        ui.label(T('今季目標label').replace('{n}',t.get('player_season_goa
        ui.button(T('プレシーズンボタン'),on_click=run_preseason).props('col

# （ハーフタイム中断は廃止）

# ランダムイベント
if game_state.get('pending_event'):
    evt=game_state['pending_event']
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(('📌 Event: ' if LANG=='en' else '📌イベント: ')+evt['tex
```

```
        with ui.row():
            for ch in evt['choices']: ui.button(ch,on_click=lambda e,c=ch

# プレスカンファレンス
if game_state.get('pending_press'):
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('プレスカンファレンス')).classes('text-h6')
        with ui.row():
            for ch in [T('強気発言'),T('慎重姿勢'),T('選手を称える'),T('批判を受
                ui.button(ch,on_click=lambda e,c=ch:do_press(c)).props('c

# 目標表示（宣言済みの場合）
if t.get('player_season_goal'):
    pass  # ダッシュボードカード内で表示済み

# 監督キャリアオファー
offers=game_state.get('manager_career_offers',[])
if offers:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('監督オファー')).classes('text-h6')
        for club_name in offers:
            ot=find_team(club_name)
            ui.label(f'{club_name} | Rep:{ot["reputation"] if ot else "?"
            ui.button(f'→ {club_name}' if LANG=='en' else f'{club_name}に
        ui.button(T('全て断る'),on_click=lambda: game_state.update({'manage

# ネーミングライツ
if game_state.get('naming_rights_offer'):
    nr=game_state['naming_rights_offer']
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('ネーミングライツ')).classes('text-h6')
        ui.label(f'{nr["company"]} | Upfront:{fmt_m(nr["upfront"])} | Wkl
        with ui.row():
            ui.button(T('契約するbtn'),on_click=accept_naming_rights).prop
            ui.button(T('断るbtn'),on_click=reject_naming_rights).props('c

# 買収オファー
if game_state.get('buyout_offer'):
    bo=game_state['buyout_offer']
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(f'💰{bo["type"]} Takeover' if LANG=='en' else f'💰{bo["t
        ui.label(f'+{fmt_m(bo["budget_boost"])}').classes('text-body2 tex
        with ui.row():
            ui.button(T('受け入れるbtn'),on_click=accept_buyout).props('col
            ui.button(T('断るbtn'),on_click=reject_buyout).props('color=ne

# スポンサー交渉
```

```
if game_state.get('pending_sponsor_negotiation'):
    data=game_state['pending_sponsor_negotiation']; o=data['offer']
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('スポンサー交渉label')).classes('text-h6')
        ui.label(f'{o["name"]} | Wkly:{fmt_m(o["weekly_income"])} | Req:{
        with ui.row():
            ui.button(T('契約するbtn'),on_click=accept_sponsor).props('col
            ui.button(T('断るbtn'),on_click=reject_sponsor).props('color=r

# メインダッシュボード
with ui.card().classes('w-full q-mb-sm'):
    ui.label(f'{t["name"]}').classes('text-h6')
    ui.label(f'{dn} #{rank} | Rep:{t["reputation"]} | Chem:{t.get("chemis
    if t.get('player_season_goal'): ui.label(('Goal: ' if LANG=='en' else
    if t.get('naming_rights'): ui.label(f'🏟 {t["naming_rights"]["company
    if t.get('sponsor'): ui.label(f'Sponsor: {t["sponsor"]["name"]} Wkly:
    # 評価バー（スマホ向けテキスト表示）
    # 数値が見えるもの（監督・ファン・ブランドは客観的）
    for lbl2,val2 in [('監督評価',game_state['manager_rating']),('ファン満足
        col='text-positive' if val2>=60 else('text-warning' if val2>=40 e
        filled=int(val2/10); bar='▉'*filled+'▒'*(10-filled)
        ui.label(f'{lbl2}: {bar} {int(val2)}').classes(f'text-caption {co
    # 数値を隠すもの（理事会・ロッカールームはテキストヒント）
    _bt,_bc=board_text(game_state['board_rating'])
    ui.label(_bt).classes(f'text-caption {_bc}')
    _lrm=game_state['locker_room_mood']
    _lrt='ロッカー: 🎉 最高の雰囲気' if _lrm>=80 else('ロッカー: 😊 良い雰囲気'
    _lrc='text-positive' if _lrm>=60 else('text-grey' if _lrm>=40 else 't
    ui.label(_lrt).classes(f'text-caption {_lrc}')

# 直近試合
if game_state.get('last_match'):
    lm=game_state['last_match']
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('直近試合')).classes('text-h6')
        ui.label(lm['result']).classes('text-body1 text-bold')
        ui.label(f'Att:{lm["attendance"]:,} | {fmt_m(lm["revenue"])} | MO
        if lm.get('weather'): ui.label(('Weather: ' if LANG=='en' else 'ヲ
        for h in lm.get('highlights',[]): ui.label(f'  {h}').classes('tex
        # テキスト実況（後半分）
        evts=game_state.get('live_commentary',[])
        if evts:
            ui.label(T('ハイライト')).classes('text-subtitle2')
            for e in [x for x in evts if x.get('t',0)>45][:8]:
                apos="'"; ui.label(f'{e["t"]}{apos} {e["txt"]}').classes(

# ダービー通算
```

```
if t.get('derby_record'):
    with ui.card().classes('w-full q-mb-sm'):
        ui.label('⚔️ Derby' if LANG=='en' else '⚔️ダービー').classes('text
        for rv,rec in t['derby_record'].items():
            ui.label(f'{rv}: {rec["w"]}W {rec["d"]}D {rec["l"]}L' if LANG

# 実績
done=game_state.get('achievements',[])
if done:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('実績label')).classes('text-h6')
        with ui.row():
            for key in done:
                a=ACHIEVEMENTS.get(key,{})
                ui.chip(f'{a.get("icon","")}{a.get("name",key)}').classes

# 監督キャリア歴
career=game_state.get('manager_career',[])
if career:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('監督キャリア')).classes('text-h6')
        for c in career: ui.label(f'S{c["season"]}: {c["club"]} {c["div"]

# 前期成績比較サマリー
summary=game_state.get('last_season_summary',[])
if summary:
    with ui.card().classes('w-full q-mb-sm').style('border:1px solid #636
        ui.label(T('前期成績')).classes('text-h6 text-purple')
        for line in summary:
            col='text-positive' if '↑' in line or '✅' in line else('text
            ui.label(line).classes(col)
        ui.button(T('閉じる'),on_click=lambda: (game_state.update({'last_s

# クラブ歴史書
history=get_sel().get('season_history',[]) if get_sel() else []
if len(history)>=2:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('クラブ歴史書')).classes('text-h6')
        for h in reversed(history[-5:]):
            prev=history[history.index(h)-1] if history.index(h)>0 else N
            rank_arrow=('↑' if prev and h['rank']<prev['rank'] else('↓' i
            ui.label(f'S{h["season"]}: {h["div"]} #{h["rank"]}{rank_arrow

# 順位推移
rh=game_state.get('rank_history',[])
if rh:
    with ui.card().classes('w-full q-mb-sm'):
```

```
        ui.label(T('順位推移label')).classes('text-h6')
        ui.label(T('直近10週')+' → '.join(str(x['rank']) for x in rh[-10:]

# 得点/アシスト/MOMランキング
try:
    top_g,top_a,top_m=get_top_scorers(5)
    if any(pp['goals']>0 for pp in top_g):
        with ui.card().classes('w-full q-mb-sm'):
            ui.label('⚽ '+('Top Scorers' if LANG=='en' else '得点ランキン
            for ri,pp in enumerate(top_g,1):
                is_self=any(p['name']==pp['name'] for p in t['players'])
                sty='color:#60a5fa;font-weight:bold;' if is_self else ''
                ui.label(f'{ri}. {pp["name"]} ({pp["club"]}) ⚽{pp["goals
    if any(pp['assists']>0 for pp in top_a):
        with ui.card().classes('w-full q-mb-sm'):
            ui.label('🎯 '+('Top Assists' if LANG=='en' else 'アシストラン
            for ri,pp in enumerate(top_a,1):
                is_self=any(p['name']==pp['name'] for p in t['players'])
                sty='color:#60a5fa;font-weight:bold;' if is_self else ''
                ui.label(f'{ri}. {pp["name"]} ({pp["club"]}) 🎯{pp["assis
    if any(pp['mom']>0 for pp in top_m):
        with ui.card().classes('w-full q-mb-sm'):
            ui.label('🌟 '+('Top Rated' if LANG=='en' else 'MOMランキング')
            for ri,pp in enumerate(top_m,1):
                is_self=any(p['name']==pp['name'] for p in t['players'])
                sty='color:#60a5fa;font-weight:bold;' if is_self else ''
                ui.label(f'{ri}. {pp["name"]} ({pp["club"]}) 🌟{pp["mom"]
except: pass

# 他ディビジョン順位ハイライト
with ui.card().classes('w-full q-mb-sm'):
    ui.label('🌍 '+('Other Divisions' if LANG=='en' else '他ディビジョン'))
    for dname in ['D1','D2','D3']:
        if dname==dn: continue
        tbl2=sorted_table(dname)
        ui.label(f'── {dname} ──').classes('text-caption text-grey')
        for pi,tm in enumerate(tbl2[:3],1):
            s=tm['season_stats']
            ui.label(f'{pi}. {tm["name"]} {s["pts"]}pt {s["w"]}-{s["d"]}-
        if len(tbl2)>3:
            last=tbl2[-1]; sl=last['season_stats']
            ui.label(f'...{len(tbl2)}. {last["name"]} {sl["pts"]}pt').cla

# 移籍要求
req=[p for p in t['players'] if p.get('transfer_request')]
if req:
    with ui.card().classes('w-full q-mb-sm'):
```

```
            ui.label(T('移籍要求')).classes('text-h6 text-red')
            for p in req:
                _mt2,_mc2=mood_text(p.get('unhappiness',0))
                ui.label(f'{p["name"]} OVR{p["overall"]} {_mt2}').classes(f't
                ui.button(T('交渉btn'),on_click=lambda e,pid=p['id']:settle_re

        # トレーニング結果
        tr=game_state.get('training_results',[])
        if tr:
            with ui.card().classes('w-full q-mb-sm'):
                ui.label(T('トレーニング結果')).classes('text-h6')
                for r in tr: ui.label(f'  {r}').classes('text-body2 text-positive

        # セーブ（ダッシュボード内）
        with ui.card().classes('w-full q-mb-sm'):
            with ui.row().classes('w-full items-center justify-between'):
                ui.label(T('セーブロード')).classes('text-h6')
                with ui.row():
                    ui.button(T('セーブ'),on_click=save_game).props('color=primary
                    ui.upload(label='📂ロード',on_upload=lambda e:load_game(e.cont

        # ニュース
        with ui.card().classes('w-full q-mb-sm'):
            ui.label('📰 '+('News' if LANG=='en' else 'ニュース')).classes('text-h
            cats=['全て','match','transfer','player','club','cup','injury']
            def set_f(c): game_state['news_filter']=c; refresh_ui()
            with ui.row():
                for c in cats:
                    active=game_state.get('news_filter','全て')==c
                    ui.button(c,on_click=lambda e,x=c:set_f(x)).props(f'{"color=p
            flt=game_state.get('news_filter','全て')
            for n in [x for x in game_state['news'] if flt=='全て' or x.get('cat')
                ui.label(f'・{n["text"]}').classes('text-body2')

def render_squad():
    dashboard_box.clear()
    with dashboard_box:
        t=get_sel()
        if not t: ui.label(T('クラブ未選択')); return

        # スターティングイレブン
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('スターティングXI')).classes('text-h6')
            xi_ids=set(t.get('starting_xi',[]))
            ui.label(f'Selected: {len(xi_ids)}/11 {"✅ Set" if len(xi_ids)==11 el
            # フォーメーション表示（11人選択済みの場合）
            if len(xi_ids)==11:
```

```
        render_formation_view(t,xi_ids)
        ui.separator()
    avail=[p for p in t['players'] if p['injury_weeks']<=0 and p.get('int
    for p in sorted(avail,key=lambda x:(-x['overall'],x['pos'])):
        inxi=p['id'] in xi_ids
        def toggle(e,pid=p['id'],cur=inxi):
            xi=set(t.get('starting_xi',[]));
            if cur: xi.discard(pid)
            elif len(xi)<11: xi.add(pid)
            t['starting_xi']=list(xi); refresh_ui()
        inj='🏥' if p['injury_weeks']>0 else ''
        with ui.row().classes('w-full items-center q-mb-xs').style('gap:4
            ui.button(f'{"✓" if inxi else "○"} {p["name"]} {p["pos"]} {p[
            ui.button(T('詳細'),on_click=lambda e,pid=p['id']:show_player(
    with ui.row():
        ui.button(T('確定'),on_click=lambda: set_starting_xi(t.get('starti
        ui.button(T('自動選出'),on_click=auto_lineup).props('color=seconda:

# PKシューター・セットプレー担当設定
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('PKセットプレー')).classes('text-h6')
    pk_id=t.get('pk_taker_id')
    ck_id=t.get('ck_taker_id')
    fk_id=t.get('fk_taker_id')
    pk_p=next((p for p in t['players'] if p['id']==pk_id),None)
    ck_p=next((p for p in t['players'] if p['id']==ck_id),None)
    fk_p=next((p for p in t['players'] if p['id']==fk_id),None)
    if pk_p: ui.label(f'PK Taker: {pk_p["name"]} (SHT:{pk_p["attrs"].get(
    if ck_p: ui.label(f'CK Taker: {ck_p["name"]} (TEC:{ck_p["attrs"].get(
    if fk_p: ui.label(f'FK Taker: {fk_p["name"]} (SHT:{fk_p["attrs"].get(
    ui.label(T('PKシューター指定')).classes('text-subtitle2')
    top_shooters=sorted(t['players'],key=lambda p:p['attrs'].get('SHT',50
    with ui.row():
        for p in top_shooters:
            ui.button(f'{p["name"]} SHT:{p["attrs"].get("SHT",50)}',on_cl
    ui.label(T('CK担当指定')).classes('text-subtitle2')
    top_tec=sorted(t['players'],key=lambda p:p['attrs'].get('TEC',50),rev
    with ui.row():
        for p in top_tec:
            ui.button(f'{p["name"]} TEC:{p["attrs"].get("TEC",50)}',on_cl
    ui.label(T('FK担当指定')).classes('text-subtitle2')
    with ui.row():
        for p in top_shooters:
            ui.button(f'{p["name"]} SHT:{p["attrs"].get("SHT",50)}',on_cl

# 戦術
with ui.card().classes('w-full q-mb-sm'):
```

```
    ui.label(T('戦術label')).classes('text-h6')
    with ui.row():
        for tn in TACTICS: ui.button(tn,on_click=lambda e,x=tn:set_tactic
    ui.label(T('フォーメーション')).classes('text-subtitle2')
    with ui.row():
        for fn in FORMATIONS: ui.button(fn,on_click=lambda e,x=fn:set_for

# 監督スキル
with ui.card().classes('w-full q-mb-sm'):
    ui.label(f'🎓 Manager Skills (exp:{game_state.get("manager_exp",0)} S
    with ui.row():
        for sk,data in MANAGER_SKILLS.items():
            learned=sk in game_state.get('manager_skills',[])
            req_ok=game_state.get('manager_exp',0)>=data['req_exp']
            col='positive' if learned else('primary' if req_ok else 'grey
            ui.button(f'{"✅" if learned else ""}{sk} {data["desc"]}',on_

# トップチーム
with ui.card().classes('w-full q-mb-sm'):
    ui.label(('First Team (' if LANG=='en' else 'トップチーム (')+str(len(t
for p in sorted(t['players'],key=lambda x:(x['pos'],-x['overall'])):
    inj=f'🏥{p["injury_severity"]}({p["injury_weeks"]}週)' if p['injury_w
    intl=f'🌍({p.get("intl_weeks",0)}週)' if p.get('intl_weeks',0)>0 else
    req='⚠️' if p.get('transfer_request') else ''
    cap='©' if p['id']==t.get('captain_id') else ''
    xi_mark='✓' if p['id'] in set(t.get('starting_xi',[])) else ''
    retire='💐' if p.get('retiring') else ''
    state=PLAYER_STATES.get(p.get('state','normal'),PLAYER_STATES['normal
    agent='🤝' if p.get('has_agent') else ''
    local='🏠' if p.get('is_local') else ''
    convert=f'🔄→{p.get("target_pos","")}({p.get("convert_weeks",0)}週)'
    with ui.card().classes('w-full q-mb-xs'):
        ui.label(f'{xi_mark}{cap}{p["name"]} | {p["pos"]} OVR{p["overall"
        _mt,_mc=mood_text(p.get('unhappiness',0)); _mo,_moc=morale_text(p
        ui.label(f'{p["age"]}y | Wage:{fmt_m(p["wage"])} | {p["contract_y
        with ui.row().style('gap:6px;flex-wrap:wrap;'):
            ui.label(_mt).classes(f'text-caption {_mc}')
            ui.label(_mo).classes(f'text-caption {_moc}')
            ui.label(_st).classes(f'text-caption {_stc}')
        with ui.row().classes('items-center').style('gap:6px;flex-wrap:wr
            g=p['stats']['goals']; a=p['stats']['assists']; m=p['stats'][
            apps=p['stats']['apps']
            ui.label(f'{"Apps:" if LANG=="en" else "出場"}{apps}').classes
            if g>0: ui.label(f'⚽{g}G').classes('text-body2 text-positive
            if a>0: ui.label(f'🎯{a}A').classes('text-body2 text-blue tex
            if m>0: ui.label(f'🌟{m}MOM').classes('text-body2 text-yellow
            if p['stats'].get('yellow_cards',0)>0: ui.label(f'🟨{p["stats
```

```
            if p['stats'].get('red_cards',0)>0: ui.label(f'🟥{p["stats"][
        attr_str=' '.join(f'{ATTR_ICONS.get(k,k)}{v}' for k,v in p['attrs
        ui.label(attr_str).classes('text-caption')
        with ui.row():
            ui.button('Renew' if LANG=='en' else '延長',on_click=lambda e,
            ui.button('Detail' if LANG=='en' else '詳細',on_click=lambda e
            ui.button('©C',on_click=lambda e,pid=p['id']:set_captain(pid
            ui.button('🔄 Loan' if LANG=='en' else '🔄放出',on_click=lamb
        # トレーニング・役割設定
        focus=p.get('training_focus','')
        with ui.row():
            ui.label('Focus:' if LANG=='en' else '集中:').classes('text-ca
            for attr in list(p['attrs'].keys())[:4]:
                ui.button(f'{ATTR_ICONS.get(attr,attr)}',on_click=lambda
            ui.button(('Role:' if LANG=='en' else '役割:')+p.get('role_wis
        # コンバート
        with ui.row():
            ui.label('Convert→' if LANG=='en' else 'コンバート→').classes('
            for pos in [pp for pp in POSITIONS if pp!=p['pos']]:
                ui.button(pos,on_click=lambda e,pid=p['id'],tp=pos:conver

# レンタル等
lo=loaned_out()
if lo:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('レンタル放出中')).classes('text-h6')
        for p,dest in lo:
            ui.label(f'{p["name"]}→{dest} {p["loan_weeks"]}w' if LANG=='e
            ui.button(T('召還btn'),on_click=lambda e,pid=p['id']:recall_lo
if t.get('loan_in'):
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('レンタル借用中')).classes('text-h6')
        for p in t['loan_in']:
            ui.label(f'{p["name"]}({p.get("loan_origin","?")}) {p["pos"]}
# ユース
qps=[player_by_id(pid) for pid in game_state.get('youth_decision_queue',[
if qps:
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('18歳ユース')).classes('text-h6')
        for p in qps:
            ui.label(f'{p["name"]} {p["age"]}y {p["pos"]} OVR{p["overall"
            with ui.row():
                ui.button(T('昇格btn'),on_click=lambda e,pid=p['id']:promo
                ui.button(T('残留btn'),on_click=lambda e,pid=p['id']:retai
                ui.button(T('放出btn2'),on_click=lambda e,pid=p['id']:rele
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('ユースアカデミー')).classes('text-h6')
```

```
            with ui.row():
                for pol in YOUTH_POLICIES: ui.button(pol,on_click=lambda e,p=pol:
            qset=set(game_state.get('youth_decision_queue',[]))
            for p in sorted([x for x in t['youth'] if x['id'] not in qset],key=la
                tag='🌟' if p.get('is_wonderkid') else ''
                ui.label(f'{tag}{p["name"]} {p["age"]}y {p["pos"]} OVR{p["overall

def render_matches():
    dashboard_box.clear()
    with dashboard_box:
        t=get_sel()
        if not t: ui.label(T('クラブ未選択')); return
        dn=div_of(t['name'])

        # シーズン成績サマリー（試合タブ最上部）
        summary=game_state.get('last_season_summary',[])
        if summary:
            with ui.card().classes('w-full q-mb-sm').style('border:1px solid #636
                ui.label('📊 '+('Last Season' if LANG=='en' else '前期シーズン成績')
                for line in summary:
                    col='text-positive' if '↑' in line or '✅' in line else('text
                    ui.label(line).classes(col)
                ui.button(T('閉じる'),on_click=lambda: (game_state.update({'last_s

        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('試合進行label')).classes('text-h6')
            ui.label((f'{dn} | Round {game_state["week"]}/{SEASON_WEEKS} | Weathe
            if game_state.get('preseason_phase') and not t.get('preseason_done'):
                ui.button(T('プレシーズン実施'),on_click=run_preseason).props('color
            else:
                # メインボタン
                ui.button(
                    '▶ '+('Next Week' if LANG=='en' else '次の週へ進む'),
                    on_click=play_next_week
                ).props('color=primary').classes('w-full')
                # 危険ボタンは間隔を空けて小さく配置
                ui.separator()
                ui.label('⚠️ '+('Auto-advance (cannot undo)' if LANG=='en' else '
                with ui.row().style('gap:8px;'):
                    remaining=SEASON_WEEKS-game_state['week']+1
                    ui.button(
                        '⏩ '+('To Season End' if LANG=='en' else f'シーズン終了まで
                        on_click=lambda: bulk_advance(SEASON_WEEKS-game_state['we
                    ).props('flat color=grey size=sm')
            cup=game_state.get('domestic_cup')
            if cup and not cup.get('winner'):
                in_cup=any(x['name']==t['name'] for x in cup['teams'])
```

```
                ui.label(f'🏆 Domestic Cup R{cup["round"]} / {"Active" if in_cup
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(('Round '+str(game_state['week'])+' Fixtures') if LANG=='en'
            for hn,an in get_week_pairs(dn,game_state['week']):
                ht2=find_team(hn); at2=find_team(an)
                derby='🔥' if at2 and hn in at2.get('rivals',[]) else ''
                is_my=(t['name'] in (hn,an))
                label=f'{derby}{hn} vs {an}'
                if is_my:
                    with ui.row().classes('w-full items-center').style('backgroun
                        ui.label('⭐').classes('text-caption')
                        ui.label(label).classes('text-body2 text-white text-bold'
                else:
                    ui.label(label).classes('text-caption text-red' if derby else
        # 直近試合スコア
        last=game_state.get('last_match',{})
        if last:
            with ui.card().classes('w-full q-mb-sm'):
                result_col='text-positive' if (last['result'].split(' ')[1] if '
                ui.label(('Last: ' if LANG=='en' else '直近: ')+last['result']).cl
                ui.label(('Att:' if LANG=='en' else '観客')+f'{last["attendance"]:
        for dname in ['D1','D2','D3']:
            # 自チームのいるディビジョンのみデフォルト展開、他は折りたたみ
            is_my_div=(dname==dn)
            with ui.card().classes('w-full q-mb-sm'):
                ui.label(f'{"⭐" if is_my_div else ""}' + (f'Table [{dname}]' if
                for i,tm in enumerate(sorted_table(dname),1):
                    s=tm['season_stats']; is_self=(tm['name']==t['name'])
                    row_style='background:#1d4ed8;border-radius:6px;padding:2px 6
                    with ui.row().classes('w-full').style(row_style):
                        ui.label(f'{i}.').classes('text-caption').style('width:16
                        ui.label(tm['name']).classes('text-caption text-bold' if
                        ui.label(f'{s["pts"]}pt').classes('text-caption text-posi
                        ui.label(f'{s["w"]}-{s["d"]}-{s["l"]}').classes('text-cap
                        ui.label(f'GD{s["gd"]}').classes('text-caption text-grey'

def render_transfers():
    dashboard_box.clear()
    with dashboard_box:
        t=get_sel()
        if not t: ui.label(T('クラブ未選択')); return
        if not in_transfer_window():
            with ui.card().classes('w-full q-mb-sm'):
                ui.label(T('移籍ウィンドウ外label')).classes('text-h6')
                ui.label(T('移籍ウィンドウ説明')).classes('text-body2')
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('国内スカウト')).classes('text-h6')
```

```
    with ui.row():
        ui.button('Refresh' if LANG=='en' else '候補更新',on_click=create_s
        ui.button('Poach Youth' if LANG=='en' else 'ユース引き抜き',on_click
    if 'scout_pool' not in game_state: create_scout_pool()
    for p in game_state['scout_pool']:
        est=f'推定OVR {p.get("scout_ovr_min",p["overall"])}〜{p.get("scout
        grade=p.get('scout_grade','C')
        with ui.card().classes('w-full q-mb-xs').style('padding:8px;'):
            with ui.row().classes('w-full items-start').style('gap:8px;')
                # レーダーチャート
                with ui.column().classes('items-center').style('min-width
                    render_radar_chart(p,160)
                # 選手情報
                with ui.column().style('flex:1;'):
                    ui.label(f'{p["name"]}').classes('text-body2 text-bol
                    ui.label(f'{p["pos"]} {p["age"]}y {est} (Grade:{grade
                    ui.label(f'{fmt_m(p["value"])} {"🤝" if p.get("has_ag
                    # 属性テキスト表示
                    with ui.row().style('flex-wrap:wrap;gap:2px;'):
                        for k,v in p['attrs'].items():
                            col='text-positive' if v>=70 else('text-warni
                            ui.label(f'{ATTR_ICONS.get(k,k)}{ATTR_NAMES.g
                    ui.button(T('獲得'),on_click=lambda e,pid=p['id']:sigr
# 海外スカウト
# フリーエージェント市場
fa=game_state.get('fa_market',[])
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('FA市場')).classes('text-h6')
    ui.label(T('FA説明')).classes('text-caption text-grey')
    if not fa:
        ui.label(T('FA空')).classes('text-caption text-grey')
    else:
        for p in sorted(fa,key=lambda x:-x['overall']):
            _mt,_mc=mood_text(p.get('unhappiness',0))
            with ui.card().classes('w-full q-mb-xs').style('padding:8px;b
                with ui.row().classes('w-full items-center justify-betwee
                    ui.label(f'{p["name"]}').classes('text-body2 text-bol
                    ui.label(f'{p.get("fa_weeks",0)}w' if LANG=='en' else
                ui.label(f'{p["pos"]} {p["age"]}y OVR{p["overall"]}' if L
                fa_wage=p.get('fa_wage_demand',p.get('wage',10000))
                ui.label(f'Wage demand: {fmt_m(fa_wage)} / Sign fee: {fmt
                ui.label(f'From: {p.get("nationality","?")}' if LANG=='en
                # 属性サマリー
                top_attrs=sorted(p['attrs'].items(),key=lambda x:-x[1])[:
                attr_str=' '.join(f'{ATTR_ICONS.get(k,k)}{ATTR_NAMES.get(
                ui.label(attr_str).classes('text-caption')
                with ui.row():
```

```
                    ui.button(T('獲得'),on_click=lambda e,pid=p['id']:sigr
                    ui.button('Detail' if LANG=='en' else '詳細',on_click=

with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('海外スカウト')).classes('text-h6')
    os_data=game_state.get('overseas_scout')
    if os_data: ui.label(f'Scout in {os_data["country"]} ({os_data["weeks
    else:
        with ui.row():
            for country in COUNTRIES:
                ui.button(country,on_click=lambda e,c=country:dispatch_ov
        ui.label(T('海外費用')).classes('text-caption')
    # 海外スカウット候補
    op=game_state.get('overseas_pool',[])
    if op:
        ui.label(T('海外候補')).classes('text-subtitle2')
        for p in op:
            est=f'推定OVR {p.get("scout_ovr_min",p["overall"])}〜{p.get("s
            grade=p.get('scout_grade','C')
            grade_col='text-positive' if grade in ['A','B'] else('text-wa
            with ui.card().classes('w-full q-mb-xs').style('padding:8px;'
                with ui.row().classes('w-full items-start').style('gap:8p
                    with ui.column().classes('items-center').style('min-w
                        render_radar_chart(p,160)
                    with ui.column().style('flex:1;'):
                        ui.label(f'{p["name"]}').classes('text-body2 text
                        ui.label(f'{p["nationality"]} | {p["pos"]} | {p["
                        with ui.row().classes('items-center').style('gap:
                            ui.badge(grade,color='green' if grade in ['A'
                            ui.label(est).classes('text-caption')
                        ui.label(fmt_m(p['value'])).classes('text-caption
                        with ui.row().style('flex-wrap:wrap;gap:2px;'):
                            for k,v in p['attrs'].items():
                                col='text-positive' if v>=70 else('text-w
                                ui.label(f'{ATTR_ICONS.get(k,k)}{ATTR_NAM
                        ui.button(T('獲得'),on_click=lambda e,pid=p['id']:
# 国内オファー
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('国内移籍オファー')).classes('text-h6')
    if not game_state['transfer_offers']: ui.label(T('なし')).classes('tex
    else:
        for o in game_state['transfer_offers']:
            p2=player_by_id(o['player_id'])
            agent_note=f'🤝手数料{p2["agent_fee_pct"]}%' if p2 and p2.get(
            ui.label(f'{o["player_name"]} ← {o["buyer"]} / {fmt_m(o.get("
            with ui.row():
                ui.button(T('承諾btn'),on_click=lambda e,pid=o['player_id'
```

```
                ui.button(T('カウンター'),on_click=lambda e,pid=o['player_i
                ui.button(T('買戻条項'),on_click=lambda e,pid=o['player_id
                ui.button(T('拒否btn'),on_click=lambda e,pid=o['player_id'
        # 海外オファー
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('海外オファー')).classes('text-h6')
            if not game_state['foreign_offers']: ui.label(T('なし')).classes('text
            else:
                for o in game_state['foreign_offers']:
                    p2=player_by_id(o['player_id'])
                    if p2:
                        ui.label(f'{p2["name"]} ← {o["club"]} / {fmt_m(o["fee"])}
                        with ui.row():
                            ui.button(T('承諾btn'),on_click=lambda e,pid=o['player
                            ui.button(T('拒否btn'),on_click=lambda e,pid=o['player
        # レンタル
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('レンタル借用オファー')).classes('text-h6')
            if not game_state.get('loan_offers'): ui.label(T('なし')).classes('tex
            else:
                for o in game_state['loan_offers']:
                    ui.label(f'{o["player_name"]}({o["from_club"]}) {o["player_po
                    with ui.row():
                        ui.button(T('借りるbtn'),on_click=lambda e,pid=o['player_i
                        ui.button(T('断るbtn'),on_click=lambda e,pid=o['player_id'
        with ui.card().classes('w-full q-mb-sm'):
            ui.label(T('レンタル放出label')).classes('text-h6')
            for p in sorted([x for x in t['players'] if not x.get('loan_club')],k
                ui.label(f'{p["name"]} {p["pos"]} OVR{p["overall"]}').classes('te
                ui.button(T('放出btn2'),on_click=lambda e,pid=p['id']:send_on_loar
        # 選手比較
        if game_state.get('scout_pool') and t['players']:
            with ui.card().classes('w-full q-mb-sm'):
                ui.label(T('選手比較label')).classes('text-h6')
                best=max(t['players'],key=lambda p:p['overall'])
                cand=game_state['scout_pool'][0]
                with ui.row():
                    with ui.column():
                        ui.label(f'Mine: {best["name"]} OVR{best["overall"]}' if
                        for k,v in best['attrs'].items(): ui.label(f'{ATTR_ICONS.
                    with ui.column():
                        ui.label(f'Target: {cand["name"]} OVR{cand["overall"]}' i
                        for k,v in cand['attrs'].items():
                            diff=v-best['attrs'].get(k,0); col='text-positive' if
                            ui.label(f'{ATTR_ICONS.get(k,k)}{k}: {v} ({"+"+str(di

def render_management():
```

```
dashboard_box.clear()
with dashboard_box:
    t=get_sel()
    if not t: ui.label(T('クラブ未選択')); return
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('施設管理')).classes('text-h6')
        fac_names={'youth':'ユース','training':'トレーニング','medical':'メディカル
        for ft in FACILITY_TYPES:
            lv=game_state['facilities'][ft]; nm=fac_names[ft]
            ui.label(f'{nm}: Lv{lv}/5 ({'Upgrade:' if LANG=="en" else '強化費:
            if ft=='stadium': ui.button(T('スタジアム拡張btn'),on_click=expand_s
            elif ft=='youth': ui.button(T('ユース強化btn'),on_click=upgrade_you
            else: ui.button(('Upgrade ' if LANG=='en' else '')+nm+(' ' if LAN
        vip=t.get('vip_level',0)
        ui.label(f'VIP Area: Lv{vip}/3' if LANG=='en' else f'VIPエリア: Lv{vip
        if vip<3: ui.button(f'Upgrade VIP ({fmt_m(300000*(vip+1))})' if LANG=
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('スタッフ')).classes('text-h6')
        ui.button(T('スタッフ雇用'),on_click=hire_staff).props('color=primary')
        effs={'ヘッドコーチ':'強度+','スカウト':'スカウット質+','フィジオ':'怪我率-','
        for s in game_state['staff']:
            ui.label(f'{s["type"]} {effs.get(s["type"],"")} skill:{s["skill"]
    # 財務
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('財務サマリー')).classes('text-h6')
        f=game_state['season_finance']
        items=[('スポンサー','sponsor'),('観客','matchday'),('放映権','broadcast
        for lbl,k in items:
            v=f.get(k,0); col='text-positive' if v>0 else('text-negative' if
            ui.label(f'{lbl}: {fmt_m(v)}').classes(f'text-body2 {col}')
        dn=div_of(t['name']); cap=SALARY_CAP.get(dn,999999)
        total_wage=sum(p['wage'] for p in t['players'])
        ui.label(f'Wages: {fmt_m(total_wage)} / Cap: {fmt_m(cap)}' if LANG=='
    # 銀行ローン・保険
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('銀行ローン')).classes('text-h6')
        loans=game_state.get('bank_loans',[])
        if loans:
            for l in loans: ui.label(f'{l["remaining_weeks"]}w {int(l["intere
        else: ui.label(T('ローンなし')).classes('text-body2')
        with ui.row():
            for amt in [200000,500000,1000000]:
                ui.button(('Borrow ' if LANG=='en' else '')+fmt_m(amt)+(' ' i
    with ui.card().classes('w-full q-mb-sm'):
        ui.label(T('選手保険')).classes('text-h6')
        ins=game_state.get('injury_insurance_active',False)
        ui.label(f'Status: {"Active ($8k/wk)" if ins else "Inactive"}' if LAN
```

```
    if not ins: ui.button('Activate Insurance ($8k/wk)' if LANG=='en' els
# 予算配分
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('予算配分')).classes('text-h6')
    ui.label(f'Transfer:{fmt_m(t.get("budget_transfer",0))} Facility:{fmt
    with ui.row():
        ui.button(T('攻撃的配分'),on_click=lambda: set_budget_allocation(60
        ui.button(T('バランス配分'),on_click=lambda: set_budget_allocation(
        ui.button(T('育成重視配分'),on_click=lambda: set_budget_allocation(
# 国際大会
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('🏆コンチネンタルカップ')).classes('text-h6')
    cl=game_state.get('cc_data')
    if not cl:
        ui.label(T('CL今季なし')).classes('text-caption text-grey')
    elif cl.get('winner'):
        ui.label(T('コンカップ優勝').replace('{n}',cl['winner'])).classes('t
    else:
        in_cl=cl.get('in_cl',False)
        stage_name='グループステージ' if cl['stage']=='group' else '決勝トーナ
        ui.label(f'Current: {stage_name}' if LANG=='en' else f'現在: {stag
        if in_cl:
            ui.label(T('コンカップ出場中')).classes('text-caption text-posit
        else:
            ui.label(T('コンカップ出場なし')).classes('text-caption text-gre
        ui.label(T('CL進行').replace('{a}',str(CC_GROUP_START)).replace('{
        # グループ順位表
        if cl['stage']=='group':
            for gname,gteams in cl['groups'].items():
                ui.label(f'Group {gname}:' if LANG=='en' else f'グループ{gr
                for i,tm in enumerate(sorted(gteams,key=lambda x:(-x.get(
                    mark='⭐' if tm['name']==t['name'] else ''
                    ui.label(f'  {i}. {mark}{tm["name"]} {tm.get("cl_pts"
        elif cl['stage']=='knockout':
            ui.label(f'{len(cl["ko_teams"])} clubs left' if LANG=='en' el
            for tm in cl['ko_teams']:
                mark='⭐' if tm['name']==t['name'] else ''
                ui.label(f'  {mark}{tm["name"]}').classes('text-body2'+('
    # CL賞金テーブル
    with ui.expansion('CL賞金テーブル').classes('text-caption'):
        for stage,prize in [('出場ボーナス',CC_PRIZE['group']//2),('GS突破',
            ui.label(f'  {stage}: {fmt_m(prize)}').classes('text-caption
# セーブ
with ui.card().classes('w-full q-mb-sm'):
    ui.label(T('セーブロードlabel')).classes('text-h6')
    ui.button(T('セーブbtn2'),on_click=save_game).props('color=primary')
    ui.upload(label='JSONロード',on_upload=lambda e:load_game(e.content.re
```

```
def show_player(pid):
    p=player_by_id(pid)
    if not p: return
    with ui.dialog() as dlg, ui.card():
        ui.label(f'{p["name"]}').classes('text-h6')
        ui.label(f'{p["club"]} / {p["nationality"]} {"🏠 Local" if p.get("is_loca
        ui.label(f'{p["age"]}y | {p["pos"]} | OVR{p["overall"]} | POT{p["potentia
        tr=p.get('trait','なし')
        tr_desc=TRAIT_DESC.get(tr,'')
        ui.label(f'Style: {p["playstyle"]}' if LANG=='en' else f'スタイル: {p["play
        ui.label(f'Character: {p["personality"]} | Trait: {tr}' if LANG=='en' els
        if tr_desc: ui.label(f'  → {tr_desc}').classes('text-caption text-blue')
        ui.label(f'Growth:{p.get("growth_type","Normal")} | Form:{PLAYER_STATES.g
        _mt3,_mc3=mood_text(p.get('unhappiness',0)); _mo3,_moc3=morale_text(p.get
        _st3,_stc3=stamina_text(p.get('stamina',100))
        ui.label(f'Contract:{p["contract_years"]}y | Wage:{fmt_m(p["wage"])}' if
        with ui.row().style('gap:8px;flex-wrap:wrap;'):
            ui.label(_mt3).classes(f'text-caption {_mc3}')
            ui.label(_mo3).classes(f'text-caption {_moc3}')
            ui.label(_st3).classes(f'text-caption {_stc3}')
        sev=p.get('injury_severity','なし')
        if sev!='なし':
            _it,_ic=injury_text(p.get('injury_weeks',0),sev)
            ui.label(_it).classes(f'text-body2 {_ic}')
        if p.get('intl_weeks',0)>0: ui.label(('International duty '+str(p['intl_w
        if p.get('has_agent'): ui.label(f'Agent (fee:{p["agent_fee_pct"]}%)' if L
        if p.get('retiring'): ui.label(T('引退予定')).classes('text-red')
        if p.get('convert_weeks',0)>0: ui.label(f'🔄 Converting→{p.get("target_pc
        ui.label(f'{T("役割希望").replace("{n}",p.get("role_wish","Any" if LANG=="e
        # レーダーチャート
        render_radar_chart(p,200)
        # 属性リスト（アイコン+名前+数値）
        with ui.row().style('flex-wrap:wrap;gap:4px;'):
            for k,v in p['attrs'].items():
                col='text-positive' if v>=70 else('text-warning' if v>=55 else 't
                with ui.card().style('padding:4px 8px;min-width:80px;text-align:c
                    ui.label(f'{ATTR_ICONS.get(k,"")}{ATTR_NAMES.get(k,k)}').clas
                    ui.label(str(v)).classes(f'text-body1 text-bold {col}')
        st=p['stats']
        ui.label(f'Apps:{st["apps"]} G:{st["goals"]} A:{st["assists"]} MOM:{st["m
        if p.get('buyback_fee'): ui.label(f'Buyback:{fmt_m(p["buyback_fee"])}' if
        ui.button(T('閉じるbtn'),on_click=dlg.close)
    dlg.open()

def render_current_tab():
    {'dashboard':render_dashboard,'squad':render_squad,'matches':render_matches,'
```

```
def refresh_ui(): _render_header(); render_status(); render_current_tab()
def switch_tab(tab): nav_state['tab']=tab; refresh_ui()

# ============================================================
# 初期化・起動
# ============================================================
game_state=build_world('England'); init_stats()
for d in game_state['divisions']:
    for t in game_state['divisions'][d]:
        t['sponsor']=_gen_sponsor(t['country'],d)

ui.add_head_html('''
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale
<meta name="mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
<meta name="apple-mobile-web-app-title" content="The Gaffer">
<meta name="theme-color" content="#1e293b">
<link rel="manifest" href="/static/manifest.json">
<style>
*{box-sizing:border-box;}
body{background:#0f172a;color:#e5e7eb;-webkit-tap-highlight-color:transparent;mar
.q-card{background:#1e293b!important;color:#e5e7eb!important;border-radius:12px!i
.q-header{background:#0f172a!important;border-bottom:1px solid #334155!important;
.q-footer{background:#0f172a!important;border-top:1px solid #334155!important;pad
/* ボタンをスマホ最適化 */
.q-btn{min-height:40px!important;font-size:12px!important;padding:4px 8px!importa
/* テキストサイズ調整 */
.text-h5{font-size:16px!important;font-weight:700!important;}
.text-h6{font-size:14px!important;font-weight:600!important;margin-bottom:4px!imp
.text-body1{font-size:13px!important;}
.text-body2{font-size:12px!important;}
.text-caption{font-size:11px!important;color:#94a3b8!important;}
.text-subtitle1,.text-subtitle2{font-size:12px!important;font-weight:600!importan
/* スクロール */
.q-page{overflow-y:auto!important;-webkit-overflow-scrolling:touch!important;padd
/* 横スクロール防止 */
.q-page-container{overflow-x:hidden!important;}
/* circular_progressを小さく */
.q-circular-progress{font-size:10px!important;}
/* フッターナビ */
.footer-nav .q-btn{flex:1!important;font-size:10px!important;min-height:48px!impo
/* ヘッダー国選択 */
.country-btns .q-btn{font-size:9px!important;min-height:28px!important;padding:2p
/* カード内の行間 */
.q-card .q-separator{margin:6px 0!important;}
```

```
/* rowのwrap */
.q-gutter-sm{gap:4px!important;}
</style>
<script>
if('serviceWorker' in navigator){
  window.addEventListener('load',function(){
    navigator.serviceWorker.register('/static/sw.js');
  });
}
</script>
''')

# ヘッダー（コンパクト）
def _render_header():
    header_box.clear()
    with header_box:
        with ui.row().classes('w-full items-center justify-between q-px-sm').styl
            ui.label(APP_TITLE).classes('text-h5 text-white')
            # 言語切替（常時表示）
            with ui.row().classes('items-center').style('gap:4px;'):
                ui.button('JA',on_click=lambda: set_lang('ja')).props(
                    f'{"color=primary" if LANG=="ja" else "flat"} dense').style('
                ui.button('EN',on_click=lambda: set_lang('en')).props(
                    f'{"color=primary" if LANG=="en" else "flat"} dense').style('
            # クラブ未選択時のみ国選択を表示
            if not game_state.get('selected_club'):
                with ui.row().classes('country-btns items-center'):
                    for c in COUNTRIES:
                        ui.button(c[:3],on_click=lambda e,cc=c:new_world(cc)).pro
            else:
                has_offer=bool(game_state.get('manager_career_offers',[]))
                if has_offer:
                    ui.label('📩 Offer!' if LANG=='en' else '📩オファーあり').class

with ui.header().classes('q-pa-none'):
    header_box=ui.row().classes('w-full')
_render_header()

# メインコンテンツ（1カラム縦積み）
with ui.column().classes('w-full q-pa-xs'):
    status_box=ui.column().classes('w-full')
    dashboard_box=ui.column().classes('w-full')

# フッターナビ（固定）
with ui.footer().classes('footer-nav'):
    with ui.row().classes('w-full justify-around items-center'):
        for lbl,tab,icon in [(T('ホーム'),'dashboard','🏠'),(T('チーム'),'squad','👥
```

```
            ui.button(f'{icon}\n{lbl}',on_click=lambda e,t=tab:switch_tab(t)).pro

refresh_ui()
from nicegui import app as nicegui_app
import os
# staticフォルダを公開（manifest.json, sw.js用）
static_dir = os.path.join(os.path.dirname(__file__), 'static')
if os.path.exists(static_dir):
    nicegui_app.add_static_files('/static', static_dir)

ui.run(
    title=APP_TITLE,
    host='0.0.0.0',
    port=int(os.environ.get('PORT', 8080)),
    favicon='⚽',
    dark=True,
)
```